

TECHASSIST

ELOY RUIZ ZAPATA

PROYECTO INTERMODULAR DE
DESARROLLO DE APLICACIONES
MULTIPLATAFORMA (0492)

Desarrollo de aplicaciones
multiplataforma 2025-2026

I.E.S. Castillo de Luna

Agradecimientos

Deseo expresar mi más sincero agradecimiento a todas las personas que han hecho posible la realización de este proyecto de fin de grado. En primer lugar, a los docentes del I.E.S. Castillo de Luna, especialmente a mi tutor, Jesús Arcis Díaz, por su guía constante, paciencia y valiosos consejos técnicos que me han permitido orientar este desarrollo hacia un estándar de madurez profesional.

A mi compañero Sergio, con quien he compartido horas de código, debates filosóficos y un apoyo mutuo incalculable durante todo el ciclo formativo. Por último, y de manera muy especial, a mi madre, por su comprensión, aliento e infinito apoyo incondicional durante estos últimos dos años de duración del grado. A todos ellos, gracias.

Resumen

El presente proyecto de fin de grado describe el diseño, desarrollo e implementación de TechAssist, un Asistente Inteligente de Soporte a la Toma de Decisiones concebido como un Producto Mínimo Viable (MVP) para entornos de mantenimiento industrial. El objetivo principal de la aplicación es optimizar la asignación técnica de utillaje y guías operacionales a los operarios en planta, mitigando errores humanos y agilizando las tareas de mantenimiento correctivo y preventivo.

Para solventar las severas restricciones físicas y de conectividad habituales en las naves industriales, el software se ha diseñado bajo una estricta arquitectura offline-first. La aplicación se ejecuta de forma completamente autónoma en dispositivos móviles Android utilizando un motor de base de datos relacional embebido SQLite, el cual aloja de manera integrada tanto el censo de usuarios como la matriz de reglas fijas del sistema de recomendación basado en reglas.

Asimismo, para mitigar la brecha digital presente en las plantillas técnicas, el sistema incorpora un módulo evaluador interactivo denominado Minitest, el cual clasifica dinámicamente el nivel de competencias digitales del operario (Básico, Intermedio o Avanzado) y adapta en consecuencia la profundidad terminológica y el lenguaje de las recomendaciones visualizadas a través de un componente flotante BottomSheetDialog. El software ha sido desarrollado de forma nativa en Kotlin empleando componentes visuales avanzados de Material Design 3, garantizando un consumo controlado de memoria volátil y una alta robustez en condiciones de uso continuo en planta.

Palabras clave: Sistema de recomendación basado en reglas, Mantenimiento Industrial, Offline-First, SQLite, Kotlin, Material Design 3, Brecha Digital, Toma de Decisiones.

Abstract

This final degree project describes the design, development, and implementation of TechAssist, an Intelligent Decision-Support Assistant conceived as a Minimum Viable Product (MVP) for industrial maintenance environments. The main objective of the application is to optimize the technical assignment of tools and operational guides to shop floor operators, mitigating human errors and streamlining corrective and preventive maintenance tasks.

To overcome the severe physical and connectivity constraints common in industrial manufacturing facilities, the software has been designed under a strict offline-first architecture. The application runs completely autonomously on Android mobile devices using an embedded SQLite relational database engine, which seamlessly hosts both the user census and the rule-based recommendation system's fixed rules matrix .

Furthermore, to mitigate the digital divide among technical staff, the system incorporates an interactive evaluation module called Minitest. This module dynamically classifies the operator's digital competence level (Basic, Intermediate, or Advanced) and adapts accordingly the terminological depth and language of the recommendations displayed via a BottomSheetDialog floating component. The software has been natively developed in Kotlin utilizing advanced Material Design 3 visual components, ensuring a controlled volatile memory consumption and high robustness under continuous shift operations.

Keywords: Rule-Based Recommendation System, Industrial Maintenance, Offline-First, SQLite, Kotlin, Material Design 3, Digital Divide, Decision Support.

Índice

Agradecimientos.....	1
Resumen.....	2
Abstract.....	3
Índice.....	4
Índice de figuras.....	10
Índice de Tablas.....	11
1. Introducción general del proyecto.....	12
1.1 Contexto del proyecto.....	13
1.2 Identificación del problema.....	14
1.3 Alcance del proyecto.....	15
1.4 Objetivo general y objetivos específicos.....	16
1.5 Entregables esperados y estructura de la memoria.....	17
2. Marco teórico y estudio del sector.....	18
2.1 Marco conceptual y tecnológico.....	19
2.1.1 Agentes inteligentes y sistemas de recomendación.....	20
2.1.2 Aplicaciones de soporte a la toma de decisiones.....	21
2.2 Ecosistema TIC en mantenimiento industrial.....	22
2.3 Empresas y modelos organizativos del sector.....	23
2.4 Necesidades reales detectadas en el sector.....	24
2.5 Oportunidades y justificación del proyecto.....	25
2.6 Tipo de proyecto y solución propuesta.....	26
2.7 Caracterización general de la solución.....	27
3. Análisis de usuario y propuesta de valor.....	28
3.1 Definición de perfiles de usuario (Customer Persona).....	28
3.1.1 Persona 1: Técnico de mantenimiento.....	28
3.1.2 Persona 2: Supervisora de mantenimiento.....	29
3.1.3 Persona 3: Usuario de administración de aplicación.....	29
3.2 Tareas clave y problemas detectados (JTBD).....	30
3.3 Propuesta de valor.....	30
3.4 Curva de valor y diferenciación frente a alternativas.....	32
3.5 Modelo de Negocio.....	33
3.5.1 Justificación de los flujos de ingresos.....	33
3.5.2 Alianzas estratégicas y escalabilidad.....	34
3.5.3 Estructura de costes técnicos.....	34
4. Especificación de requisitos.....	36
4.1 Historias de usuario.....	37
Historia de Usuario -01- Recomendación de herramienta.....	37
Historia de Usuario -02- Explicación del criterio de recomendación.....	37
Historia de Usuario -03- Guía de primeros pasos.....	38
Historia de Usuario -04- Minitest de nivel digital.....	38
Historia de Usuario -05- Registro de procesos confusos.....	38
Historia de Usuario -06- Gestión de base de herramientas.....	39

4.2 Requisitos funcionales.....	39
4.2.1 Gestión de recomendación de herramientas.....	39
4.2.2 Explicación del criterio de recomendación.....	40
4.2.3 Guía inicial de uso.....	40
4.2.4 Evaluación del nivel digital.....	40
4.2.5 Registro y monitorización.....	41
4.2.6 Gestión administrativa de herramientas.....	41
4.3 Requisitos no funcionales.....	41
4.3.1 Rendimiento.....	42
4.3.2 Disponibilidad y resiliencia.....	42
4.3.3 Usabilidad.....	42
4.3.4 Seguridad y control de acceso.....	42
4.3.5 Mantenibilidad y extensibilidad.....	43
4.3.6 Compatibilidad y portabilidad.....	43
4.4 Diagrama de casos de uso.....	43
4.5 Tabla de priorización impacto-esfuerzo.....	46
4.6 Prototipado de aplicación.....	48
4.6.1 Acceso y Control de Perfiles (Login).....	48
4.6.2 Evaluación del Nivel Digital (Minitest).....	49
4.6.3 Interacción con el Agente y Recomendación.....	50
4.6.4 Gestión de Estados: Modo sin conexión (Offline).....	51
4.6.5 Panel de Gestión y Analítica (Supervisor).....	52
5. Diseño de la solución.....	53
5.1 Diseño de la experiencia de usuario (UX).....	54
5.1.1 Flujos principales de interacción.....	54
5.1.2 Escenarios de uso del agente.....	55
5.2 Diseño de la interfaz de usuario (UI).....	56
5.2.1 Identidad visual de la solución.....	56
5.2.2 Tipografía y Jerarquía Visual.....	57
5.2.3 Estética del módulo de acceso (Login).....	57
5.2.4 Estética de la Evaluación del Nivel Digital (Minitest).....	59
5.2.5 Estética de la Interacción Agente y Recomendación.....	60
5.2.6 Estética de la pantalla de Supervisión (Panel de Gestión).....	61
5.3 Arquitectura general del sistema.....	63
5.3.1 Arquitectura por capas.....	64
5.3.2 Componentes principales del sistema.....	65
5.4 Stack tecnológico.....	66
5.4.1 Entorno de Desarrollo (IDE): Android Studio.....	66
5.4.2 Lenguaje de Programación: Kotlin.....	66
5.4.3 Interfaz de Usuario: Android Views (XML).....	66
5.4.4 Arquitectura de Software: Patrón MVC (Modelo-Vista-Controlador).....	67
5.4.5 Persistencia de Datos: SQLite.....	68
5.4.6 Automatización y Orquestación: n8n (Low-Code Backend).....	69
5.5 Modelo de datos.....	70

5.5.1 Entidades principales del sistema.....	70
5.5.2 Definición de tablas.....	70
5.5.3 Relaciones entre entidades.....	72
5.5.4 Soporte al funcionamiento del agente.....	72
5.5.5 Consideraciones de diseño.....	73
5.5.6 Diagrama entidad-relación.....	73
6. Planificación y gestión del proyecto.....	74
6.1 Metodología de trabajo:.....	75
6.2 Product Backlog.....	76
6.2.1 Estructura del Product Backlog.....	76
6.2.2 Listado de elementos del backlog.....	77
6.2.3 Priorización del backlog.....	78
6.2.4 Relación con MVP.....	78
6.3 Definición del MVP.....	79
6.3.1 Funcionalidades incluidas en el MVP.....	79
6.3.2 Funcionalidades excluidas del MVP.....	80
6.3.3 Alcance del MVP.....	80
6.3.4 Criterios de validación del MVP.....	80
6.4 Sprints y fases del proyecto.....	81
6.4.1 Estructura de los sprints.....	81
6.4.2 Descripción de los sprints.....	82
6.4.3 Enfoque iterativo y validación continua.....	82
6.5 Recursos y estimación temporal.....	83
6.5.1 Recursos humanos.....	83
6.5.2 Recursos técnicos.....	83
6.5.3 Distribución del esfuerzo.....	83
6.6 Cronograma general.....	84
6.6.1 Estructura temporal del proyecto.....	84
6.6.2 Planificación por sprints.....	84
Sprint 1 – Base del sistema.....	85
Sprint 2 – Núcleo del agente.....	85
Sprint 3 – Interfaz y experiencia de usuario.....	85
Sprint 4 – Funcionalidades adicionales.....	85
Sprint 5 – Pruebas ajustes y redacción.....	85
6.6.3 Representación del cronograma.....	86
6.6.4 Hitos principales del proyecto.....	87
6.6.5 Consideraciones del cronograma.....	87
6.7 Criterios de aceptación.....	88
6.7.1 Objetivo de los criterios de aceptación.....	88
6.7.2 Criterios de aceptación del MVP.....	88
Funcionalidad: Recomendación de herramienta.....	88
Funcionalidad: Explicación del criterio de recomendación.....	88
Funcionalidad: Guía de primeros pasos.....	88
Funcionalidad: Minitest de nivel digital.....	89

Funcionalidad: Adaptación al nivel digital.....	89
Funcionalidad: Persistencia de datos.....	89
6.7.3 Validación de requisitos no funcionales.....	89
6.7.4 Procedimiento de verificación.....	89
6.7.5 Relación con la validación del proyecto.....	90
6.8 Gestión de cambios y desviaciones.....	90
6.8.1 Exclusión de la integración con n8n.....	90
6.8.2 Exclusión de la gamificación.....	91
6.8.3 Incorporación de funcionalidades no planificadas inicialmente.....	91
6.8.4 Ajuste de banco de preguntas del minitest.....	91
7. Gestión de riesgos, seguridad y cumplimiento.....	92
7.1 Identificación de riesgos.....	92
7.1.1 Riesgos técnicos.....	92
7.1.2 Riesgos operativos.....	92
7.1.3 Riesgos humanos.....	92
7.2 Evaluación de impacto y plan de mitigación.....	93
Riesgo 1: Recomendaciones incorrectas del sistema.....	93
Riesgo 2: Baja adopción por parte de los técnicos.....	93
Riesgo 3: Rendimiento insuficiente en dispositivos industriales.....	94
Riesgo 4: Errores en la base de datos o pérdida de información.....	94
Riesgo 5: Dependencia excesiva del sistema por parte del usuario.....	94
Riesgo 6: Limitaciones del enfoque basado en reglas.....	95
7.3 Seguridad y protección de datos.....	96
7.3.1 Enfoque de seguridad del sistema.....	96
7.3.2 Gestión de datos almacenados.....	96
7.3.3 Control de acceso y autenticación.....	97
7.3.4 Protección en la comunicación de datos.....	97
7.3.5 Integridad y consistencia de la información.....	97
7.3.6 Consideraciones de cumplimiento.....	98
7.3.7 Evaluación global de seguridad.....	98
7.4 Consideraciones éticas y accesibilidad.....	99
7.4.1 Consideraciones éticas.....	99
7.4.2 Accesibilidad y usabilidad inclusiva.....	100
7.4.3 Impacto en la adopción tecnológica.....	100
7.5 Licencias y sostenibilidad.....	101
7.5.1 Licencias de software.....	101
7.5.2 Sostenibilidad técnica.....	101
7.5.3 Sostenibilidad operativa.....	102
7.5.4 Sostenibilidad económica.....	102
7.5.5 Escalabilidad y evolución futura.....	103
8. Desarrollo del MVP.....	104
Repositorio de GitHub.....	104
8.1 Entorno de desarrollo.....	105
8.2 Descripción funcional del MVP.....	106

8.2.1 Minitest de nivel digital.....	107
8.2.2 Recomendador de herramientas.....	108
8.2.3 Guía de primeros pasos.....	109
8.2.4 Panel de supervisión.....	110
8.3 Casos de uso implementados.....	111
8.3.1 Caso de Uso 1: Registro, Evaluación e Inicialización del Perfil Digital del Técnico.....	112
8.3.2 Caso de Uso 2: Consulta de Tarea mediante Buscador y Despliegue de Recomendación.....	113
8.3.3 Caso de Uso 3: Control de Acceso Dinámico a la Interfaz de Gestión y Auditoría de Planta.....	115
8.4 Principales retos técnicos y soluciones encontradas.....	117
8.4.1 Reto 1: Desbordamiento visual y solapamiento del Label en el componente de búsqueda predictiva.....	117
8.4.2 Reto 2: Consistencia visual y pérdida del redondeo de esquinas en el TextInputEditText.....	118
8.4.3 Reto 3: Compatibilidad de subrutinas y persistencia síncrona en entornos restringidos bajo la API 24.....	118
9. Despliegue e instalación.....	119
9.1 Requisitos previos.....	120
9.1.1 Requisitos de Software y Entorno de Ingeniería.....	120
9.1.2 Requisitos de Hardware del Terminal Industrial.....	120
9.2 Proceso de despliegue.....	121
9.3 Instalación y puesta en marcha.....	123
9.3.1 Configuración de Seguridad en el Terminal de Destino.....	123
9.3.2 Transferencia y Ejecución del Binario (APK).....	123
9.3.3 Inicialización y Carga del Motor de Persistencia Local (SQLite).....	123
9.4 Manual básico de usuario.....	124
9.4.1 Flujo A: Autenticación, Registro y Evaluación Inicial (Técnico Nuevo).....	124
9.4.2 Flujo B: Consulta Operativa de Herramientas (Técnico Registrado).....	125
9.4.3 Flujo C: Auditoría y Control Analítico de Planta (Supervisor / Administrador).....	125
9.5 Documentación técnica de ejecución.....	126
9.5.1 Árbol de Directorios y Estructura del Proyecto.....	126
9.5.2 Gestión de Ciclos de Vida e Intercambio Seguro de Contextos.....	127
10. Pruebas, validación y control.....	130
10.1 Plan de pruebas.....	131
10.2 Resultados de pruebas.....	132
10.3 Gestión de incidencias.....	135
10.4 Validación con usuarios.....	136
10.5 Verificación del cumplimiento de requisitos.....	136
11. Conclusiones y trabajo futuro.....	137
11.1 Conclusiones generales.....	138
11.2 Valoración personal del proyecto.....	139
11.3 Mejoras y ampliaciones futuras.....	140
12. Glosario.....	141

13. Referencias bibliográficas.....	143
14. Anexos.....	144
Anexo A: Modelo de Entidad-Relación de la Base de Conocimiento Local (SQLite).....	144
Anexo B: Rúbrica de Validación del MVP e Indicadores Técnicos.....	145

Índice de figuras

Figura 2.3 -Estructura jerárquica y silos de información en la industria.....	23
Figura 2.4 - Impacto de la curva del olvido en la retención de procedimientos.....	24
Figura 3.3 - Curva de valor y matiz de competitividad de TechAssist.....	31
Figura 3.5 - Business Model Canvas para el despliegue de TechAssist.....	33
Figura 4.4 - Diagrama de casos de uso del sistema TechAssist.....	45
Figura 4.6.1 - Prototipo de la interfaz de acceso (Login).....	48
Figura 4.6.2 - Prototipo de la interfaz de evaluación digital (Minitest).....	49
Figura 4.6.3 - Prototipo del panel principal del Agente Recomendador.....	50
Figura 4.6.4 - Prototipado del agente recomendador y del panel de rendimiento del grupo.	51
Figura 4.6.5 - Prototipo del panel de analítica para supervisión.....	52
Figura 5.2.3 - Interfaz de alta fidelidad para el módulo de acceso.....	58
Figura 5.2.4 - Interfaz de alta fidelidad para evaluación competencial (Minitest).....	59
Figura 5.2.5 - Interfaz de alta fidelidad del Agente y adaptación visual ante pérdida de conexión.....	61
Figura 5.2.6 - Interfaz de alta fidelidad del panel de supervisión y gestión de estados de red..	62
Figura 5.4.4 - Esquema lógico del patrón arquitectónico Modelo-Vista-Controlador (MVC).	67
Figura 5.5.6 - Diagrama Entidad-Relación de la base de datos local del sistema.....	73
Figura 6.6.3 - Cronograma de desarrollo del proyecto estructurado por Sprints.....	86
Figura 8 - Repositorio remoto oficial del proyecto TechAssist en GitHub.....	104
Figura 8.2 - Interfaz de usuario final del módulo de acceso (Login).....	106
Figura 8.2.1 - Flujo interactivo del Minitest de nivel digital e interfaz de asignación de perfil....	107
Figura 8.2.2 - Interfaz de usuario principal de la aplicación.....	108
Figura 8.2.3 - Componente emergente interactivo con guía de primeros pasos y justif.....	110
Figura 8.2.4 - Interfaz de usuario final del panel de analítica y supervisión de grupo.....	111
Figura 8.3 - Código fuente en Kotlin de la inicialización de tablas en la base de datos SQLite local.....	111
Figura 8.3.1 - Código fuente en Kotlin del algoritmo de validación criptográfica y enrutamiento dinámico.....	113
Figura 8.3.2 - Código fuente en Kotlin de la inicialización y control reactivo del BottomSheetDialog de recomendación.....	115
Figura 8.3.3 - Código fuente en Kotlin del procesamiento de agregados relacionales y métricas para el cuadro de mando.....	117
Figura 9.2 - Configuración del script de construcción del módulo de la aplicación en build.gradle.kts.....	121
Figura 9.5.1 - Árbol de directorios de la aplicación y organización de paquetes en Android Studio.....	126
Figura 9.5.2a - Transferencia de parámetros de usuario mediante intents y funciones de alcance.....	127
Figura 9.5.2b - Invocación del motor de interferencia local y flujo transaccional asíncrono	128
Figura 9.5.2c - Consulta estructurada y control de recursos mediante operador .use.....	129
Figura 14 - Diagrama entidad-relación del esquema físico de la base de datos local.....	144
Figura 15 - Resultado de CPU Profiler - tiempo de respuesta del DatabaseHelper.....	145

Índice de Tablas

Tabla 4.4 - Trazabilidad entre Casos de Uso, HU y Requisitos Funcionales.....	44
Tabla 4.5 - Tabla de priorización impacto-esfuerzo.....	46
Tabla 5.2.1 - Especificaciones del Sistema de Diseño (UI).....	56
Tabla 5.2.2 - Escala tipográfica.....	57
Tabla 5.5.2a - Usuario.....	70
Tabla 5.5.2b - NivelDigital.....	71
Tabla 5.5.2c - Herramienta.....	71
Tabla 5.5.2d - Regla.....	71
Tabla 5.5.2e - Guía.....	71
Tabla 5.5.2f - Consulta.....	72
Tabla 6.2.2 - Listado de elementos del backlog.....	77
Tabla 6.4.1 - Estructura de los sprints.....	81
Tabla 6.6.1 - Estructura temporal del proyecto.....	84
Tabla 10.2a - Recomendación de herramienta.....	132
Tabla 10.2b - Explicación del criterio de recomendación.....	132
Tabla 10.2c - Guía de primeros pasos.....	133
Tabla 10.2d - Minitest de nivel digital.....	133
Tabla 10.2e - Control de acceso y panel de supervisión.....	133
Tabla 10.2f - Persistencia de datos.....	134
Tabla 10.5 - Verificación del cumplimiento de requisitos.....	136
Tabla Anexo B - Rúbrica de validación del MVP e indicadores Técnicos.....	145

1.Introducción general del proyecto

En el marco de la denominada Cuarta Revolución Industrial o Industria 4.0, las plantas de fabricación y los talleres de mantenimiento han experimentado un proceso acelerado de digitalización. La introducción de sistemas hiperconectados, sensores del Internet de las Cosas (IoT) y sistemas centralizados de gestión de activos como los softwares de Gestión de Mantenimiento Asistida por Ordenador (GMAO/CMMS) o de Planificación de Recursos Empresariales (ERP) ha transformado radicalmente la forma en que se monitoriza y diagnostica la maquinaria pesada. Sin embargo, este despliegue masivo de infraestructura tecnológica y flujos automatizados de datos contrasta frecuentemente con la realidad operativa del eslabón más crítico de la cadena de valor: el operario de mantenimiento a pie de máquina.

El mantenimiento industrial moderno exige una respuesta inmediata, síncrona y de máxima precisión ante averías correctivas o paradas de línea imprevistas, escenarios donde cada minuto de inactividad se traduce en severas pérdidas financieras y operativas para la compañía. En este contexto hostil, los técnicos de planta deben enfrentarse a diario a manuales de ingeniería extremadamente densos, catálogos de utillaje heterogéneos y estrictas normativas de seguridad laboral, todo ello en entornos dinámicos, de alta fatiga y ruidosos. TechAssist nace en el seno de esta coyuntura industrial como una solución de movilidad nativa bajo el ecosistema Android, diseñada específicamente para actuar como un compañero digital y asistente inteligente que asiste, guía y valida las decisiones del trabajador en el punto exacto de la intervención técnica.

1.1 Contexto del proyecto

En el marco de la denominada Cuarta Revolución Industrial o Industria 4.0, las plantas de fabricación y los talleres de mantenimiento han experimentado un proceso acelerado de digitalización. La introducción de sistemas hiperconectados, sensores del Internet de las Cosas (IoT) y sistemas centralizados de gestión de activos (ERP o CMMS) ha transformado la forma en que se monitoriza la maquinaria pesada. Sin embargo, este despliegue de infraestructura tecnológica contrasta frecuentemente con la realidad operativa del eslabón más crítico de la cadena de valor: el operario de mantenimiento a pie de máquina.

El mantenimiento industrial moderno exige una respuesta inmediata y precisa ante averías correctivas o paradas de línea imprevistas, donde cada minuto de inactividad se traduce en severas pérdidas financieras para la compañía. En este contexto, los técnicos de planta deben enfrentarse a diario a manuales de ingeniería extremadamente densos, catálogos de utillaje heterogéneos y estrictas normativas de seguridad laboral en entornos dinámicos y ruidosos. TechAssist nace en el seno de esta coyuntura industrial como una solución de movilidad nativa, diseñada específicamente para actuar como un compañero digital síncrono que asiste, guía y valida las decisiones del trabajador en el punto exacto de mantenimiento.

1.2 Identificación del problema

A través del análisis preliminar realizado en diferentes entornos productivos de planta, se han detectado tres problemáticas troncales que merman la eficiencia y la seguridad de las intervenciones técnicas:

- 1. La brecha digital y heterogeneidad de las plantillas:** El personal técnico que integra los departamentos de mantenimiento suele ser demográfica y tecnológicamente muy diverso. Conviven operarios seniors con una experiencia empírica sobresaliente de maquinaria pero con competencias digitales básicas, junto a técnicos juniors habituados al entorno móvil pero con menor recorrido en procesos específicos de utillaje. Los sistemas tradicionales de información corporativa ignoran esta realidad, mostrando interfaces genéricas y densas que frustran al usuario o inducen a fallos de interpretación.
- 2. Entornos de baja conectividad e interferencias:** Las naves industriales están físicamente construidas con estructuras metálicas pesadas, hormigón armado y blindajes electromagnéticos generados por los propios motores de los activos. Esta configuración física provoca constantes zonas de sombra o caídas absolutas de cobertura Wi-Fi y redes móviles. Confiar el soporte técnico a aplicaciones dependientes de la nube (*cloud-only*) o arquitecturas web tradicionales inhabilita la herramienta en los puntos más profundos e inaccesibles de la planta, justo donde la asistencia es más urgente.
- 3. Falta de trazabilidad pasiva en las consultas:** La línea de supervisión y los ingenieros de procesos carecen en muchas ocasiones de un registro fidedigno de qué dudas o necesidades operacionales experimentan los técnicos durante sus turnos de trabajo. Las consultas informales o los errores en la selección de herramientas quedan ocultos a menos que provoquen un accidente o una rotura de utillaje, impidiendo el diseño de planes de formación interna basados en datos objetivos de planta.

1.3 Alcance del proyecto

El alcance de este proyecto intermodular se delimita formalmente en la concepción, diseño e implementación física de un Producto Mínimo Viable (MVP) operativo para terminales Android. TechAssist no pretende sustituir a los grandes sistemas centralizados de la empresa, sino integrarse en el extremo final de la operación mediante las siguientes fronteras funcionales:

- **Inclusión del Motor de Inferencia Experto Local:** La aplicación debe contener un catálogo precargado de reglas de negocio que crucen de forma síncrona la categoría de la tarea (eléctrica, mecánica, neumática, etc.) con el nivel técnico detectado, resolviendo la recomendación en milisegundos sin llamadas de red.
- **Módulo Interactivo de Clasificación:** Se acota el diseño de un *Minitest* inicial obligatorio para nuevos usuarios que determina el perfil del técnico. El sistema guardará esta información en una persistencia protegida para modular las vistas subsiguientes.
- **Componente de Interacción Guía-Checklist:** La visualización de la asistencia técnica se restringe a un formato de tarjeta intuitivo con un listado secuencial de tres pasos de seguridad obligatorios, dotado de casillas de verificación interactivas para asegurar que el operario asimila la pauta antes de cerrar la pantalla.
- **Exclusión de conectividad activa en el MVP:** Toda transferencia síncrona con backends remotos o flujos automatizados complejos con plataformas de automatización como *n8n* quedan formalmente clasificados como líneas de ampliación futuras, garantizando la estabilidad absoluta offline del binario actual.

1.4 Objetivo general y objetivos específicos

- **Objetivo general:**

Diseñar y desarrollar una aplicación móvil nativa para el sistema operativo Android que actúe como un asistente inteligente de soporte a la toma de decisiones en el mantenimiento industrial, garantizando una autonomía absoluta sin conexión a internet (offline-first) y adaptando dinámicamente su interfaz para mitigar la brecha digital de la plantilla operativa.

- **Objetivos específicos:**

- **OE-01:** Implementar una arquitectura de datos relacional local altamente eficiente utilizando el motor SQLite nativo de Android, asegurando la atomicidad de las consultas y la protección criptográfica de los datos sensibles mediante hashing SHA-256.
- **OE-02:** Desarrollar un sistema de inferencia estático basado en reglas parametrizadas que vincule categorías técnicas y perfiles de usuario para determinar el utillaje óptimo de intervención.
- **OE-03:** Diseñar una interfaz de usuario fluida, limpia y accesible siguiendo las directrices de Material Design 3, optimizando los controles de navegación táctil para entornos industriales hostiles donde se requiere el uso de equipos de protección individual (EPIs).
- **OE-04:** Configurar un módulo de auditoría pasiva local accesible únicamente para perfiles con rol de Supervisor, que extraiga datos históricos directamente de las tuplas de SQLite para analizar el comportamiento operacional de los operarios.
- **OE-05:** Validar el rendimiento del software mediante un plan de pruebas unitarias y de caja negra, certificando que el binario final (APK) cumpla estrictamente con los requisitos no funcionales de consumo eficiente de hardware (<150 MB de memoria RAM).

1.5 Entregables esperados y estructura de la memoria

Al término del proyecto, los entregables aportados a la ingeniería del centro corresponden al código fuente estructurado en Kotlin DSL, el archivo ejecutable final compilado (*TechAssist-v1.0-release.apk*) listo para su instalación masiva, y la presente memoria técnica descriptiva.

El documento se estructura formalmente en capítulos secuenciales para guiar al lector a lo largo de la ingeniería del proyecto:

- **Capítulo 2 y 3:** Sientan las bases conceptuales del proyecto mediante el análisis del sector del mantenimiento industrial, los perfiles de usuario y la propuesta de valor de la solución.
- **Capítulo 4 y 5:** Definen la especificación formal de requisitos funcionales y no funcionales, los casos de uso y el diseño técnico de la solución, incluyendo la arquitectura MVC, el modelo de datos y las decisiones de UX/UI.
- **Capítulo 6 y 7:** Abordan la planificación del proyecto mediante metodología ágil, el Product Backlog, la definición del MVP y la gestión de riesgos, seguridad y cumplimiento normativo.
- **Capítulo 8 y 9:** El capítulo 8 describe la implementación técnica del MVP con los casos de uso implementados y los retos técnicos resueltos, mientras que el capítulo 9 cubre el despliegue, instalación y manual de usuario.
- **Capítulo 10:** Valida el sistema mediante el plan de pruebas, resultados, gestión de incidencias y verificación del cumplimiento de requisitos.
- **Capítulo 11:** Recoge las conclusiones generales, la valoración personal del proyecto y la hoja de ruta para la evolución tecnológica del sistema.

2. Marco teórico y estudio del sector

Este capítulo presenta el marco teórico y el análisis del sector en el que se enmarca el proyecto, con el objetivo de contextualizar la solución propuesta desde una perspectiva conceptual, tecnológica y profesional. A través del estudio del ecosistema del mantenimiento industrial y de las tecnologías asociadas, se justifica la necesidad de desarrollar una herramienta de apoyo que facilite la adopción y el uso eficiente de soluciones digitales en este ámbito.

A lo largo del capítulo se introducen los conceptos clave que fundamentan el proyecto, se analizan las características del sector del mantenimiento industrial y se identifican las oportunidades que respaldan la propuesta de una solución orientada a la asistencia en la toma de decisiones técnicas.

2.1 Marco conceptual y tecnológico

El desarrollo de soluciones software dirigidas a entornos industriales requiere una base conceptual sólida que integre aspectos tecnológicos, organizativos y humanos. En el ámbito del mantenimiento industrial, la transformación digital ha impulsado la incorporación de múltiples herramientas informáticas destinadas a la gestión de activos, la planificación de tareas y el control de procesos.

Sin embargo, esta proliferación de soluciones digitales ha incrementado la complejidad operativa para los técnicos, especialmente en contextos donde conviven distintos perfiles profesionales con niveles heterogéneos de experiencia y competencia digital. En este escenario, adquieren relevancia los sistemas capaces de asistir al usuario en la selección y el uso adecuado de dichas herramientas

El presente proyecto se apoya en conceptos como los agentes inteligentes, los sistemas de recomendación y las aplicaciones de soporte a la toma de decisiones, entendidos como mecanismos que permiten ofrecer orientación contextualizada al usuario, reducir la carga cognitiva y minimizar errores derivados del uso inadecuado de las herramientas disponibles.

2.1.1 Agentes inteligentes y sistemas de recomendación

En el marco de este proyecto, un agente inteligente se define como una entidad software capaz de procesar información procedente del entorno (entradas de usuario, contexto de la tarea, etc.) con el fin de generar respuestas orientadas al cumplimiento de un objetivo específico.

A diferencia de los asistentes digitales de propósito general, el agente propuesto se especializa en la intermediación digital dentro del entorno del mantenimiento industrial. Su funcionamiento se basa en una lógica de un sistema de recomendación basado en reglas sustentada en reglas, donde las recomendaciones no tienen carácter probabilístico, sino normativo y procedimental.

El agente propuesto se apoya en un modelo interno del entorno digital y del contexto operativo del usuario, lo que permite extender el enfoque reactivo clásico hacia un agente reactivo basado en modelos. Este enfoque resulta adecuado para tareas analíticas propias del mantenimiento industrial, como la clasificación de solicitudes, la evaluación del contexto, la monitorización del estado del sistema y el diagnóstico de necesidades operativas. No obstante, dado que la selección de la herramienta adecuada depende de la meta actual del operario, el sistema incorpora elementos deliberativos para garantizar una recomendación coherente con los objetivos de la tarea.

2.1.2 Aplicaciones de soporte a la toma de decisiones

Las aplicaciones de soporte a la toma de decisiones son sistemas diseñados para ayudar a los usuarios a evaluar opciones, seleccionar acciones y resolver problemas en contextos complejos. En entornos industriales, estas aplicaciones no sustituyen la experiencia del técnico, sino que actúan como apoyo para mejorar la calidad y coherencia de las decisiones operativas.

En el ámbito del mantenimiento industrial, este tipo de soluciones se emplean para priorizar tareas, sugerir procedimientos o facilitar el acceso a información relevante en el momento adecuado. Su valor reside en la capacidad de reducir errores, optimizar tiempos de intervención y estandarizar procesos.

TechAssist se encuadra dentro de esta categoría ya que su objetivo principal es asistir al técnico en la elección de la herramienta correcta para cada situación, actuando como un apoyo experto que orienta la toma de decisiones sin inferir directamente en la ejecución de las tareas de mantenimiento.

2.2 Ecosistema TIC en mantenimiento industrial

El ecosistema TIC del mantenimiento industrial está compuesto por una amplia variedad de herramientas y sistemas, entre los que se incluyen aplicaciones móviles, plataformas de gestión de mantenimiento (GMAO), sistemas de monitorización, repositorios documentales y paneles de control industrial.

Aunque estas soluciones aportan un alto valor funcional, su coexistencia genera un entorno fragmentado en el que el técnico debe identificar qué herramienta utilizar en cada momento. Esta fragmentación puede provocar ineficiencias, errores de uso y resistencia a la adopción tecnológica, especialmente cuando no existe una guía clara o formación adecuada.

En este contexto, resulta necesario introducir mecanismos de orientación que ayuden a los usuarios a interactuar de forma eficaz con el ecosistema digital existente, sin necesidad de desarrollar nuevas plataformas de gestión o sustituir los sistemas ya implantados.

2.3 Empresas y modelos organizativos del sector

Las organizaciones del sector industrial, desde grandes plantas de producción hasta empresas de logística, presentan estructuras donde la digitalización avanza a ritmos distintos según el departamento. En los modelos organizativos actuales, la toma de decisiones estratégicas está altamente digitalizada, pero existe una desconexión en la base operativa.

El modelo de trabajo suele ser jerárquico, donde el conocimiento técnico-mecánico reside en los operarios de campo, mientras que el control de datos recae en departamentos de IT o supervisión. Esta estructura genera “islas de conocimiento” donde la introducción de nuevas herramientas a menudo no va acompañada de una adaptación cultural, provocando que los procedimientos digitales se perciban como una carga administrativa adicional en lugar de una ayuda operativa.

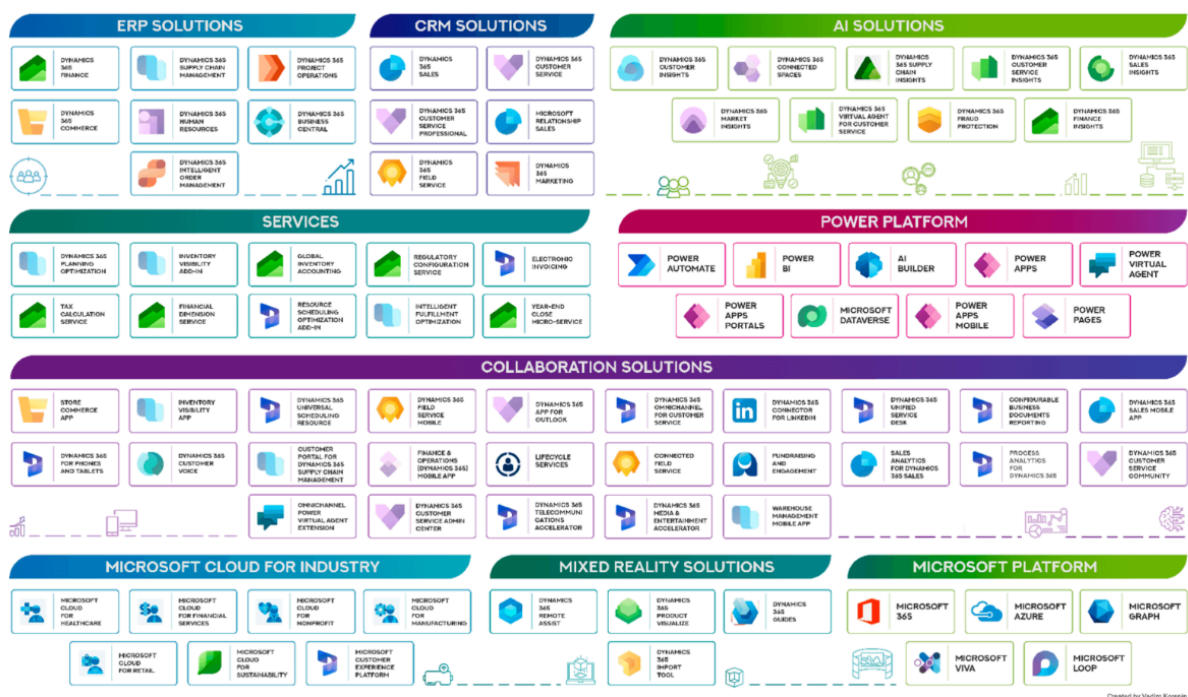


Figura 2.3 -Estructura jerárquica y silos de información en la industria.

2.4 Necesidades reales detectadas en el sector

A partir del análisis del entorno industrial y los perfiles profesionales implicados, se han identificado las siguientes necesidades críticas que fundamentan este proyecto:

- **Mitigación de la brecha digital generacional:** Existe un volumen importante de técnicos con alta cualificación mecánica que presentan dificultades ante interfaces de usuario complejas o cambios frecuentes en el software corporativo.
- **Reducción de la dependencia del supervisor:** La falta de autonomía en el manejo de herramientas genera cuellos de botella, ya que los técnicos deben consultar dudas básicas sobre el uso de aplicaciones, deteniendo el **flujo de trabajo**.
- **Gestión de la curva del olvido:** El mantenimiento industrial moderno comprende tareas críticas pero de baja periodicidad (calibraciones semestrales, paradas anuales, etc.). Desde una perspectiva neurocientífica, estudios sobre el aprendizaje basados en la célebre Curva del Olvido de Ebbinghaus demuestran que el cerebro humano llega a perder hasta el 50% de la información adquirida en solo 24 horas, alcanzando un 90% de olvido a los pocos días si no existen refuerzos. Al no interactuar diariamente con los menús digitales o el utillaje específico, el operario olvida el estándar. La ausencia de un asistente interactivo que ofrezca microaprendizaje en el momento exacto de la tarea (just-in-time learning) deriva de forma directa en errores de reporte, selección inadecuada de herramientas o la omisión de pasos críticos de seguridad en planta.



Figura 2.4 - Impacto de la curva del olvido en la retención de procedimientos

2.5 Oportunidades y justificación del proyecto

La digitalización de la industria (Industria 4.0) ofrece una oportunidad estratégica para implementar soluciones de “capa intermedia”. Este proyecto está justificado no como una herramienta de gestión adicional, sino como un facilitador de eficiencia.

La oportunidad reside en superar el modelo de formación tradicional, que suele ser a posteriori, puntual y descontextualizada, para transformarlo en un apoyo preventivo/proactivo basado en la asistencia en el momento de la tarea. Al optimizar la interacción del técnico con el software, la empresa no solo mejora la efectividad del aprendizaje, sino que garantiza la integridad de los datos introducidos en sus sistemas centrales. Esto permite una mejor analítica de negocio y una reducción de los tiempos muertos por errores de procedimiento digital.

2.6 Tipo de proyecto y solución propuesta

El proyecto se define como una aplicación de asistencia operativa y capacitación digital, desarrollada de forma nativa para Android utilizando el entorno Android Studio y el lenguaje de programación **Kotlin**. El desarrollo se basa en el sistema de **Android Views (XML)** para la construcción de la interfaz de usuario, empleando componentes de Material Design para garantizar una experiencia de uso profesional y estandarizada.

Es fundamental subrayar que esta solución no constituye un sistema **GMAO** (Gestión de mantenimiento Asistida por Ordenador). Mientras que un **GMAO** tiene un enfoque administrativo centrado en la gestión de activos y la base de datos de mantenimiento, TechAssist se posiciona como una capa de asistencia proactiva. Su objetivo no es la gestión de la orden de trabajo en sí, sino asegurar que el técnico identifique la herramienta correcta y comprenda su funcionamiento mediante un asistente inteligente. La aplicación actúa como puente entre el operario y el ecosistema TIC de la empresa, integrando misiones de entrenamiento y guías interactivas sin interferir en los procesos de escritura de datos de los sistemas de producción.

2.7 Caracterización general de la solución

TechAssist se define por ser una herramienta ligera, pedagógica y centrada en el usuario, con las siguientes características técnicas y funcionales:

1. **Patrón de Arquitectura MVC (Modelo-Vista-Controlador):** Se implementa este patrón para organizar el código de forma modular. El **Modelo** gestiona la base de datos de herramientas y lógica del agente; la **Vista** se define mediante Layouts XML; y el **Controlador** (clases Kotlin) actúa como intermediario, procesando las entradas del operario y actualizando la interfaz en tiempo real según las directrices del agente.
2. **Interfaz Basada en Vistas (Android Views):** El uso de layouts XML permite un control preciso sobre la jerarquía de vistas, asegurando que la aplicación sea fluida y accesible incluso en dispositivos industriales con recursos limitados.
3. **Agente de Asistencia No Intrusivo:** La solución funciona como un compañero de apoyo. El agente traduce las dudas del técnico en instrucciones visuales y pasos a seguir, reduciendo la curva de aprendizaje de las herramientas propietarias de la empresa.
4. **Detección de Necesidades de Refuerzo:** A través de un sistema de monitorización de eventos de usuario, la aplicación identifica patrones de duda (como la superación del umbral de pasos en la “regla de los 3 clics”). Esto permite que el sistema detecte lagunas de conocimiento y active automáticamente recordatorios basados en la curva del olvido.
5. **Gamificación Operativa:** Se incorporan elementos de motivación como el progreso por niveles y la consecución de insignias, transformando la formación continua en un proceso dinámico y gratificante para el empleado.

3. Análisis de usuario y propuesta de valor

Este capítulo tiene como finalidad identificar a los usuarios principales del sistema, comprender sus necesidades reales y definir la propuesta de valor que ofrece la aplicación. A través del análisis de perfiles (**Customer Persona**), la identificación de tareas clave (**Jobs To Be Done**) y la comparación frente a alternativas existentes, se fundamenta la pertinencia de la solución propuesta.

3.1 Definición de perfiles de usuario (Customer Persona)

Para diseñar una solución realmente útil, es imprescindible comprender el contexto, motivaciones y limitaciones de los usuarios. A continuación, se presentan los perfiles principalmente identificados.

3.1.1 Persona 1: Técnico de mantenimiento

Nombre: Juan Ramos.

Edad: 35 años.

Rol: Técnico de mantenimiento industrial.

Contexto profesional:

Trabaja en planta realizando intervenciones correctivas y preventivas. Posee alta competencia técnica mecánica y eléctrica, pero nivel medio-bajo en competencias digitales.

Metas:

- Identificar rápidamente la herramienta correcta para cada tarea.
- Reducir errores en registros digitales.
- Evitar depender del supervisor para operaciones básicas.
- Resolver incidencias urgentes sin bloqueos administrativos.

Frustraciones:

- No saber qué aplicación utilizar en cada situación.
- Interfaces poco intuitivas.
- Olvidar procedimientos poco frecuentes.
- Sensación de que las herramientas complican más el trabajo.

Tareas clave (JTBD):

Cuando tiene que registrar una intervención o reportar una incidencia, quiere saber qué aplicación utilizar y cómo iniciar el proceso para evitar errores y pérdidas de tiempo.

Barreras:

- Desconfianza hacia nuevas herramientas.
- Miedo a cometer errores digitales que afecten a los datos.

Criterios de éxito

- Finalizar tareas sin ayuda externa.

- No cometer errores en la selección de herramientas.
- Reducir el tiempo de inicio de tareas digitales.

3.1.2 Persona 2: Supervisora de mantenimiento

Nombre: Marta Vázquez.

Edad: 50 años.

Rol: Supervisora de mantenimiento.

Contexto profesional:

Responsable del equipo técnico y de la calidad de los datos introducidos en los sistemas corporativos.

Metas:

- Reducir errores de selección de herramientas.
- Incrementar la autonomía del equipo.
- Disminuir tiempos muertos por dudas operativas.
- Detectar procesos que generan mayor confusión.

Frustraciones:

- Alta heterogeneidad en alfabetización digital.
- Consultas repetitivas sobre procesos básicos.
- Falta de visibilidad sobre qué tareas generan más errores.

Tareas clave (JTBD):

Cuando detecta errores en registros digitales, necesita identificar qué procesos están generando dudas para reforzar la formación del equipo.

Criterios de éxito

- Reducción medible de errores digitales.
- Menos interrupciones por consultas básicas.
- Mayor autonomía del equipo técnico.

3.1.3 Persona 3: Usuario de administración de aplicación

Rol: Usuario interno encargado de configurar el sistema.

Metas:

- Mantener actualizada la base de herramientas.
- Ajustar reglas del agente.
- Monitorizar patrones de uso.

Criterios de éxito

- Sistema estable.

- Facilidad de mantenimiento.
- Adaptabilidad ante cambios organizativos.

3.2 Tareas clave y problemas detectados (JTBD)

A partir del análisis de los perfiles, se identifican las siguientes tareas clave:

- 1) Seleccionar la herramienta correcta para una tarea específica.
- 2) Iniciar correctamente un proceso digital poco frecuente.
- 3) Confirmar que la herramienta utilizada es adecuada.
- 4) Reducir el tiempo de búsqueda entre aplicaciones.
- 5) Identificar procesos que generan confusión recurrente.

El problema central detectado es la dificultad para identificar rápidamente qué herramienta utilizar en cada contexto operativo. Esta situación genera:

- Pérdida de tiempo.
- Errores en los registros.
- Dependencia del supervisor.
- Rechazo progresivo a la digitalización.

3.3 Propuesta de valor

La aplicación propone un agente de asistencia operativa que actúa como intermediario entre el técnico y el ecosistema TIC de la empresa.

Valor para el técnico:

- Reducción de la incertidumbre digital.
- Orientación inmediata en el momento de la tarea.
- Mayor autonomía.
- Disminución del estrés en situaciones urgentes.

Valor para la supervisión:

- Mejora en la calidad de datos.
- Reducción de consultas básicas.
- Identificación de patrones de duda.
- Optimización de refuerzos formativos.

La solución no sustituye los sistemas existentes, sino que facilita su uso mediante recomendaciones normativas basadas en reglas.

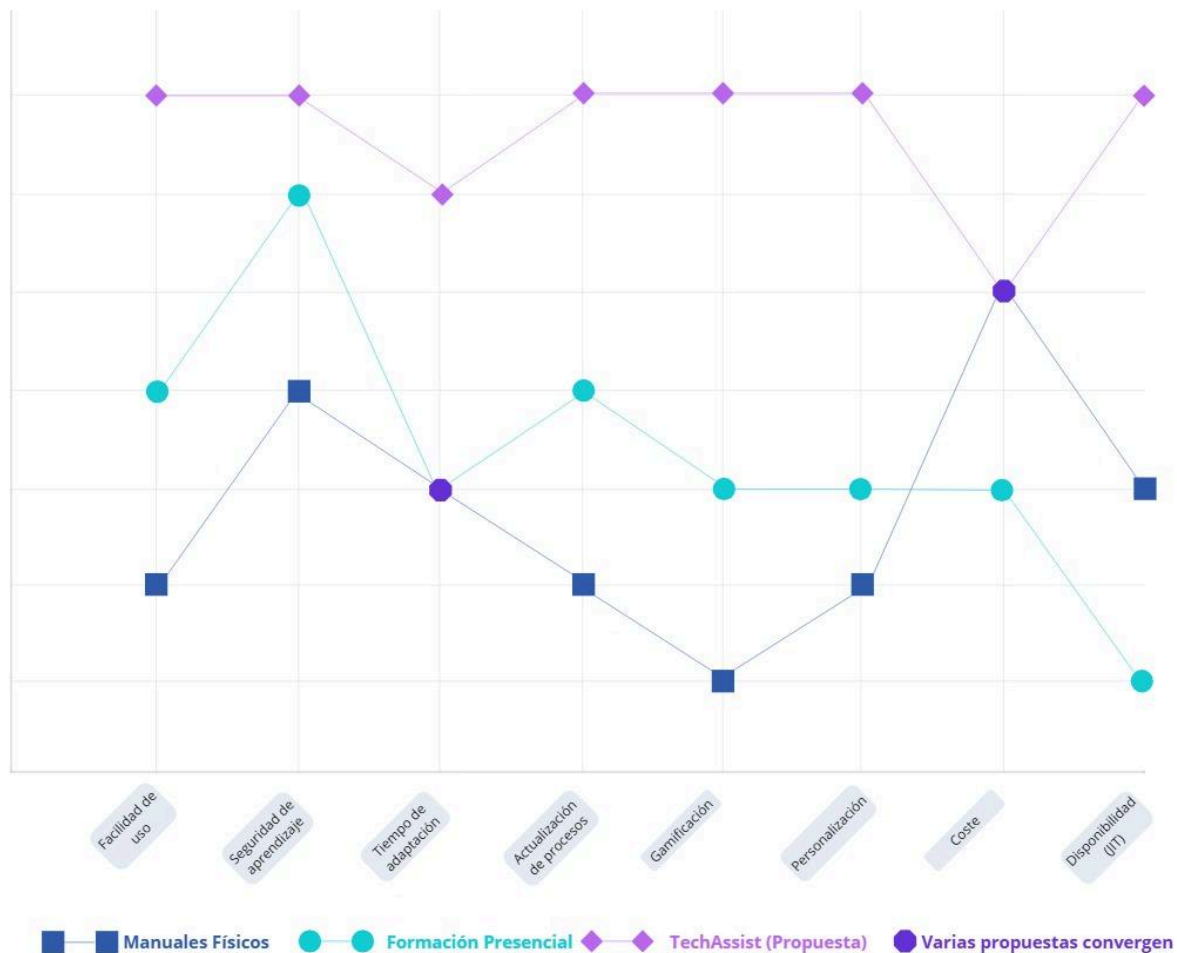


Figura 3.3 - Curva de valor y matiz de competitividad de TechAssist

La **figura 3.3** representa la Curva de Valor de **TechAssist** en comparación con las soluciones tradicionales del sector (manuales físicos y sistemas ERP/GMAO complejos). De este análisis se extraen las siguientes conclusiones estratégicas:

1. **Eliminación de la complejidad:** Mientras que los sistemas actuales requieren largas formaciones, TechAssist reduce la curva de aprendizaje al mínimo mediante una interfaz asistida.
2. **Foco en la movilidad real:** A diferencia de los manuales en PDF o software de escritorio, nuestra solución destaca en **portabilidad y funcionamiento 100% offline**, requisitos críticos en entornos industriales con mala cobertura.
3. **Personalización Dinámica:** Es el factor donde TechAssist crea un "Océano Azul". Ninguna herramienta actual adapta el nivel de detalle de las instrucciones basándose en un **minitest de nivel digital** previo del operario.
4. **Inmediatez:** Se prioriza la velocidad de recomendación sobre la profundidad de funciones administrativas, atacando directamente el problema de la "parálisis por análisis" del técnico.

3.4 Curva de valor y diferenciación frente a alternativas

Actualmente, las alternativas disponibles se centran en:

- Formación tradicional puntual.
- Manuales PDF o documentación técnica.
- Soporte informal de supervisor.
- GMAOs con interfaces complejas.

Estas soluciones convergen en un mismo patrón: requieren que el usuario recuerde información previamente aprendida o que navegue por sistemas complejos.

TechAssist introduce un cambio de enfoque al ofrecer:

- Asistencia contextual en tiempo real.
- Recomendación directa de herramientas.
- Explicación del criterio utilizado.
- Detección automática de procesos confusos.

La diferenciación radica en su papel como capa intermedia ligera, sin integración directa en sistemas de producción.

Solución	Tipo de apoyo	Ventajas	Limitaciones
Formación tradicional	Formación inicial o cursos puntuales	Permite adquirir conocimientos estructurados	No se produce en el momento de la tarea; alta curva de olvido
Manuales y documentación	Consulta de procedimientos	Información detallada disponible	Requiere tiempo de búsqueda y lectura
Soporte del supervisor	Ayuda directa entre trabajadores	Resolución rápida de dudas	Genera interrupciones y dependencia
Sistemas GMAO	Gestión centralizada del mantenimiento	Control y registro de operaciones	Interfaces complejas y no orientadas a asistencia
TechAssist	Agente de asistencia contextual	Recomendación inmediata y guía inicial	Funcionalidad limitada al apoyo operativo

3.5 Modelo de Negocio

Tras definir la propuesta de valor y la diferenciación estratégica, se presenta el modelo de negocio mediante la herramienta **Business Model Canvas**. Este análisis permite visualizar de forma integrada cómo TechAssist genera valor para el sector industrial, cuáles son sus alianzas estratégicas y la estructura de costes y beneficios.

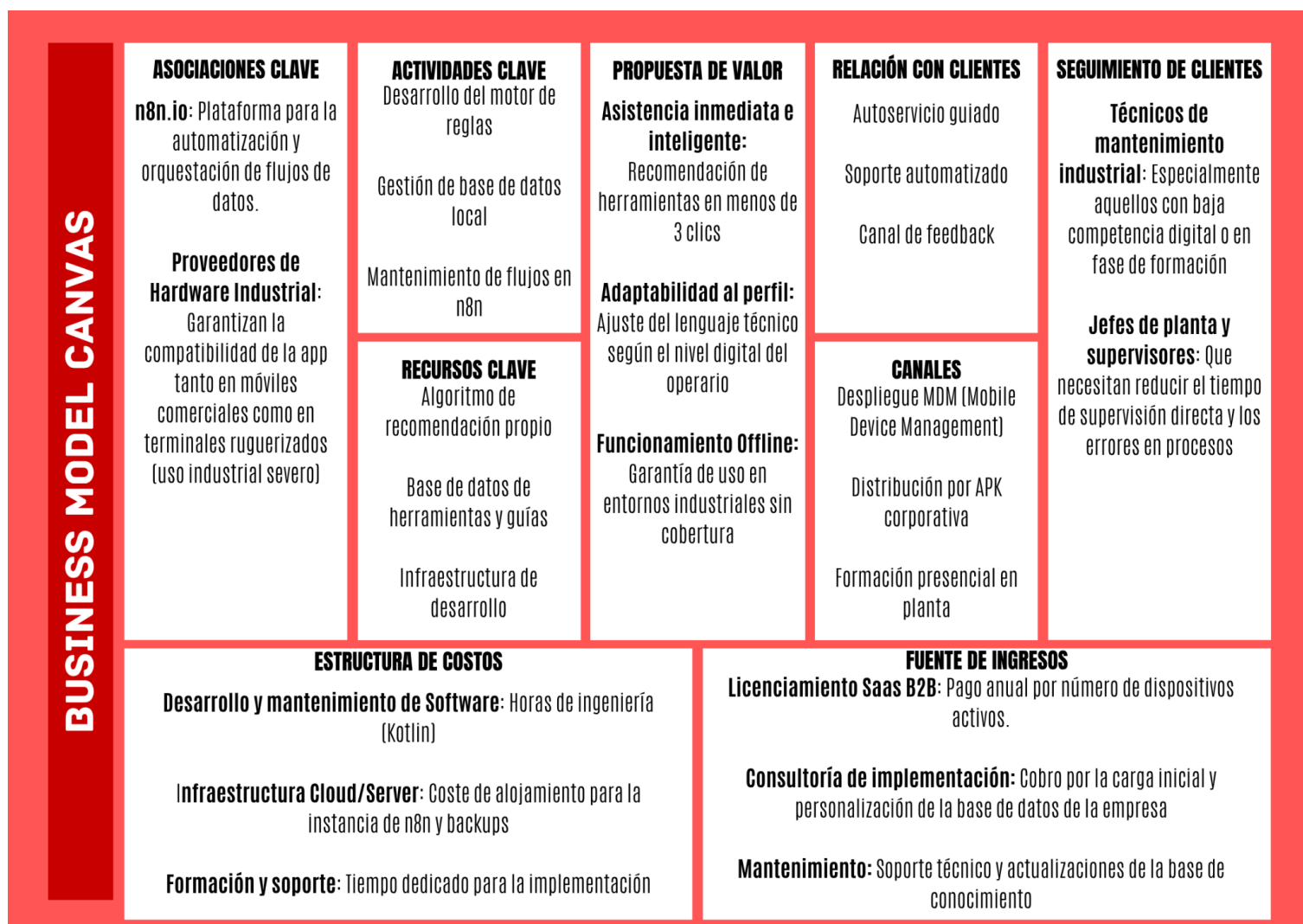


Figura 3.5 - Business Model Canvas para el despliegue de TechAssist

3.5.1 Justificación de los flujos de ingresos

Se ha seleccionado un modelo de suscripción B2B (SaaS) debido a que permite un mantenimiento continuo del software y la actualización constante de la base de datos de reglas. Además, el servicio de consultoría inicial asegura que la aplicación se adapte a la maquinaria específica de cada planta industrial, convirtiéndose en un ingreso directo por implementación.

3.5.2 Alianzas estratégicas y escalabilidad

Las alianzas con proveedores de hardware y el uso de la plataforma **n8n** no son solo decisiones técnicas, sino estratégicas. **n8n** permite que TechAssist escale sin necesidad de desarrollar un backend complejo desde cero, reduciendo drásticamente los costes operativos iniciales (OPEX).

3.5.3 Estructura de costes técnicos

El grueso de la inversión se concentra en el **desarrollo del motor de reglas** y la garantía de **operatividad offline**. A diferencia de otras soluciones que dependen de servidores en la nube costosos, TechAssist minimiza costes al procesar la lógica de recomendación de forma local en el dispositivo del técnico mediante SQLite.3.6 Validación inicial del enfoque

El enfoque del proyecto se valida a partir de los siguientes supuestos:

1. Los técnicos necesitan ayuda para identificar la herramienta adecuada.
2. Los supervisores buscan reducir consultas básicas.
3. Un agente basado en reglas puede resolver el problema sin integración compleja.

Los objetivos SMART definidos permiten medir:

- Precisión del minitest inicial.
- Reducción de errores digitales.
- Tiempo de inicio de tareas asistidas.

Dado que los requisitos priorizados son técnicamente abordables y el riesgo es controlable, se decide preservar con el enfoque actual del MVP, centrado en la recomendación de herramientas y la guía inicial de uso.

Validación preliminar del enfoque

Como parte del proceso de validación conceptual, se realizó una revisión informal del enfoque con dos perfiles cercanos al ámbito técnico, con el objetivo de evaluar la pertinencia del problema planteado y la utilidad potencial de la solución propuesta,

Durante esta revisión preliminar, se presentaron los siguientes elementos del proyecto:

- Descripción del problema detectado en el uso de herramientas.
- Concepto de agente de asistencia operativa.
- Ejemplo simplificado del flujo de recomendación de herramientas.

El feedback obtenido confirmó que la dificultad para identificar rápidamente qué herramienta utilizar es un problema frecuente en entornos técnicos donde coexisten múltiples aplicaciones corporativas.

Asimismo, se valoró la idea de disponer de una herramienta de orientación inmediata que reduzca la dependencia del supervisor y facilite la adopción de herramientas por parte de técnicos con distintos niveles de alfabetización tecnológica.

Aunque esta validación tiene carácter exploratorio, los resultados obtenidos refuerzan la pertinencia del enfoque adoptado para el desarrollo del MVP.

A partir del análisis de perfiles de usuario, de las tareas clave identificadas y de la propuesta de valor definida, se han podido determinar las funcionalidades esenciales que debe ofrecer la solución propuesta.

Estas funcionalidades se traducen en un conjunto de requisitos que describen el comportamiento esperado del sistema, incluyendo tanto las capacidades del agente de recomendación como los mecanismos de asistencia al usuario.

En el siguiente capítulo se presenta la especificación detallada de estos requisitos, estructurada mediante historias de usuario, requisitos funcionales y requisitos no funcionales.

4. Especificación de requisitos

El presente capítulo define los requisitos del sistema propuesto, estableciendo la forma estructurada que debe hacer la aplicación y bajo qué condiciones debe operar. La especificación de requisitos constituye el puente entre análisis conceptual desarrollado en los capítulos anteriores y el diseño técnico de la solución.

La identificación de requisitos se ha realizado a partir del análisis de perfiles de usuario (Customer Persona), de las tareas clave detectadas (Jobs To Be Done) y de la propuesta de valor definida previamente. De este modo, cada funcionalidad propuesta responde directamente a una necesidad real identificada en el contexto del mantenimiento industrial.

Los requisitos se clasifican en:

- Historias de usuario.
- Requisitos funcionales.
- Requisitos no funcionales.
- Representación gráfica mediante casos de uso.
- Priorización mediante matriz impacto-esfuerzo.

4.1 Historias de usuario.

Las historias de usuario permiten describir las funcionalidades del sistema desde la perspectiva de los distintos perfiles identificados en el capítulo anterior. Este enfoque facilita mantener la coherencia entre las necesidades reales detectadas en el sector y las características técnicas implementadas en la solución.

Se emplea el formato estándar:

Como [rol], quiero [funcionalidad], para [beneficio].

Cada historia incorpora criterios de aceptación que permiten validar objetivamente su cumplimiento durante la fase de pruebas.

Historia de Usuario -01- Recomendación de herramienta.

Como técnico de mantenimiento **quiero** introducir la tarea que voy a realizar **para** que el sistema me recomiende la herramienta adecuada.

- **Descripción:**
El técnico debe poder describir brevemente la tarea o seleccionar una categoría predefinida. El sistema procesará la entrada del usuario y generará una recomendación basada en la información proporcionada.
- **Criterios de aceptación:**
 - El sistema debe permitir introducir una descripción textual o seleccionar una categoría.
 - Debe mostrarse una única herramienta recomendada como resultado principal.
 - La recomendación debe incluir el nombre identificativo de la herramienta.
 - El tiempo de respuesta no debe superar los 10 segundos.

Prioridad: Must (funcionalidad núcleo del MVP).

Historia de Usuario -02- Explicación del criterio de recomendación.

Como técnico de mantenimiento **quiero** conocer por qué se me recomienda una herramienta concreta **para** aumentar mi confianza en el sistema y comprender el proceso.

- **Descripción:**
El sistema debe proporcionar una explicación breve y clara del criterio utilizado para emitir la recomendación (por ejemplo: “Esta aplicación es la oficial para el registro de intervenciones correctivas”).
- **Criterios de aceptación:**
 - Debe existir una opción visible “¿Por qué esta herramienta?”.
 - La explicación debe mostrarse en lenguaje claro y no técnico.
 - La respuesta debe generarse de forma inmediata tras la solicitud del usuario.

Prioridad: Must.

Historia de Usuario -03- Guía de primeros pasos.

Como técnico de mantenimiento **quiero** recibir una guía inicial de uso **para** iniciar correctamente el proceso digital sin cometer errores.

- **Descripción:**
Una vez recomendada la herramienta, el sistema mostrará entre uno y tres pasos iniciales estructurados que permitan comenzar el procedimiento correctamente.
- **Criterios de aceptación:**
 - La guía debe estar organizada en pasos numerados.
 - La información debe estar disponible sin conexión a internet.
 - El contenido debe adaptarse al nivel digital del usuario cuando corresponda.

Prioridad: Must

Historia de Usuario -04- Minitest de nivel digital.

Como técnico de mantenimiento **quiero** realizar un breve test inicial **para** que el sistema adapte la asistencia a mi nivel de competencia digital.

- **Descripción:**
El sistema incluirá un cuestionario inicial de evaluación compuesto por preguntas cerradas. En función de la puntuación obtenida, el usuario será clasificado en nivel básico, intermedio o avanzado.
- **Criterios de aceptación:**
 - El test debe contener un número limitado de preguntas (5-10).
 - El resultado del test debe quedar vinculado al perfil del usuario para su posterior uso por el sistema
 - El nivel asignado debe influir en la profundidad de las explicaciones mostradas.

Prioridad: Should

Historia de Usuario -05- Registro de procesos confusos.

Como supervisor/supervisora de mantenimiento **quiero** conocer qué procesos generan un mayor número de consultas **para** planificar refuerzos formativos específicos.

- **Descripción:**

El sistema debe registrar de forma agregada los procesos que presentan un mayor número de solicitudes de ayuda o repeticiones de consulta.

- **Criterios de aceptación:**

- El sistema debe almacenar eventos de consulta asociados a procesos.
- Debe existir una vista básica de resumen para el perfil supervisor.
- La información debe mostrarse de forma agregada, no individualizada.

Prioridad: Should.

Historia de Usuario -06- Gestión de base de herramientas

Como administrador del sistema **quiero** poder añadir, modificar o eliminar herramientas **para** mantener actualizada la base de conocimiento del agente.

- **Descripción:**

El sistema debe permitir la actualización de la base de herramientas y de las reglas asociadas a cada tipo de tarea.

- **Criterios de aceptación:**

- Debe existir un acceso restringido al perfil administrador.
- Se deben poder crear, editar y eliminar registros.
- Los cambios deben persistir en el sistema para que la base de conocimiento se mantenga actualizada.

Prioridad: Should.

4.2 Requisitos funcionales

Los requisitos funcionales describen de forma precisa las funcionalidades que el sistema debe implementar para satisfacer las necesidades identificadas para el análisis de usuario. Cada requisito se formula de manera verificable, permitiendo su validación posterior durante la fase de pruebas.

Los requisitos se codifican como RF-XX para facilitar su trazabilidad con las historias de usuario y los casos de prueba.

4.2.1 Gestión de recomendación de herramientas

RF-01. El sistema debe permitir al usuario introducir una descripción textual de la tarea a realizar.

RF-02. El sistema debe permitir seleccionar una categoría de tarea predefinida desde una lista estructurada.

RF-03. El sistema debe procesar la entrada del usuario mediante un conjunto de reglas internas definidas en la base de conocimiento.

RF-04. El sistema debe mostrar la herramienta más adecuada para la tarea, pudiendo ser digital (software, app, plataforma) o física (instrumento de medición, utillaje), según el contexto operativo definido en la base de conocimiento.

RF-05. La recomendación debe incluir el nombre identificativo de la herramienta y su categoría.

RF-06. El tiempo de respuesta del sistema ante una solicitud de recomendación no debe superar los 10 segundos.

4.2.2 Explicación del criterio de recomendación

RF-07. El sistema debe ofrecer una opción visible que permita al usuario consultar el criterio utilizado para la recomendación.

RF-08. El sistema debe mostrar una explicación clara, concisa y en lenguaje no técnico.

RF-09. La explicación debe generarse de forma inmediata tras la solicitud del usuario.

4.2.3 Guía inicial de uso

RF-10. El sistema debe mostrar una guía inicial compuesta por entre uno y tres pasos numerados.

RF-11. La guía debe estar asociada a cada herramienta registrada en la base de datos.

RF-12. La información de la guía debe estar disponible sin conexión a internet.

RF-13. El contenido mostrado debe adaptarse al nivel digital del usuario cuando esta funcionalidad esté activa.

4.2.4 Evaluación del nivel digital

RF-14. El sistema debe presentar un cuestionario inicial de evaluación del nivel digital del usuario.

RF-15. El sistema debe clasificar al usuario en nivel básico, intermedio o avanzado en función de sus respuestas.

RF-16. El nivel asignado debe almacenarse en la base de datos local del dispositivo.

RF-17. El sistema debe garantizar la persistencia del nivel digital asignado para que la personalización sea permanente entre sesiones de uso.

4.2.5 Registro y monitorización

RF-18. El sistema debe registrar de forma automática cada evento de consulta realizado por el técnico para su posterior análisis.

RF-19. El sistema debe asociar cada consulta al perfil de usuario y a la herramienta recomendada por el agente.

RF-20. El sistema debe implementar un Panel de Supervisión que permita visualizar de forma gráfica un resumen agregado de los procesos y herramientas con mayor número de consultas.

RF-21. La interfaz del supervisor debe garantizar que la información se presente de forma agregada para analizar tendencias de grupo, protegiendo la privacidad individual del técnico.

4.2.6 Gestión administrativa de herramientas

RF-22. El sistema debe permitir el acceso restringido a un perfil administrador.

RF-23. El perfil administrador debe poder crear nuevas herramientas en la base de datos.

RF-24. El perfil administrador debe poder modificar herramientas existentes.

RF-25. El perfil administrador debe poder eliminar herramientas de la base de datos.

RF-26. El sistema debe permitir la modificación de las reglas asociadas a cada tipo de tarea.

4.3 Requisitos no funcionales

Los requisitos no funcionales (RNF) describen las propiedades, características o restricciones que debe cumplir el sistema más allá de las funcionalidades específicas. Mientras que los requisitos funcionales determinan qué hace el sistema, los no funcionales determinan cómo lo hace, garantizando calidad, eficiencia y seguridad.

Para TechAssist, los principales requisitos no funcionales se clasifican en las siguientes categorías:

4.3.1 Rendimiento.

- **RNF-01. Tiempo de respuesta:**

El sistema debe procesar cualquier solicitud de recomendación y mostrar la herramienta sugerida en menos de 10 segundos, incluso en dispositivos con recursos limitados.

- **RNF-02. Optimización de recursos:**

La aplicación debe ejecutarse de manera eficiente, minimizando el consumo de CPU y memoria, para no interferir con otras aplicaciones industriales en ejecución.

* Documentado en el Anexo B

4.3.2 Disponibilidad y resiliencia.

- **RNF-03. Disponibilidad offline:**

Las guías iniciales, las reglas de recomendación y la base de herramientas deben estar totalmente accesibles sin conexión a internet.

- **RNF-04. Robustez:**

El sistema debe manejar errores de entrada del usuario o condiciones inesperadas sin bloquear la aplicación, mostrando mensajes de error claros y seguros.

4.3.3 Usabilidad.

- **RNF-05. Interfaz intuitiva:**

El usuario deberá poder acceder a la funcionalidad principal en un máximo de 3 interacciones, el tiempo de respuesta del sistema no deberá de superar los 2 segundos y la aplicación deberá funcionar completamente sin conexión a internet.

- **RNF-06. Adaptación al nivel digital:**

La complejidad de las explicaciones y guías debe ajustarse automáticamente al nivel digital del usuario (básico, intermedio o avanzado).

- ☐ **Criterio de verificación:**

El sistema debe mostrar diferencias visibles en la cantidad de pasos o nivel de detalle entre usuarios clasificados como básico y avanzado

4.3.4 Seguridad y control de acceso.

- **RNF-07. Gestión de perfiles:**

El acceso a la funcionalidad de administración debe estar restringido a usuarios con perfil administrador, mediante autenticación local.

- **RNF-08. Integridad de datos:**

La aplicación debe garantizar que las modificaciones en la base de datos de herramientas y reglas sean consistentes y no generen pérdidas de información.

4.3.5 Mantenibilidad y extensibilidad

- **RNF-09. Modularidad del código:**

El sistema debe diseñarse de forma modular para permitir modificaciones futuras en la lógica del agente o en la interfaz sin que una afecte a la otra

- ☐ **Criterio de verificación:**

La modificación de la lógica del agente no debe requerir cambios en las clases de interfaz de usuario (UI), validado mediante separación en capas MVC.

- **RNF-10. Actualización de reglas:**

Las reglas del sistema de recomendación basado en reglas deben poder modificarse o ampliarse fácilmente por el administrador sin necesidad de recompilar la aplicación.

4.3.6 Compatibilidad y portabilidad

- **RNF-11. Dispositivos Android:**

La aplicación debe ser compatible con versiones de Android desde la 8.0 (Oreo) en adelante y adaptarse a distintos tamaños de pantalla de dispositivos industriales.

- **RNF-12. Persistencia local:**

El sistema debe asegurar que la información guardada sea compatible con estándares actuales para permitir exportaciones o copias de seguridad sin pérdida de información.

4.4 Diagrama de casos de uso

El diagrama de casos de uso permite representar de forma gráfica las interacciones entre los distintos perfiles de usuario identificados y las funcionalidades principales del sistema. Este tipo de diagramas facilita comprender cómo los actores externos utilizan la aplicación y qué servicios ofrece el sistema para cada uno de ellos.

En el caso de TechAssist, se identifican tres actores principales:

- **Técnico de mantenimiento**, usuario principal de la aplicación, que utiliza el agente para recibir recomendaciones de herramientas y guías de uso.
- **Supervisor de mantenimiento**, que consulta información agregada sobre los procesos que generan mayor número de consultas o dificultades.

- **Administrador del sistema**, responsable de mantener actualizada la base de herramientas y las reglas del agente.

A partir de estos actores, se definen los casos de uso principales que representan las funcionalidades del sistema, como la recomendación de herramientas, la consulta de explicaciones sobre la recomendación, la visualización de guías de uso o la gestión administrativa de la base de conocimiento.

El diagrama permite visualizar cómo cada actor interactúa con el sistema y qué funcionalidades están disponibles para cada perfil, facilitando así la comprensión global del comportamiento de la aplicación.

Tabla 4.4 - Trazabilidad entre Casos de Uso, HU y Requisitos Funcionales

ID	Caso de Uso	Actor	HU	RF
CU-01	Realizar Minitest inicial	Técnico	HU-04	RF-14, RF-15, RF-16, RF-17
CU-02	Obtener recomendación	Técnico	HU-01	RF-01, RF-02, RF-03, RF-04, RF-05, RF-06
CU-03	Consultar Guía	Técnico	HU-03	RF-10, RF-11, RF-12, RF-13
CU-04	Ver Panel Supervisor	Supervisor	HU-05	RF-18- RF-19, RF-20, RF-21
CU-05	Consultar explicación	Técnico	HU-02	RF-07, RF-08, RF-09
CU-06	Gestión administrativa	Admin	HU-06	RF-22, RF-23, RF-24, RF-25, RF-26

Actores del sistema:

- **Técnico de mantenimiento:**
Realizar minitest de nivel digital.
Solicitar recomendación de herramienta.
Consultar explicación de recomendación.
Consultar guía inicial de uso.
- **Supervisor:**
Consultar procesos con mayor número de consultas.
- **Administrador:**
Crear herramienta.
Modificar herramienta.

Eliminar herramienta.
Modificar reglas del sistema.

Casos de uso principales

Los casos de uso identificativos son:

- ☐ Realizar minitest de nivel digital.
- ☐ Solicitar recomendación de herramienta.
- ☐ Consultar explicación de recomendación.
- ☐ Visualizar guía de primeros pasos.
- ☐ Consultar resumen de procesos confusos.
- ☐ Gestionar base de herramientas.

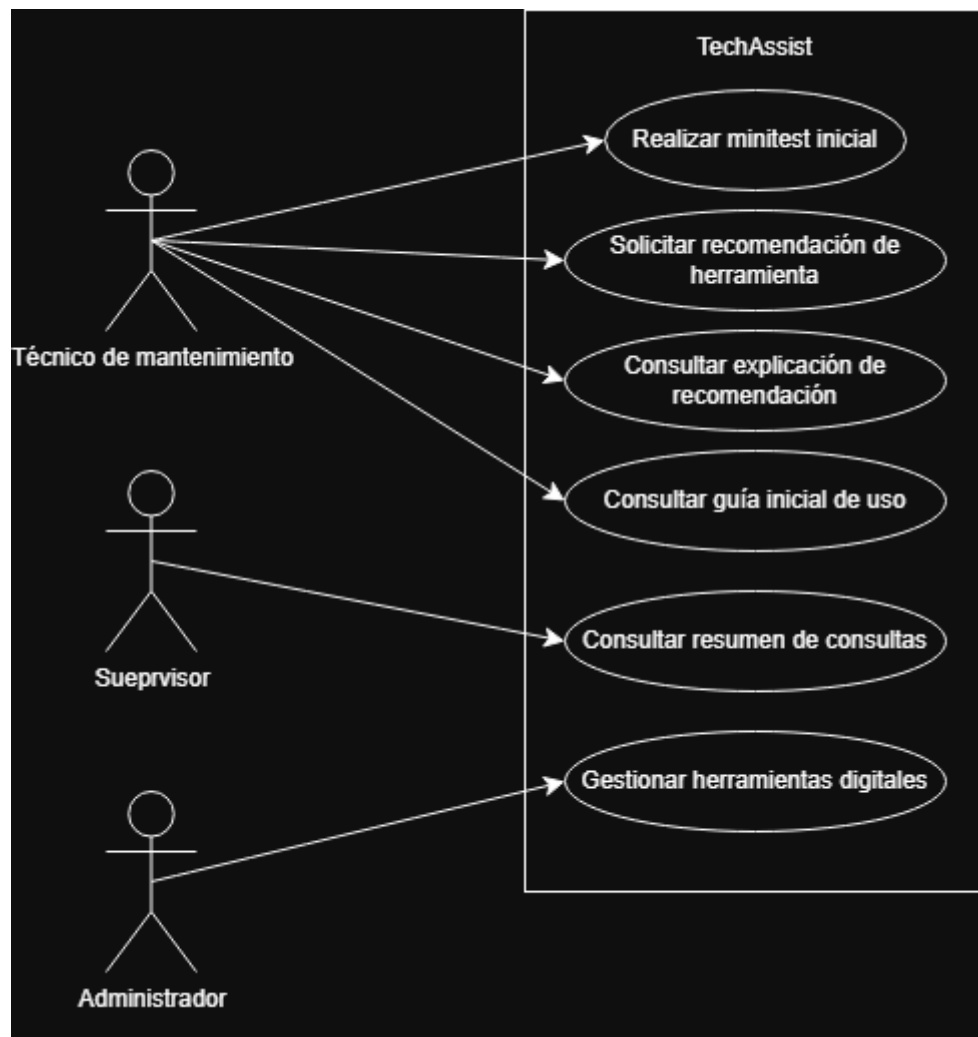


Figura 4.4 - Diagrama de casos de uso del sistema TechAssist.

4.5 Tabla de priorización impacto-esfuerzo

La priorización de funcionalidades del sistema se ha realizado mediante una tabla impacto-esfuerzo. Este método permite evaluar cada necesidad identificada en función de dos dimensiones principales: el impacto que genera en el valor del sistema para el usuario y el esfuerzo estimado necesario para su implementación técnica.

El impacto se refiere al grado en que la funcionalidad contribuye a resolver los problemas detectados en el análisis de usuarios, mientras que el esfuerzo considera la complejidad técnica, el tiempo de desarrollo y los recursos necesarios para su implementación.

El objetivo de esta evaluación es identificar aquellas funcionalidades que aportan mayor valor con un esfuerzo razonable, permitiendo definir de forma clara el alcance del Producto Mínimo Viable (MVP).

Tabla 4.5 - Tabla de priorización impacto-esfuerzo

Necesidad	Evidencia clave	Impacto	Esfuerzo	Prioridad
Formación digital básica	Técnicos con bajo manejo digital necesitan guía para operar aplicaciones	Alto	Medio	Principal, Alta
Curva del olvido / Refuerzo de procesos	Tareas poco frecuentes generan errores	Alto	Medio	Alta
Explicación del criterio de recomendación	Los usuarios necesitan confiar en la recomendación del sistema	Medio	Bajo	Media
Gamificación y motivación	Resistencia al aprendizaje digital	Medio	Bajo	Media
Monitorización de dudas recurrentes	Supervisores necesitan identificar procesos problemáticos	Medio	Medio	Media

A partir de esta evaluación, se determina que las funcionalidades que forman parte del MVP inicial del sistema son:

- “Recomendación de herramienta”
- “Explicación del criterio de recomendación”
- Guía inicial de uso”

- “Minitest de nivel digital”

La monitorización y gamificación se reservan para futuras versiones.

4.6 Prototipado de aplicación

Con el objetivo de validar la interacción del usuario con el sistema antes de su implementación, se han desarrollado una serie de wireframes que representan las principales pantallas de la aplicación. Estos prototipos permiten visualizar la estructura de la interfaz, los flujos de navegación y las funcionalidades clave definidas en los requisitos.

El prototipado se ha centrado en cubrir los escenarios principales de uso identificados en las historias de usuario, garantizando coherencia con el alcance definido en el MVP.

4.6.1 Acceso y Control de Perfiles (Login)

La primera interacción del sistema consiste en un control de acceso centralizado que determina la interfaz que se cargará en función del rol asignado al usuario.

- **Interfaz:** Pantalla de acceso con campos para identificación y contraseña.
- **Lógica de Navegación:** El sistema evalúa el perfil:
 - **Técnico (Primera vez):** Redirige a la pantalla de evaluación de nivel.
 - **Técnico (Recurrente):** Carga el Dashboard del Agente.
 - **Supervisor/Administrador:** Carga el Panel de Rendimiento de Grupo.

The wireframe shows a mobile application login screen. At the top, there is a header bar with the text 'TechAssist - Login'. Below the header, there are two text input fields. The first field is labeled 'ID de Empleado' and the second field is labeled 'Contraseña'. Below these fields is a single button labeled 'Acceder'.

Figura 4.6.1 - Prototipo de la interfaz de acceso (Login)

4.6.2 Evaluación del Nivel Digital (Minitest)

Para dar un cumplimiento a la HU-04, se ha diseñado una interfaz de cuestionario rápido.

- **Diseño:** Una pregunta por pantalla, con iconos grandes para facilitar la respuesta táctil.
- **Componentes:** Incluye una barra de progreso superior que indica al técnico su progreso, reduciendo la ansiedad y mejorando la experiencia de usuario.
- **Finalidad:** El resultado clasifica al usuario (Básico/Intermedio/Avanzado) y personaliza la densidad de las guías posteriores.

Diagrama de la interfaz de evaluación digital (Minitest) en un formato de pantalla de teléfono móvil:

- Encabezado: Minitest
- Progreso: 5/20
- Barra de progreso: Una barra horizontal con una sección verde que indica el progreso actual.
- Pregunta: Texto centrado que indica la pregunta actual.
- Opciones de respuesta: Cuatro botones rectangulares con esquinas redondeadas, etiquetados como Respuesta A, Respuesta B, Respuesta C y Respuesta D, dispuestos verticalmente.
- Botones de navegación: Dos botones rectangulares con esquinas redondeadas en la parte inferior, etiquetados como Volver y Siguiente.

Figura 4.6.2 - Prototipo de la interfaz de evaluación digital (Minitest)

4.6.3 Interacción con el Agente y Recomendación.

Esta es la interfaz núcleo del proyecto, diseñada para minimizar la carga cognitiva del técnico en planta.

- **Dashboard del Agente:** Presenta un buscador central predominante y una rejilla de "Accesos Rápidos" con botones sobredimensionados para las categorías de mantenimiento (Correctivo, Preventivo, Limpieza, etc.).
- **Ficha de Recomendación (Pop-up):** Al procesar la tarea, el sistema despliega una tarjeta central que contiene:
 1. La **Herramienta Recomendada** (HU-01).
 2. La **Justificación del Agente** ("¿Por qué?") en lenguaje no técnico (HU-02).
 3. La **Guía de 3 pasos** con checkboxes interactivos para asegurar el inicio correcto del proceso digital (HU-03).



Figura 4.6.3 - Prototipo del panel principal del Agente Recomendador

4.6.4 Gestión de Estados: Modo sin conexión (Offline)

Dado el entorno industrial de despliegue, donde la cobertura de red puede ser inestable o inexistente en ciertos puntos de la planta, se ha diseñado un estado específico para garantizar la continuidad del servicio.

- **Indicador Visual:** Se añade un banner de advertencia en la parte inferior del Dashboard y de la Ficha de Recomendación. Este elemento utiliza colores de precaución (amarillo/negro) para que el técnico sea consciente de que los datos no se están sincronizando en tiempo real con el servidor.
- **Lógica de Datos:** En este estado, la aplicación bloquea las funciones de "Exportación n8n" (que requieren red) pero mantiene totalmente operativo el motor de reglas interno del sistema para garantizar la asistencia en todo momento.
 - **La integración real con n8n queda fuera del alcance del MVP y se reserva como ampliación futura**
- **Feedback de Usuario:** Si el usuario intenta realizar una acción que requiere internet, el sistema despliega un *Snack-bar* o aviso indicando: *"Acción guardada localmente. Se sincronizará al recuperar la conexión"*.

Ficha de Trazabilidad:

- **Objetivo:** Asegurar la disponibilidad de las guías críticas en entornos sin cobertura.
- **Trazabilidad:** RNF-01 (Disponibilidad) / RNF-03 (Robustez).
- **Estado:** MVP (Requisito Crítico).

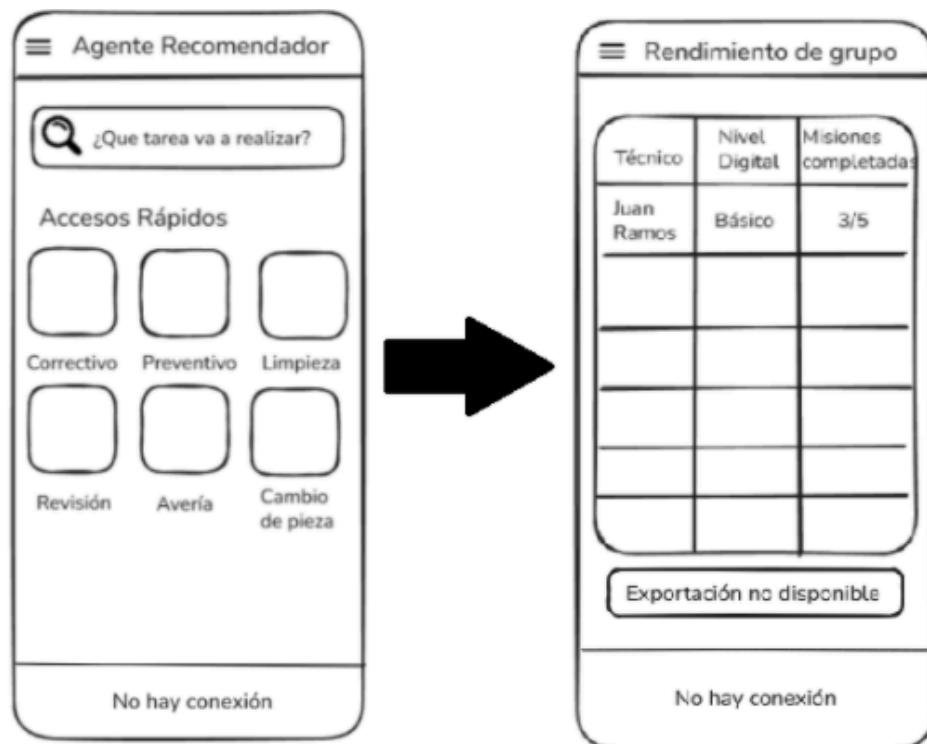


Figura 4.6.4 - Prototipado del agente recomendador y del panel de rendimiento del grupo

4.6.5 Panel de Gestión y Analítica (Supervisor)

Diseñado específicamente para la Persona 2 (Marta Vázquez), este panel **transforma los datos operativos en información de gestión**.

- **Visualización:** Se emplea una RecyclerView para listar a los técnicos del grupo asignado, mostrando su nivel digital y misiones completadas en tiempo real.
- **Integración n8n:** Se incluye un botón destacado de "Exportar Reporte". Al ser pulsado, la aplicación dispara un Webhook hacia n8n, que se encarga de procesar los datos y enviarlos fuera del ecosistema móvil (email o plataforma de BI), cumpliendo con el objetivo de monitorización proactiva (HU-05).
 - **Esta integración forma parte del diseño planificado pero queda fuera del MVP implementado, donde el botón de exportación actúa como elemento de interfaz previsto para versiones posteriores.**

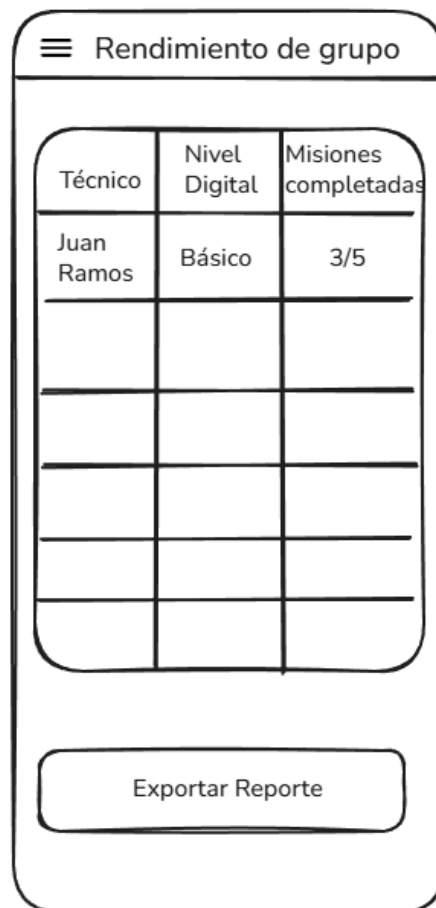


Figura 4.6.5 - Prototipo del panel de analítica para supervisión

5. Diseño de la solución

Este capítulo describe el diseño técnico de la solución propuesta para el sistema TechAssist. A partir de los requisitos definidos en el capítulo anterior, se presenta la estructura general del sistema, los principios de diseño adoptados y los componentes que permiten implementar la funcionalidad del agente de asistencia operativa.

El objetivo de este capítulo es mostrar cómo las necesidades detectadas en el análisis de usuarios y los requisitos funcionales se traducen en una arquitectura tecnológica concreta. Para ello, se abordan distintos aspectos del diseño, incluyendo la arquitectura del sistema, el modelo de datos, las decisiones tecnológicas adoptadas y el diseño de la experiencia de usuario.

El sistema se ha concebido siguiendo un enfoque modular que facilita su comprensión, mantenimiento y evolución futura. Esta modularidad permite separar claramente las responsabilidades de cada componente del sistema, evitando dependencias innecesarias y simplificando su desarrollo.

En los siguientes apartados, se detallan los distintos elementos que confirman la solución propuesta.

5.1 Diseño de la experiencia de usuario (UX)

El diseño de la experiencia de usuario (UX) en TechAssist se ha concebido como un elemento clave para garantizar la adopción de la aplicación en entornos industriales. Dado que los usuarios principales presentan niveles heterogéneos de competencia digital y operan en contextos de trabajo exigentes, la experiencia de uso debe ser clara, rápida y orientada a la acción.

A diferencia de aplicaciones convencionales, donde el usuario dispone de tiempo para explorar funcionalidades, en el contexto del mantenimiento industrial la interacción debe ser inmediata y sin ambigüedades. Por este motivo, el diseño UX de TechAssist se basa en los principios de simplicidad, accesibilidad y reducción de la carga cognitiva.

El objetivo principal es que el técnico pueda obtener una recomendación fiable y comenzar su tarea digital en el menor tiempo posible, sin necesidad de aprendizaje previo ni navegación compleja.

5.1.1 Flujos principales de interacción

Los flujos de interacción definen el recorrido que realiza el usuario dentro de la aplicación para cumplir sus objetivos. En TechAssist, estos flujos han sido diseñados para ser lineales, intuitivos y con el menor número de pasos posible.

Se identifican los siguientes flujos principales:

1. Acceso y personalización inicial

El usuario accede al sistema mediante autenticación. En el caso de un técnico que utiliza la aplicación por primera vez, se le redirige automáticamente al minitest de nivel digital. Este flujo permite adaptar la experiencia desde el primer momento, ajustando la complejidad de las explicaciones y guías.

2. Evaluación del nivel digital

El técnico realiza un cuestionario breve y guiado. Cada pregunta se presenta de forma individual para evitar sobrecarga visual. Al finalizar, el sistema asigna un nivel (básico, intermedio o avanzado), que condicionará la experiencia posterior.

3. Solicitud de recomendación

Este es el flujo principal del sistema. El usuario puede:

- Introducir una descripción de la tarea.
- Seleccionar una categoría predefinida.

El diseño prioriza elementos visuales grandes y accesibles, permitiendo una interacción rápida incluso en condiciones de trabajo adversas.

4. Recepción de recomendación

El sistema muestra una única herramienta recomendada mediante una interfaz clara y centralizada. Esta pantalla incluye:

- Nombre de la herramienta.

- Explicación del criterio de recomendación.
- Acceso a la guía inicial.

La información se presenta de forma jerárquica, destacando primero la acción principal que debe realizar el usuario

5. Consulta de guía de uso

El técnico accede a una guía estructurada en pasos numerados. Este flujo está diseñado para ser consultado rápidamente, sin necesidad de desplazamientos complejos ni lectura extensa.

6. Interacción continua

El usuario puede repetir consultas tantas veces como sea necesario. El sistema mantiene consistencia en la interacción para facilitar el aprendizaje implícito

7. Supervisión (flujo alternativo)

El perfil de supervisor accede a un panel donde puede visualizar información agregada sobre el uso del sistema. Este flujo está orientado a la toma de decisiones, no a la operación directa.

5.1.2 Escenarios de uso del agente

Para validar el diseño de la experiencia de usuario, se han definido distintos escenarios representativos del uso real del sistema en entorno industrial.

- **Escenario 1: Intervención correctiva urgente**

Un técnico necesita registrar una avería de forma inmediata. Ante la duda sobre qué aplicación utilizar, accede a TechAssist, introduce la tarea y recibe una recomendación instantánea junto con los pasos iniciales.

Resultado: reducción del tiempo de decisión y eliminación de errores en la selección de herramientas.

- **Escenario 2: Tarea poco frecuente**

El técnico debe realizar un proceso que no ejecuta habitualmente. Aunque conoce la herramienta, ha olvidado el procedimiento inicial.

Resultado: la guía de primeros pasos actúa como refuerzo de memoria, reduciendo el impacto de la curva del olvido.

- **Usuario con bajo nivel digital**

Un técnico con dificultades tecnológicas utiliza la aplicación por primera vez. El sistema adapta las explicaciones a un lenguaje más sencillo y proporciona mayor detalle en las guías.

Resultado: mejora de la confianza y reducción de la barrera de entrada digital.

- **Escenario 4: Supervisión de procesos**

La supervisora accede al sistema para identificar qué procesos generan más dudas en el equipo.

Resultado: obtención de información útil para planificar acciones formativas específicas.

En conjunto, el diseño de la experiencia de usuario se orienta a proporcionar una interacción fluida, directa y adaptada al contexto industrial, asegurando que el sistema actúa como un apoyo real en la operativa diaria y no como una carga adicional para el usuario

5.2 Diseño de la interfaz de usuario (UI)

En este apartado, se definen los elementos visuales y de interacción que dan forma a **TechAssist**. El diseño se ha centrado en la creación de una interfaz limpia, de alto contraste y adaptada a la fatiga visual propia de entornos industriales.

5.2.1 Identidad visual de la solución

Se ha establecido un sistema de colores basado en el rojo corporativo para mantener coherencia visual:

Tabla 5.2.1 - Especificaciones del Sistema de Diseño (UI)

Elemento	Especificación	Uso y justificación	Color
Color primario	Rojo #D32F2F	Identidad corporativa. Aplicado en botones de acción y cabeceras.	
Color de Fondo	Gris Neutro #F5F5F5	Reduce la fatiga visual en entornos con luz artificial intensa.	
Superficies	Blanco #FFFFFF	Máximo contraste en tarjetas y campos de entrada de datos.	
Color de Alerta	Ámbar #F9A825	Identificador exclusivo de estado sin conexión (evita confusión).	
Error Crítico	Rojo Oscuro #B00020	Fallos críticos o acciones destructivas. Diferente al rojo de marca.	
Éxito	Verde #4CAF50	Confirmación de tareas guardadas o procesos finalizados.	
Alerta	Amarillo #FFF600	Advertencia preventiva que no interrumpe el flujo de trabajo.	
Logros / Gamificación	Oro #FFC107	Gamificación: medallas, niveles y refuerzo positivo al operario.	

Este sistema de diseño no solo busca una estética profesional, sino que garantiza la usabilidad táctica en condiciones reales de planta. La diferenciación cromática entre el Rojo

Corporativo (navegación), el Ámbar (estado del sistema) y el Oro (progresión del usuario) permite que el técnico procese la información de manera subconsciente sin necesidad de leer cada mensaje, reduciendo así la carga cognitiva durante su jornada laboral.

5.2.2 Tipografía y Jerarquía Visual

La legibilidad es un factor crítico en entornos industriales donde la iluminación puede ser variable y el usuario interactúa con el dispositivo en movimiento o con equipos de protección. Por ello, se ha seleccionado la familia tipográfica Roboto.

Justificación de la fuente.

- **Estándar Android:** Al ser la fuente nativa de Google, garantiza una renderización perfecta en cualquier dispositivo móvil sin consumo extra de recursos.
- **Claridad:** Su diseño geométrico y abierto permite diferenciar caracteres similares, reduciendo errores en la lectura de códigos de error o manuales técnicos.

Tabla 5.2.2 - Escala tipográfica

Uso principal	Tamaño	Peso
Nombres de secciones y Logotipo	32sp	Bold (Negrita)
Títulos de tarjetas y preguntas	22sp	Medium
Cuerpo de texto y recomendaciones	16sp	Regular
Etiquetas de botones y campos	14sp	Medium
Mensajes de error y pies de foto	12sp	Regular

El uso de unidades sp asegura que, si el técnico tiene configurado un tamaño de fuente mayor en los ajustes de accesibilidad de su dispositivo, la aplicación se adapte de forma automática sin romper el diseño de la interfaz.

5.2.3 Estética del módulo de acceso (Login)

La interfaz de acceso se ha diseñado bajo una premisa de minimalismo industrial. El objetivo estético es transmitir seguridad y limpieza visual, evitando elementos que puedan confundir al técnico en el inicio de su jornada.

Composición Visual:

- **Equilibrio y Simetría:** Se utiliza un diseño de **columna central** con un fuerte peso visual en el tercio superior para el logotipo. Esto genera un flujo de lectura vertical natural: Identidad → Credenciales → Acción.
- **Contraste de Superficies:** Se aplica el color **Blanco (#FFFFFF)** en los contenedores de entrada sobre el fondo **Gris Neutro (#F5F5F5)** de la aplicación. Esta leve diferencia tonal ayuda a separar la zona de interacción del fondo de la pantalla sin necesidad de bordes negros agresivos.

Aplicación de Estilos UI:

- **Cromática de Marca:** El uso del **Rojo Corporativo (#D32F2F)** se reserva exclusivamente para el logotipo y el botón de acción principal (CTA). Esto crea un "anclaje visual" donde el color indica siempre el siguiente paso a seguir.
- **Redondeo de Componentes (Corner Radius):** Todos los botones y campos de texto cuentan con una curvatura de **12dp**. Estéticamente, esto suaviza la interfaz y le otorga un aspecto moderno y cercano, alejándose de los sistemas industriales antiguos de formas rígidas y angulosas.

Feedback Visual Estético:

- **Estados de Interacción:** Se han diseñado micro-interacciones para los campos de texto: cuando el técnico pulsa sobre uno, la línea inferior cambia a un grosor de 2px en rojo, indicando de forma elegante pero clara que el campo está activo (*Focused*).



Figura 5.2.3 - Interfaz de alta fidelidad para el módulo de acceso

5.2.4 Estética de la Evaluación del Nivel Digital (Minitest)

La interfaz del Minitest ha sido diseñada para maximizar el enfoque del usuario en la toma de decisiones, eliminando ruidos visuales y estructurando las preguntas de forma atómica.

Composición y Estructura:

- **Contenedores Tipo Tarjeta (Cards):** Cada pregunta se presenta dentro de una tarjeta blanca con una elevación sutil. Estéticamente, esto crea un foco de atención claro y separa el contenido del fondo, reduciendo la carga cognitiva del técnico.
- **Barra de Progreso:** Situada en la parte superior, utiliza el color **Verde (#4CAF50)** sobre un carril gris claro. Visualmente, esto comunica al usuario cuánto le falta para terminar, reduciendo la ansiedad y fomentando la finalización del test.

Estética de la Interacción:

- **Estados de Selección:** Las opciones de respuesta cambian de color y grosor de borde al ser pulsadas. Se utiliza un tono suave de gris o el color de éxito para confirmar visualmente la selección antes de pasar a la siguiente fase.
- **Uso del Color Oro (#FFC107):** Una vez finalizado el test, la pantalla de resultados utiliza el color de **Logros/Gamificación** para resaltar el nivel alcanzado (ej. "Técnico Avanzado"). Estéticamente, este tono dorado rompe con la paleta industrial de rojos y grises para generar una sensación de recompensa y reconocimiento.



Figura 5.2.4 - Interfaz de alta fidelidad para evaluación competencial (Minitest)

Jerarquía de Texto:

- Se emplea el estilo **Title Large (20ps)** para el enunciado de las preguntas, asegurando que el técnico pueda leer la cuestión de un vistazo rápido sin necesidad de acercar el dispositivo. Las respuestas utilizan el estilo **Body Large (16px)** para un equilibrio visual óptimo.

5.2.5 Estética de la Interacción Agente y Recomendación

Esta pantalla representa el centro de operaciones de **TechAssist**. El diseño estético busca transformar un panel de control complejo en una interfaz de "acceso directo" que minimice el tiempo de respuesta del técnico ante una incidencia.

Composición y Elementos de Interfaz:

- **Buscador Predictivo:** En la parte superior se ubica un campo de búsqueda con bordes redondeados y una sombra suave (*Elevation*). Estéticamente, este elemento invita a la interacción directa ("¿Qué tarea va a realizar?"), posicionándose como la vía principal de comunicación con el agente.
- **Matriz de Accesos Rápidos:** Las categorías de mantenimiento (Correctivo, Preventivo, Limpieza, etc.) se organizan en una rejilla de **iconos ilustrativos**. El uso de pictogramas con bordes negros definidos y detalles en amarillo/azul permite una identificación visual instantánea, incluso sin leer las etiquetas de texto en rojo.
- **Cabecera de Marca (App Bar):** Se utiliza el **Rojo Corporativo (#D32F2F)** como fondo de la barra superior, donde el texto en blanco garantiza un contraste máximo y una identidad visual persistente.

Estética de la Gestión de Estados (Modo Offline):

Como se observa en la comparativa de la Figura 5.2.5, la interfaz está diseñada para adaptarse visualmente a la pérdida de conectividad:

- **Banner de Advertencia:** Se ha implementado un aviso en la zona inferior utilizando el color **Ámbar (#F9A825)** definido en la paleta de alertas.
- **Contraste Crítico:** El uso de texto blanco sobre fondo ámbar rompe la estética limpia de la aplicación para generar un "ruido visual" intencionado, alertando al técnico de que el sistema está operando bajo el **Motor de Reglas local** y no sincronizando con el servidor.

Jerarquía Visual y Tipografía:

- Se aplica el estilo **Title Medium (20sp)** en rojo para el encabezado "Accesos Rápidos", delimitando claramente el área de herramientas del área de búsqueda. Las etiquetas de los iconos utilizan un tamaño reducido para no sobrecargar la composición, priorizando siempre la iconografía como lenguaje universal en planta.

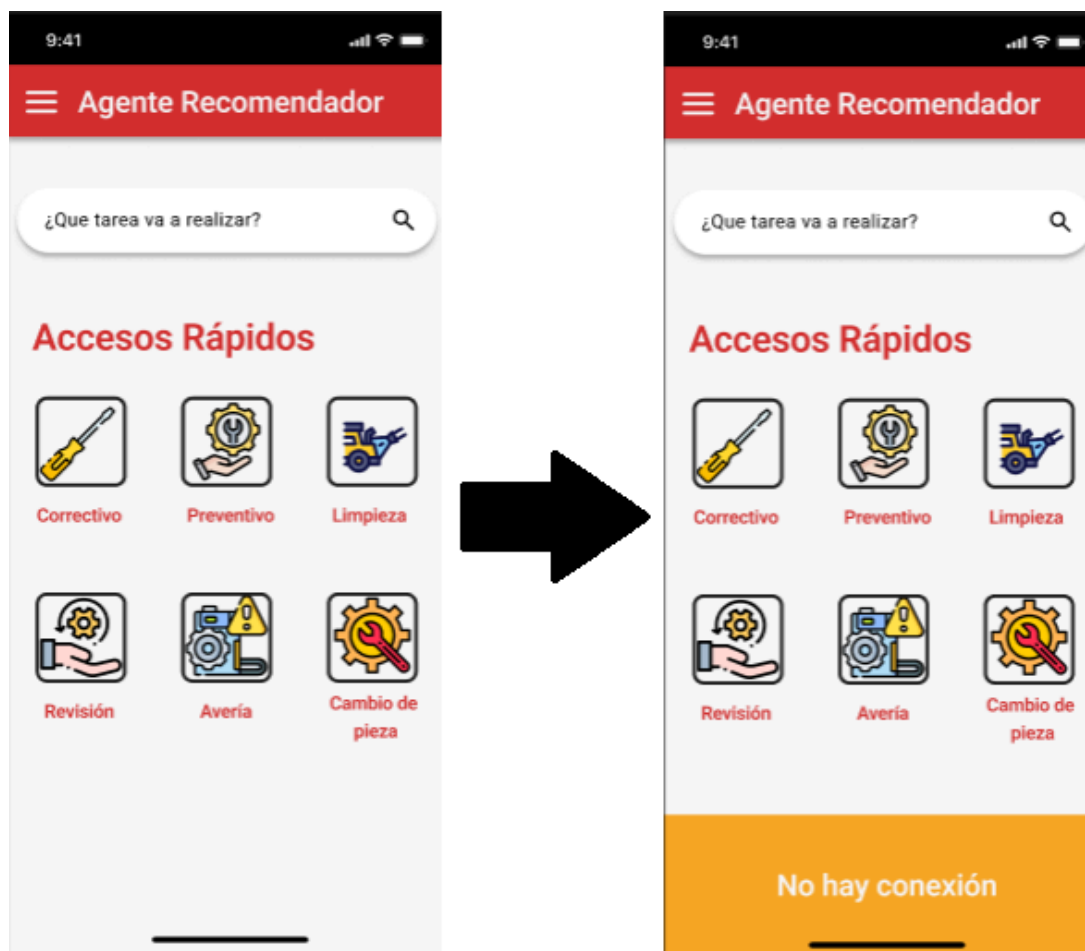


Figura 5.2.5 - Interfaz de alta fidelidad del Agente y adaptación visual ante pérdida de conexión

5.2.6 Estética de la pantalla de Supervisión (Panel de Gestión)

La interfaz de supervisión, denominada "Rendimiento de grupo", traslada la identidad visual de la aplicación a un entorno de control y gestión. A diferencia de las pantallas operativas, aquí la estética se pone al servicio de la **eficiencia administrativa** y el control de flujos de trabajo.

Composición y Visualización de Datos:

- **Tablas de Rendimiento:** Se utiliza una estructura de tabla limpia con cabeceras en gris neutro para organizar la información de los técnicos, su nivel digital y misiones. El uso de líneas de división claras asegura una lectura rápida de los KPI (Key Performance Indicators).
- **Consistencia de Marca:** La cabecera superior mantiene el **Rojo Corporativo (#D32F2F)** y el menú lateral (*Hamburger menu*), asegurando que el supervisor sienta que está dentro del mismo ecosistema que los técnicos, aunque su funcionalidad sea distinta.

Estética de Control y Restricciones:

Como se muestra en la comparativa de la **Figura 5.2.6**, la interfaz de supervisión responde dinámicamente al estado de la red para evitar errores de duplicidad de datos:

- **Inhabilitación de Acciones (Disabled State):** El botón principal de "Exportar Reporte" cambia drásticamente su estética cuando no hay conexión. Pasa de un estado activo en rojo con texto blanco a un **Gris Desactivado (#BDBDBD)** con el mensaje "Exportación no disponible". Este cambio visual es crítico para que el supervisor no intente forzar procesos que requieren sincronización con el servidor.
- **Banner de Alerta Persistente:** Se mantiene el banner inferior en **Ámbar (#F9A825)** con el texto "No hay conexión", unificando el lenguaje de advertencia en toda la plataforma.

Jerarquía de Acciones:

- El diseño prioriza el botón de acción en la parte inferior central, siguiendo la ergonomía de uso con una sola mano si el supervisor se encuentra desplazándose por la planta, manteniendo el estilo *Filled Button* con esquinas redondeadas (12dp).



Figura 5.2.6 - Interfaz de alta fidelidad del panel de supervisión y gestión de estados de red

5.3 Arquitectura general del sistema

La arquitectura del sistema define la organización estructural de la aplicación y la forma en que sus diferentes componentes interactúan entre sí. En el caso de TechAssist, se ha optado por una arquitectura sencilla y modular orientada a aplicaciones móviles con lógica local de recomendación.

El sistema se ha diseñado para funcionar principalmente en el dispositivo del usuario, reduciendo la dependencia de servicios externos y garantizando la disponibilidad de las funcionalidades básicas, incluso en entornos con conectividad limitada, situación frecuente en entornos industriales.

La arquitectura del sistema se organiza mediante una separación de responsabilidades. Para gestionar la información de forma eficiente, se utiliza un Módulo de Gestión de Datos que actúa como intermediario. Este componente es el encargado de consultar la base de datos local SQLite para que el técnico siempre tenga respuesta (incluso sin conexión) y, al mismo tiempo, gestiona el envío de reportes hacia el sistema externo n8n de forma automática cuando el dispositivo detecta internet.

De forma general, el sistema está compuesto por los siguientes elementos principales:

- **Interfaz de usuario (UI):** componente encargado de la interacción directa con el usuario, permitiendo introducir consultas, visualizar recomendaciones y acceder a las guías de uso.
- **Motor de recomendación basado en reglas:** componente central del sistema que analiza la información proporcionada por el usuario y determina la herramienta más adecuada en función de un conjunto de reglas definidas.
- **Base de datos local:** sistema de almacenamiento que contiene la información necesaria para el funcionamiento del agente, incluyendo herramientas, reglas de recomendación, resultados del test de nivel digital y registros de consultas.
- **Módulo de monitorización:** encargado de registrar las consultas realizadas por los usuarios para permitir posteriormente un análisis agregado por parte de perfiles de supervisión.

Esta organización permite mantener una estructura clara en la aplicación y facilita futuras ampliaciones del sistema, como la incorporación de nuevas reglas o herramientas.

La arquitectura se basa en el patrón Modelo-Vista-Controlador (MVC), estructurado para priorizar la disponibilidad offline. El modelo gestiona el acceso a la base de datos SQLite; la **Vista** se define mediante layouts XML; y el **Controlador** (Activities/Fragments) actúa como el cerebro que recibe las peticiones del técnico, consulta al modelo y actualiza la interfaz en tiempo real.

5.3.1 Arquitectura por capas

El sistema sigue un modelo de arquitectura por capas que separa las responsabilidades principales de la aplicación. Este enfoque facilita el mantenimiento del software y permite modificar componentes concretos sin afectar al resto del sistema.

Las capas principales del sistema son las siguientes:

- **Capa de presentación**
Es la capa encargada de la interacción con el usuario. Incluye las pantallas de la aplicación móvil, los formularios de entrada de datos y los elementos visuales que muestran las recomendaciones generadas por el agente. Su objetivo es ofrecer una interfaz clara e intuitiva que permita a los técnicos acceder rápidamente a la información necesaria para realizar sus tareas.
- **Capa lógica de aplicación:**
En esta capa se encuentra el motor del agente de recomendación. Su función es procesar la información introducida por el usuario y aplicar las reglas de decisión almacenadas en el sistema para determinar qué herramienta debe recomendarse. Esta capa también gestiona la generación de explicaciones asociadas a la recomendación y la adaptación de las guías de uso según el nivel digital del usuario.
- **Capa de datos:**
La capa de datos se encarga del almacenamiento y recuperación de la información utilizada por el sistema. Incluye la base de datos local donde se almacenan las herramientas disponibles, las reglas de recomendación, los perfiles de usuario y los registros de consultas realizadas.

La separación entre estas capas permite mantener una arquitectura clara y facilita la escalabilidad del sistema en futuras versiones.

5.3.2 Componentes principales del sistema

A nivel funcional, el sistema TechAssist se compone de varios módulos que colaboran para ofrecer la asistencia al usuario.

- **Módulo de consulta.**
Permite al técnico seleccionar la categoría de tarea que desea realizar. Esta información constituye la entrada principal del sistema y desencadena el proceso de recomendación.
- **Módulo de recomendación**
Este módulo representa el núcleo del agente inteligente. A partir de la información proporcionada por el usuario, consulta la base de reglas del sistema y determina cuál es la herramienta más adecuada para la tarea solicitada.
- **Módulo de guía de uso**
Asociado a cada herramienta, este módulo proporciona una breve guía inicial que ayuda al usuario a comenzar a utilizar la aplicación recomendada.
- **Módulo de evaluación de nivel digital**
Este componente permite clasificar al usuario según su nivel de competencia digital mediante un pequeño cuestionario inicial. El nivel obtenido se utiliza posteriormente para adaptar el grado de detalle de las explicaciones y guías mostradas.
- **Módulo de registro de consultas**
Finalmente, el sistema incluye un módulo encargado de registrar de forma agregada las consultas realizadas por los usuarios. Esta información puede ser utilizada por perfiles de supervisión para identificar procesos que generan mayor número de dudas y reforzar la formación en dichos ámbitos.

5.4 Stack tecnológico

La selección del conjunto de tecnologías que conforman el stack de este proyecto responde a la necesidad de crear una herramienta robusta, eficiente y fácilmente integrable en dispositivos industriales. Se ha priorizado el desarrollo nativo frente a soluciones híbridas para garantizar un rendimiento óptimo y un acceso sin restricciones al hardware en terminales que, en muchas ocasiones, presentan recursos limitados o versiones de sistemas operativos personalizadas.

5.4.1 Entorno de Desarrollo (IDE): Android Studio

Como entorno de desarrollo principal se ha utilizado **Android Studio**, el IDE oficial de Google. Su elección se justifica por la integración total con el SDK de Android, permitiendo una compilación nativa que aprovecha al máximo la arquitectura del procesador. Además, el uso de sus herramientas de depuración (profiler) es crítico para monitorizar el consumo de memoria y batería, asegurando que la aplicación no interfiera con el rendimiento de otras herramientas industriales críticas que el operario pueda tener en ejecución simultánea.

5.4.2 Lenguaje de Programación: Kotlin

El núcleo de la aplicación está desarrollado íntegramente en **Kotlin**. Se ha elegido este lenguaje en lugar de **Java** para modernizar la base de código y reducir la deuda técnica. Su sistema de **Seguridad frente a nulos (Null Safety)** es fundamental para evitar cierres inesperados de la app en momentos de asistencia técnica crítica. Asimismo, su sintaxis concisa y el uso de corrutinas permiten implementar una lógica de Agente fluida y reactiva, garantizando que el operario no sufra retardos mientras consulta un procedimiento de urgencia.

5.4.3 Interfaz de Usuario: Android Views (XML)

Para la construcción de la interfaz, se ha optado por el sistema tradicional de Android Views mediante archivos XML. Esta decisión técnica se fundamenta en la necesidad de un **Control Jerárquico** absoluto sobre el árbol de vistas y que el uso de Android Views permite un control exhaustivo sobre el dimensionamiento de los elementos de interacción (Touch Targets). Esto es fundamental en el sector industrial, donde el operario puede interactuar con la interfaz en condiciones de movilidad o utilizando equipos de protección individual, lo que requiere áreas de pulsación sobredimensionadas y una jerarquía visual rígida que evite errores de navegación.

5.4.4 Arquitectura de Software: Patrón MVC (Modelo-Vista-Controlador)

Se ha implementado el patrón **MVC** para centralizar la lógica del Agente en el **Controlador**. Este componente intercepta la categoría seleccionada por el técnico y dispara una consulta estructurada hacia la base de datos **SQLite**. Tras recibir el objeto de datos del **Modelo**, el Controlador actualiza dinámicamente los elementos de la **Vista**, logrando que la recomendación sea instantánea. Además, el Controlador gestiona de forma asíncrona el envío de telemetría hacia **n8n** mediante el uso de Corrutinas de Kotlin, evitando bloqueos en la interfaz.

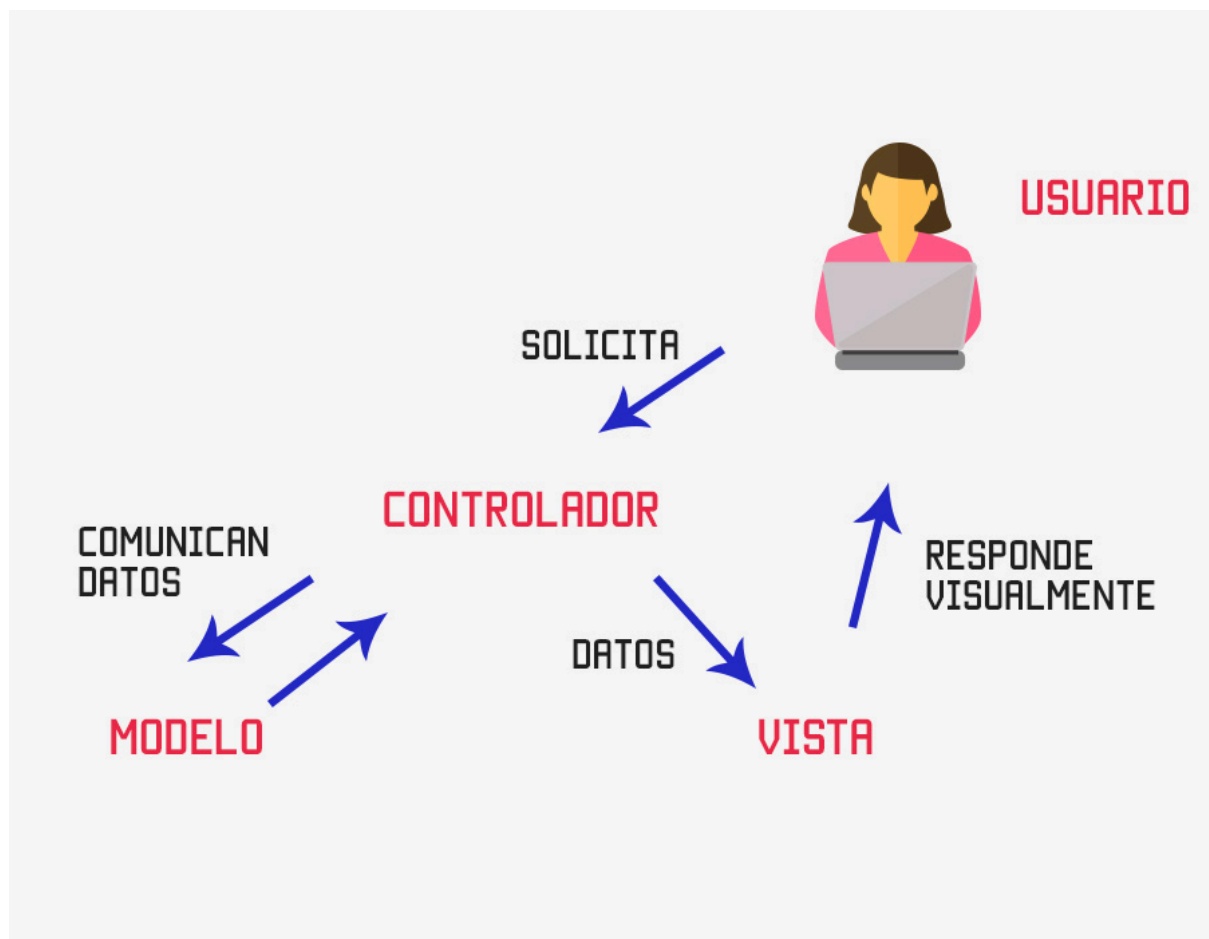


Figura 5.4.4 - Esquema lógico del patrón arquitectónico Modelo-Vista-Controlador (MVC)

5.4.5 Persistencia de Datos: SQLite

Para asegurar que el operario pueda acceder a las guías de ayuda en zonas de la planta sin cobertura de red (puntos ciegos de conectividad), se utiliza una base de datos local basada en SQLite. Para garantizar la resiliencia operativa, se utiliza SQLite gestionado a través de una clase especializada de tipo OpenHelper. Esta elección técnica permite que el Agente consulte la tabla de Reglas y la tabla de Herramientas en menos de 200ms, asegurando que el soporte técnico esté disponible incluso en 'zonas muertas' de conectividad industrial. El esquema de datos se ha normalizado para optimizar las consultas JOIN entre las tablas de categorías y guías.

La persistencia local se gestiona mediante una clase **Helper de SQLite**, que centraliza la creación de tablas y las operaciones CRUD (Crear, Leer, Actualizar, Borrar). Esto asegura que el sistema de recomendación basado en reglas pueda consultar las reglas de recomendación con latencia cero, independientemente de la cobertura de red.

5.4.6 Automatización y Orquestación: n8n (Low-Code Backend)

Para la gestión de flujos de datos externos y la comunicación con los perfiles de supervisión, se ha integrado **n8n**, una herramienta de automatización basada en nodos. Su inclusión en el stack responde a la necesidad de dotar al sistema de una capacidad de respuesta proactiva sin sobrecargar el código nativo de la aplicación.

- **Justificación técnica:** n8n permite crear Webhooks que actúan como receptores de los eventos registrados por la aplicación (como la detección de procesos confusos). Al delegar la lógica de notificación y reporte en n8n, se garantiza que la app Android consuma menos recursos de red y batería.
- **Resiliencia y escalabilidad:** Gracias a este enfoque, si en el futuro la empresa decide cambiar el destino de los reportes (por ejemplo, de un correo electrónico a un canal de Slack o un panel de Power BI), el cambio se realiza en el flujo de n8n sin necesidad de modificar ni reinstalar la aplicación en los dispositivos de los técnicos.
- **Sincronización asíncrona:** n8n permite que los reportes de rendimiento de grupo (HU-05) se procesen de forma asíncrona; la app envía el dato cuando detecta conexión y n8n se encarga de que llegue a la supervisora de forma estructurada.

Nota: aunque n8n forma parte del stack tecnológico diseñado para la solución completa, su integración efectiva queda fuera del alcance del MVP actual y se contempla como línea de evolución futura, tal y como se detalla en la sección 6.3.2 y 6.8.

5.5 Modelo de datos

El modelo de datos define la estructura de la información gestionada por el sistema TechAssist. Dado que la aplicación funciona principalmente en local, se ha diseñado un modelo relacional implementado mediante SQLite, que permite almacenar tanto la base de conocimiento del agente como la información generada durante la interacción con el usuario.

El objetivo de este modelo es dar soporte al funcionamiento del agente de recomendación, permitiendo almacenar las herramientas disponibles, las reglas de decisión, las guías de uso y los datos asociados al nivel digital del usuario.

El diseño se ha realizado siguiendo un enfoque simple y modular, acorde al alcance del MVP, evitando una complejidad innecesaria y facilitando su mantenimiento y futura ampliación.

5.5.1 Entidades principales del sistema

Las entidades principales identificadas en el sistema son las siguientes:

- Usuario
- Herramienta
- Regla
- Guía
- NivelDigital
- Consulta

Cada una de estas entidades cumple una función específica dentro del funcionamiento del agente.

5.5.2 Definición de tablas

A continuación, se describe la estructura de las tablas que componen la base de datos del sistema.

Tabla 5.5.2a - Usuario

Campo	Tipo	Descripción
id_usuario	INTEGER (PK)	Identificador único del usuario
nombre	TEXT	Nombre del usuario
rol	TEXT	Tipo de usuario (Técnico, supervisor, administrador)
nivel_digital_id	INTEGER (FK)	Referencia al nivel asignado
contrasena	TEXT	Contraseña del usuario

Tabla 5.5.2b - NivelDigital

Campo	Tipo	Descripción
id_nivel	INTEGER (PK)	Identifiador del nivel
nombre	TEXT	Nivel (básico, intermedio, avanzado)
descripcion	TEXT	Descripción del nivel

Tabla 5.5.2c - Herramienta

Campo	Tipo	Descripción
id_herramienta	INTEGER (PK)	Identificador único de la herramienta
nombre	TEXT	Nombre de la herramienta
descripcion	TEXT	Descripción básica
categoria	TEXT	Tipo de tarea asociada

Tabla 5.5.2d - Regla

Campo	Tipo	Descripción
id_regla	INTEGER (PK)	Identificador de la regla
categoria_tarea	TEXT	Categoría de tarea introducida por el usuario
herramienta_id	INTEGER(FK)	Herramienta recomendada
explicacion	TEXT	Justificación de la recomendación

Tabla 5.5.2e - Guía

Campo	Tipo	Descripción
id_guia	INTEGER (PK)	Identificador de la guía
herramienta_id	INTEGER (FK)	Herramienta asociada
paso_1	TEXT	Primer paso de la guía
paso_2	TEXT	Segundo paso (opcional)
paso_3	TEXT	Tercer paso (opcional)

Tabla 5.5.2f - Consulta

Campo	Tipo	Descripción
id_consulta	INTEGER (PK)	Identificador de la consulta
usuario_id	INTEGER(FK)	Usuario que realiza la consulta
herramienta_id	INTEGER (FK)	Herramienta recomendada
fecha	DATE	Fecha de la consulta
categoria_tarea	TEXT	Categoría de la tarea consultada

5.5.3 Relaciones entre entidades

Las principales relaciones del modelo de datos son las siguientes:

- Un usuario tiene asociado un único nivel digital.
- Una regla está asociada a una herramienta, determinando qué herramienta se recomienda para cada categoría de tarea.
- Cada herramienta dispone de una única guía inicial de uso.
- Cada consulta registra la interacción de un usuario con el sistema, incluyendo la herramienta recomendada.

Estas relaciones permiten estructurar de forma coherente la información necesaria para el funcionamiento del agente y para la posterior monitorización del uso del sistema.

5.5.4 Soporte al funcionamiento del agente

El modelo de datos propuesto permite implementar un sistema de recomendación basado en reglas de forma eficiente.

Cuando el usuario introduce una tarea o selecciona una categoría, el sistema consulta la tabla de reglas para identificar la herramienta asociada. A partir de esta información, se obtiene la herramienta recomendada y se recupera tanto la explicación del criterio como la guía inicial de uso desde las tablas correspondientes.

Este enfoque garantiza un comportamiento determinista del agente, alineado con los requisitos definidos, evitando la complejidad de modelos probabilísticos y facilitando la trazabilidad de las decisiones del sistema.

5.5.5 Consideraciones de diseño

El modelo ha sido diseñado teniendo en cuenta los siguientes principios:

- **Simplicidad:** Se han definido únicamente las tablas necesarias para cubrir el alcance del MVP.
- **Persistencia local:** Toda la información se almacena en SQLite para garantizar funcionamiento offline.
- **Extensibilidad:** El sistema permite añadir nuevas herramientas, reglas o guías sin modificar la estructura principal.
- **Trazabilidad:** El registro de consultas permite analizar el uso del sistema y detectar patrones de comportamiento.

5.5.6 Diagrama entidad-relación

A continuación, se presenta el diagrama entidad-relación del sistema, que representa de forma gráfica las entidades definidas y sus relaciones.

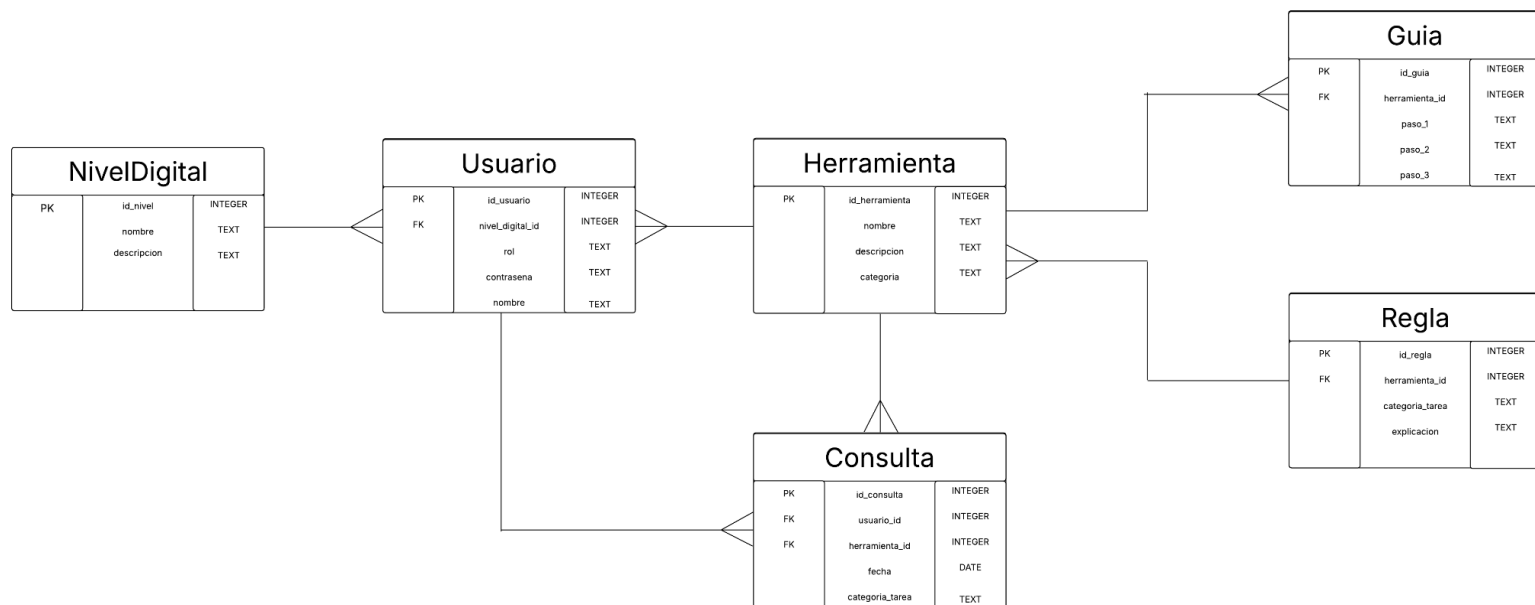


Figura 5.5.6 - Diagrama Entidad-Relación de la base de datos local del sistema

Como se observa en el diagrama, el modelo está diseñado para dar soporte al motor de reglas del agente:

- **Relación Usuario-Nivel:** Cada técnico tiene asignado un *nivel_digital_id*, lo que permite al controlador filtrar la profundidad de las guías.
- **Motor de Inferencia (Regla-Herramienta):** La tabla Regla actúa como nexo principal, vinculando una categoría de tarea con su *herramienta_id* correspondiente.
- **Trazabilidad:** La tabla *Consulta* registra la actividad permitiendo que el Panel de Supervisión genere los informes agregados requeridos en el **RF-21**.

6. Planificación y gestión del proyecto

Este capítulo describe la planificación y gestión del desarrollo del proyecto TechAssist, abordando la metodología de trabajo adoptada, la organización de tareas y la estimación de recursos necesarios para su ejecución.

A diferencia de los capítulos anteriores, centrados en el análisis y diseño de la solución, en este apartado se presenta cómo se ha estructurado el proceso de desarrollo desde una perspectiva organizativa, definiendo las fases del proyecto, la priorización de funcionalidades y la planificación temporal.

Dado que se trata de un proyecto individual con un alto componente iterativo, se ha optado por un enfoque de desarrollo ágil adaptado, que permite evolucionar la solución de forma progresiva a partir de un Producto Mínimo Viable (MVP). Este enfoque facilita la validación temprana de las funcionalidades clave y la incorporación de mejoras en función de los resultados obtenidos durante el desarrollo.

Asimismo, se incluyen elementos fundamentales de gestión de proyectos como el Product Backlog, la planificación por sprints, la estimación de recursos y la definición de criterios de aceptación, con el objetivo de garantizar la viabilidad, coherencia y trazabilidad del proceso de desarrollo.

6.1 Metodología de trabajo:

El desarrollo del proyecto TechAssist se ha llevado a cabo siguiendo un enfoque de trabajo iterativo con principios ágiles, adaptado a las características de un proyecto individual. Este enfoque ha permitido estructurar el desarrollo en fases progresivas, facilitando la validación continua de la solución y la incorporación de mejoras de forma incremental.

A diferencia de metodologías tradicionales como el modelo en cascada, donde todas las fases se ejecutan de forma secuencial, en este proyecto se ha optado por una estrategia flexible que permite redefinir requisitos y ajustar el alcance en función de los resultados obtenidos durante el desarrollo. Esta decisión se justifica por la naturaleza exploratoria del proyecto, en el que tanto la definición del problema como la solución han evolucionado a lo largo del proceso.

El enfoque adoptado combina elementos de metodologías ágiles, como la priorización de funcionalidades mediante backlog y la organización del trabajo en iteraciones, con planificación estructurada propia de entornos académicos. De este modo, se ha conseguido mantener un equilibrio entre flexibilidad y control del proyecto.

El desarrollo se ha organizado en torno a la construcción de un Producto Mínimo Viable (MVP), definido como el conjunto de funcionalidades esenciales que permiten validar la propuesta de valor del sistema. A partir de este núcleo inicial, el sistema se ha ido ampliando mediante la incorporación progresiva de nuevas funcionalidades y mejoras.

Las principales fases del proceso de desarrollo han sido las siguientes:

- Análisis del problema y del contexto del sector, identificando las necesidades reales de los usuarios.
- Definición de la propuesta de valor y de los requisitos del sistema.
- Diseño de la arquitectura y del modelo de datos.
- Desarrollo iterativo del MVP y de sus funcionalidades principales.
- Evaluación y validación del enfoque mediante revisión conceptual.

Este enfoque metodológico ha permitido mantener una visión global del proyecto, al tiempo que se ha trabajado de forma incremental sobre los distintos componentes del sistema, reduciendo riesgos y facilitando la toma de decisiones durante el desarrollo.

6.2 Product Backlog

El Product Backlog constituye el conjunto priorizado de funcionalidades, mejoras y tareas necesarias para el desarrollo del sistema TechAssist. Este artefacto recoge de forma estructurada todos los elementos identificados a partir de los requisitos definidos en el capítulo anterior, sirviendo como base para la planificación iterativa del proyecto.

En el contexto de este proyecto, el Product Backlog se ha construido a partir de:

- Las historias de usuario definidas en la sección 4.1.
- Los requisitos funcionales y no funcionales.
- La priorización obtenida mediante la tabla impacto-esfuerzo.

Dado que se trata de un desarrollo individual con enfoque ágil, el backlog no se considera un elemento estático, sino un listado dinámico que puede evolucionar a lo largo del proyecto conforme se obtienen nuevos aprendizajes o se identifican mejoras.

6.2.1 Estructura del Product Backlog

Cada elemento del backlog (Backlog Item) se describe mediante los siguientes atributos:

- Identificador
- Descripción
- Tipo (Historia de usuario, mejora, tarea técnica)
- Prioridad (Must, Should, Could)
- Valor aportado
- Estimación de esfuerzo

6.2.2 Listado de elementos del backlog

A continuación, se presentan los principales elementos que componen el Product Backlog:

Tabla 6.2.2 - Listado de elementos del backlog

ID	Elemento	Tipo	Prioridad	Valor	Esfuerzo
PB-01	Implementación del sistema de recomendación de herramientas	HU-01	Must	Muy alto	Alto
PB-02	Desarrollo de la explicación del criterio de recomendación	HU-02	Must	Alto	Bajo
PB-03	Implementación de la guía de primeros pasos	HU-03	Must	Muy alto	Medio
PB-04	Desarrollo de minitest de nivel digital	HU-04	Should	Alto	Medio
PB-05	Registro de consultas y procesos confusos	HU-05	Should	Medio	Medio
PB-06	Gestión de base de herramientas (CRUD)	HU-06	Should	Alto	Medio
PB-07	Diseño de interfaz de usuario	Tarea técnica	Must	Muy alto	Alto
PB-08	Implementación de base de datos SQLite	Tarea técnica	Must	Muy alto	Medio
PB-09	Desarrollo del motor de reglas del agente	Tarea técnica	Must	Muy alto	Alto
PB-10	Adaptación de contenido según nivel digital	Mejora funcional	Should	Alto	Medio
PB-11	Implementación del sistema de login y perfiles	Tarea técnica	Must	Alto	Medio
PB-12	Panel de supervisión	Mejora funcional	Should	Medio	Medio
PB-13	Integración con n8n para exportación de datos	Tarea técnica	Should	Medio	Medio

6.2.3 Priorización del backlog

La priorización del Product Backlog se ha realizado siguiendo un enfoque basado en valor, donde se priorizan aquellas funcionalidades que:

- Resuelven directamente el problema principal del usuario.
- Permiten validar la propuesta de valor del sistema.
- Son necesarias para la construcción del MVP.

En este sentido, los elementos con prioridad Must constituyen el núcleo del sistema y deben implementarse obligatoriamente en las primeras iteraciones. Los elementos Should aportan valor significativo, pero pueden posponerse si existen restricciones de tiempo. Por último, los elementos Could representan mejoras o ampliaciones que no son críticas para el funcionamiento inicial del sistema.

6.2.4 Relación con MVP

El Product Backlog permite identificar de forma clara qué elementos forman parte del Producto Mínimo Viable. En concreto, el MVP estará compuesto por los ítems de prioridad Must, junto con aquellos Should que resulten necesarios para garantizar una experiencia de usuario coherente.

Este enfoque asegura que el sistema pueda ser validado en etapas tempranas del desarrollo, reduciendo riesgos y facilitando la evolución progresiva del proyecto.

6.3 Definición del MVP

El Producto Mínimo Viable (MVP) de TechAssist se define como la versión inicial del sistema que incorpora el conjunto mínimo de funcionalidades necesarias para validar la propuesta de valor en un entorno real de uso.

El objetivo del MVP no es ofrecer una solución compleja, sino comprobar que el sistema es capaz de resolver el problema principal identificado: **la dificultad de los técnicos para seleccionar correctamente la herramienta adecuada en cada contexto operativo.**

Para la definición del MVP se han considerado los siguientes criterios:

- Priorización de funcionalidades con mayor impacto en el usuario.
- Reducción del esfuerzo de desarrollo para garantizar viabilidad.
- Validación temprana de la propuesta de valor.
- Coherencia con las historias de usuario principales.

6.3.1 Funcionalidades incluidas en el MVP

A partir del Product Backlog y la tabla impacto-esfuerzo, se han seleccionado las siguientes funcionalidades como núcleo del sistema:

1. Recomendación de herramienta (HU-01)

Funcionalidad principal del sistema que permite al técnico introducir una tarea y recibir una recomendación directa de la herramienta adecuada.

2. Explicación del criterio de recomendación (HU-02)

El sistema proporciona una justificación clara y en lenguaje no técnico sobre por qué se ha seleccionado dicha herramienta, aumentando la confianza del usuario.

3. Guía de primeros pasos (HU-03)

Se ofrece una guía estructurada en pasos simples que permite al técnico iniciar correctamente el proceso digital sin errores.

4. Minitest de nivel digital (HU-04)

El sistema evalúa el nivel de competencia digital del usuario para adaptar la complejidad de las explicaciones y mejorar la experiencia de uso.

5. Panel de gestión y supervisión (HU-05): Interfaz dedicada al perfil de supervisor que permite visualizar el rendimiento digital de los técnicos y el estado de las intervenciones.

6. Sistema de login y gestión básica de perfiles

Permite identificar el rol del usuario (técnico, supervisor o administrador) y adaptar la navegación dentro de la aplicación.

7. Base de datos local (SQLite)

Almacena herramientas, reglas, guías y datos del usuario, garantizando el funcionamiento offline del sistema.

8. Motor de recomendación basado en reglas

Componente central encargado de procesar la entrada del usuario y generar la recomendación correspondiente.

6.3.2 Funcionalidades excluidas del MVP

Las siguientes funcionalidades han sido identificadas como valiosas, pero se posponen para futuras versiones con el objetivo de reducir la complejidad inicial del sistema.

- **Analítica Predictiva Avanzada:** Procesamiento de Big Data para predecir necesidades de formación antes de que ocurran las incidencias.
- **Integración Bidireccional con ERP:** Automatización total donde n8n no solo recibe datos, sino que modifica órdenes de trabajo en el sistema central de la empresa.
- **Gamificación:** El sistema de insignias y niveles de experiencia para incentivar el uso de la aplicación.
- **Sincronización entre dispositivos:** En el MVP, los datos se almacenan de forma local en cada dispositivo. La consolidación de consultas de múltiples técnicos en un servidor central queda pendiente de la integración con n8n en versiones posteriores. La validación del sistema se realiza sobre un único dispositivo.

Estas funcionalidades se consideran parte de una fase de evolución del sistema una vez validado el MVP.

6.3.3 Alcance del MVP

El MVP de TechAssist se centra en ofrecer una experiencia funcional completa para el perfil de técnico de mantenimiento, permitiéndole:

- Identificar rápidamente qué herramienta utilizar.
- Comprender el motivo de la recomendación.
- Iniciar correctamente el proceso asociado.

El sistema operará de forma local en dispositivos Android, sin necesidad de conexión a internet, garantizando su uso en entornos industriales con limitaciones de conectividad.

6.3.4 Criterios de validación del MVP

La validez del MVP se evaluará en función de los siguientes indicadores:

- Reducción del tiempo necesario para iniciar tareas digitales.
- Disminución de errores en la selección de herramientas.
- Nivel de comprensión de las recomendaciones por parte del usuario.

- Grado de autonomía del técnico en el uso del sistema.

Estos criterios permitirán determinar si la solución propuesta cumple con los objetivos definidos y si es viable continuar con su evolución.

6.4 Sprints y fases del proyecto

El desarrollo del sistema TechAssist se ha organizado mediante un enfoque iterativo basado en sprints, siguiendo principios de metodologías ágiles adaptadas a un proyecto individual. Este enfoque permite dividir el desarrollo en bloques temporales manejables, facilitando la implementación progresiva de funcionalidades y la validación continua del sistema.

Cada sprint representa una iteración de desarrollo con un objetivo concreto, centrado en la entrega de un conjunto funcional del sistema. Esta organización permite priorizar las funcionalidades críticas del MVP y reducir el riesgo asociado al desarrollo.

Para este proyecto, se ha definido una planificación basada en sprints de duración aproximada de una a dos semanas, en función de la complejidad de las tareas incluidas.

6.4.1 Estructura de los sprints

La planificación del desarrollo se ha dividido en cinco sprints principales, alineados con la construcción progresiva del MVP:

Tabla 6.4.1 - Estructura de los sprints

Sprint	Objetivo principal	Funcionalidades
Sprint 1	Configuración base del sistema	Login, estructura MVC, base de datos SQLite
Sprint 2	Desarrollo del motor de recomendación	Reglas, lógica del agente, consulta de herramientas
Sprint 3	Diseño de interfaz y experiencia de usuario	Pantallas principales, flujo de navegación
Sprint 4	Implementación de funcionalidades clave	Explicación de recomendación y guía de uso
Sprint 5	Mejora, personalización y validación	Minitest, adaptación por nivel, pruebas

6.4.2 Descripción de los sprints

Sprint 1: Configuración base del sistema

En esta fase inicial se establece la estructura fundamental de la aplicación. Se implementa el sistema de autenticación básico, la arquitectura MVC y la base de datos local mediante SQLite. Este sprint sienta las bases sobre las que se construirá el resto del sistema.

Sprint 2: Desarrollo del motor de recomendación

Se desarrolla el núcleo funcional del sistema: el agente basado en reglas. Se implementa la lógica que permite procesar la entrada del usuario y asociarla con una herramienta específica. Este sprint es crítico, ya que materializa la propuesta de valor principal.

Sprint 3: Diseño de interfaz y experiencia de usuario

Se implementan las pantallas principales de la aplicación, incluyendo el dashboard del agente y los flujos de interacción definidos en el diseño UX. Se prioriza la simplicidad, la accesibilidad y la reducción de la carga cognitiva del usuario.

Sprint 4: Implementación de funcionalidades clave

Se incorporan funcionalidades esenciales que complementan la recomendación, como la explicación del criterio y la guía de primeros pasos. Estas funcionalidades refuerzan la usabilidad del sistema y aumentan la confianza del usuario.

Sprint 5: Mejora, personalización y validación

En este sprint final se implementa el minitest de nivel digital y la adaptación de contenido en función del perfil del usuario. Además, se realizan pruebas funcionales, ajustes de interfaz y validación del sistema en relación con los requisitos definidos.

6.4.3 Enfoque iterativo y validación continua

El enfoque basado en sprints permite validar progresivamente cada componente del sistema, asegurando que cada iteración aporta valor funcional. Esta estrategia facilita la detección temprana de errores, la mejora continua del diseño y la adaptación del sistema a las necesidades reales identificadas durante el desarrollo.

Asimismo, la construcción incremental del MVP permite disponer de versiones funcionales parciales del sistema, lo que resulta especialmente útil en un contexto académico donde es necesario demostrar avances tangibles en cada fase del proyecto.

6.5 Recursos y estimación temporal

En este apartado se definen los recursos necesarios para el desarrollo del proyecto TechAssist, así como una estimación temporal del esfuerzo requerido para la implementación del Producto Mínimo Viable (MVP).

Dado que se trata de un proyecto individual, la planificación se centra principalmente en la gestión del tiempo y en los recursos técnicos utilizados durante el desarrollo.

6.5.1 Recursos humanos

El desarrollo del proyecto ha sido realizado por un único desarrollador, que asume todos los roles necesarios dentro del ciclo de vida del software:

- Análisis de requisitos.
- Diseño de la solución.
- Desarrollo de la aplicación.
- Pruebas y validación.
- Documentación del proyecto.

Este enfoque implica una alta carga de trabajo, pero permite mantener una visión global del sistema y una coherencia en todas las fases del desarrollo.

6.5.2 Recursos técnicos

Para la implementación del sistema se han utilizado los siguientes recursos técnicos:

- **Equipo de desarrollo:** ordenador personal con capacidad suficiente para ejecutar el entorno de desarrollo Android Studio.
- **Entorno de desarrollo (IDE):** Android Studio.
- **Lenguaje de programación:** Kotlin.
- **Sistema de persistencia:** base de datos local SQLite.
- **Sistema operativo objetivo:** dispositivos Android (versión 8.0 o superior).
- **Herramientas de automatización:** n8n (para funcionalidades de integración y exportación de datos).
- **Control de versiones:** sistema de control de código (Git).

Estos recursos permiten desarrollar una aplicación robusta, optimizada para entornos industriales y con capacidad de funcionamiento offline.

6.5.3 Distribución del esfuerzo

El esfuerzo del proyecto se distribuye principalmente en tres grandes bloques:

- **Desarrollo del núcleo funcional (motor de recomendación):** mayor complejidad técnica y peso dentro del sistema.

- **Diseño e implementación de la interfaz de usuario:** clave para la adopción en entornos industriales.
- **Pruebas y validación:** necesarias para garantizar el cumplimiento de requisitos y la calidad del sistema.

Este reparto permite priorizar aquellas áreas que aportan mayor valor al usuario final.

6.6 Cronograma general

El cronograma general del proyecto TechAssist recoge la planificación temporal de las distintas fases de desarrollo, organizadas en base a los sprints definidos previamente. Su objetivo es proporcionar una visión global del avance del proyecto, permitiendo identificar la secuencia de actividades, la duración estimada de cada fase y los hitos principales alcanzados.

Dado que se trata de un desarrollo individual con enfoque iterativo, el cronograma se ha estructurado de forma flexible, permitiendo ajustes durante la ejecución sin comprometer los objetivos del Producto Mínimo Viable (MVP).

6.6.1 Estructura temporal del proyecto

El desarrollo se organiza en un total de cinco sprints, precedidos por una fase inicial de análisis y seguidos por una fase final de validación y documentación.

La duración total estimada del proyecto es de 10 a 12 semanas, distribuida de la siguiente forma:

Tabla 6.6.1 - Estructura temporal del proyecto

Fase	Descripción	Duración estimada
Análisis y definición	Estudio del problema, definición de requisitos y propuesta de valor	1 semanas
Sprint 1	Infraestructura base (SQLite, login, estructura del proyecto)	2 semanas
Sprint 2	Motor de recomendación y lógica del agente	2 semanas
Sprint 3	Interfaz de usuario y experiencia de interacción	2 semanas
Sprint 4	Funcionalidades complementarias y mejoras	2 semanas
Sprint 5	Pruebas, ajustes finales y redacción	1 semanas

6.6.2 Planificación por sprints

A continuación, se detalla la distribución de tareas principales a lo largo de los sprints:

Sprint 1 – Base del sistema

- Configuración del entorno de desarrollo.
- Implementación de la base de datos SQLite.
- Desarrollo del sistema de login y gestión de perfiles.
- Definición de la estructura del proyecto (MVC).

Hito: Sistema base funcional con persistencia de datos.

Sprint 2 – Núcleo del agente

- Desarrollo del motor de reglas.
- Implementación del sistema de recomendación de herramientas.
- Generación de explicaciones asociadas a cada recomendación.

Hito: Agente capaz de recomendar herramientas correctamente.

Sprint 3 – Interfaz y experiencia de usuario

- Diseño e implementación de la interfaz principal.
- Desarrollo del flujo de interacción con el agente.
- Implementación de la guía de primeros pasos.

Hito: Aplicación usable con flujo completo técnico-agente.

Sprint 4 – Funcionalidades adicionales

- Desarrollo del minitest de nivel digital.
- Adaptación de contenido según nivel.
- Implementación del registro de consultas.
- Desarrollo básico del panel de supervisión.

Hito: MVP completo con funcionalidades principales operativas.

Sprint 5 – Pruebas ajustes y redacción

- Ejecución del plan de pruebas funcionales.
- Resolución de incidencias detectadas durante la validación.
- Ajustes de rendimiento y verificación de indicadores técnicos del MVP.

Hito: MVP validado, estable y documentado.

6.6.3 Representación del cronograma

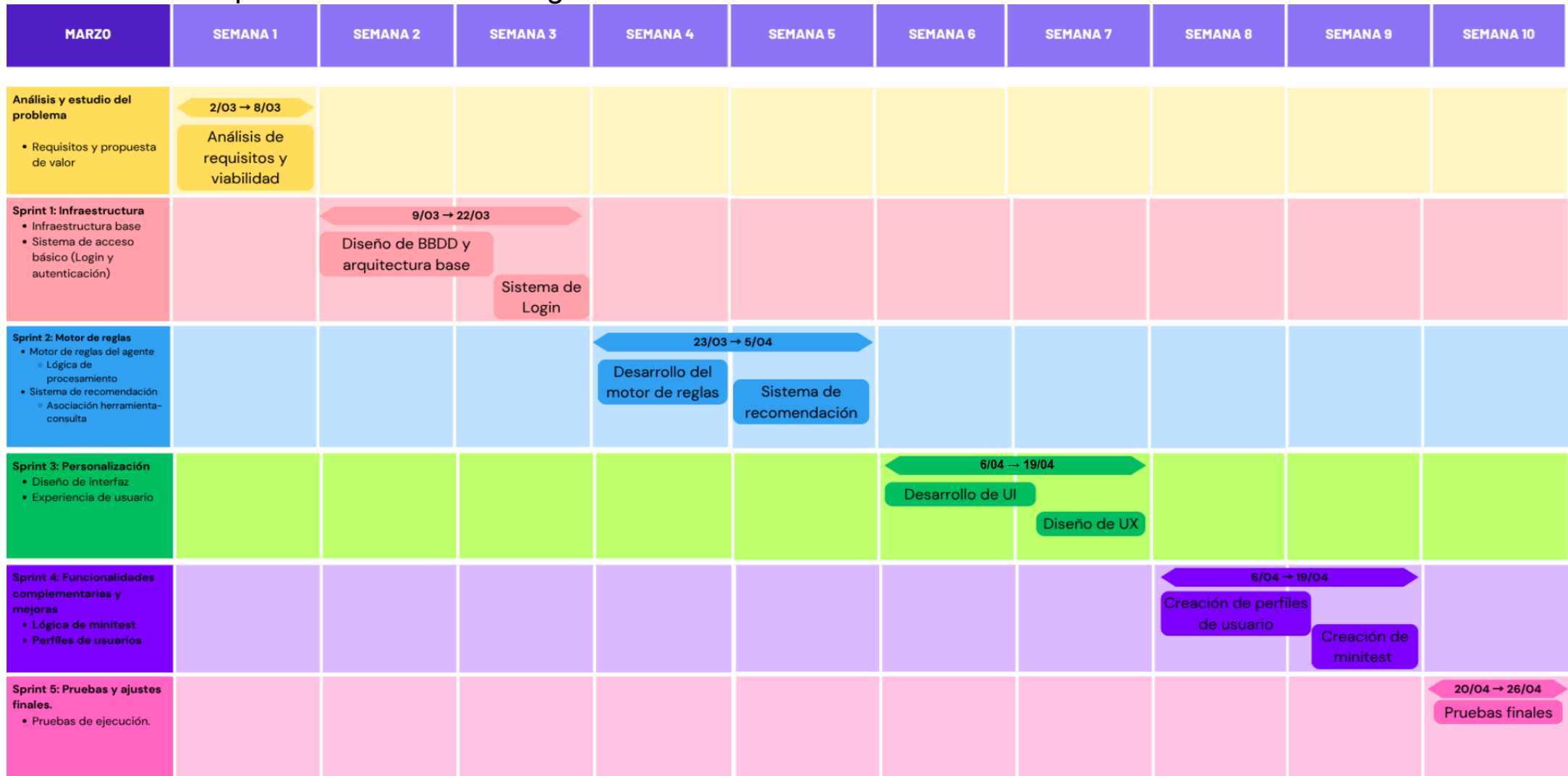


Figura 6.6.3 - Cronograma de desarrollo del proyecto estructurado por Sprints

El diagrama de Gantt presentado refleja una planificación detallada de **10 semanas de desarrollo**, estructuradas bajo una metodología ágil adaptada. La estrategia se basa en los siguientes puntos clave:

- **Ciclo de Vida en Cascada Agrupada:** Se ha optado por una progresión lógica donde la infraestructura y el motor de reglas (Sprints 1 y 2) actúan como cimientos críticos antes de avanzar hacia la interfaz de usuario.
- **Distribución de Sprints:** Se han definido **4 Sprints de 2 semanas** cada uno y 1 Sprint de una semana. Esta cadencia permite una validación constante de los requisitos funcionales y una gestión de riesgos eficiente, asegurando que el núcleo lógico (el recomendador) esté operativo desde las fases intermedias.
- **Hitos de Control:** Cada final de sprint marca un hito de entrega funcional, culminando en el **Sprint 5** con la integración de flujos externos mediante n8n y el control de calidad final (QA).
- **Paralelismo en Documentación:** La tarea de documentación se extiende durante todo el ciclo de vida del proyecto, garantizando que la memoria técnica refleje fielmente el proceso de desarrollo y las decisiones arquitectónicas tomadas en tiempo real.

6.6.4 Hitos principales del proyecto

A lo largo del desarrollo se identifican los siguientes hitos clave:

- Definición completa de requisitos del sistema.
- Implementación de la base de datos y arquitectura.
- Desarrollo del motor de recomendación.
- Implementación de la interfaz de usuario funcional.
- Finalización del MVP.
- Validación y cierre del proyecto.

6.6.5 Consideraciones del cronograma

El cronograma ha sido diseñado teniendo en cuenta:

- La naturaleza iterativa del desarrollo.
- La disponibilidad de recursos (proyecto individual).
- La prioridad de las funcionalidades definidas en el MVP.
- La necesidad de incluir tiempo para pruebas y ajustes.

Asimismo, se contempla la posibilidad de realizar ajustes en la planificación en función de la complejidad técnica encontrada durante el desarrollo, manteniendo siempre el foco en la entrega de un MVP funcional y validable.

6.7 Criterios de aceptación

Los criterios de aceptación definen las condiciones que debe cumplir cada funcionalidad del sistema para considerarse correctamente implementada. Estos criterios permiten validar de forma objetiva el comportamiento del sistema durante la fase de pruebas, asegurando que la solución desarrollada responde a los requisitos definidos previamente.

En el contexto del proyecto TechAssist, los criterios de aceptación se derivan directamente de las historias de usuario y de los requisitos funcionales, garantizando la trazabilidad entre el análisis, el diseño y la implementación.

6.7.1 Objetivo de los criterios de aceptación

El establecimiento de criterios de aceptación persigue los siguientes objetivos:

- Verificar que cada funcionalidad cumple con lo especificado.
- Reducir ambigüedades en la interpretación de requisitos.
- Facilitar el diseño del plan de pruebas.
- Garantizar la calidad del MVP desarrollado.
- Servir como referencia para la validación final del sistema.

6.7.2 Criterios de aceptación del MVP

A continuación, se presentan los criterios de aceptación asociados a las funcionalidades principales incluidas en el MVP:

Funcionalidad: Recomendación de herramienta

- El sistema permite introducir una descripción de la tarea o seleccionar una categoría.
- El sistema devuelve una única herramienta como recomendación principal.
- La recomendación incluye el nombre identificativo de la herramienta.
- El tiempo de respuesta es inferior a 10 segundos.
- La recomendación es coherente con las reglas definidas en la base de conocimiento.

Funcionalidad: Explicación del criterio de recomendación

- Existe una opción visible para consultar la explicación.
- La explicación se muestra de forma inmediata tras la solicitud.
- El contenido está redactado en lenguaje claro y comprensible.
- La explicación corresponde a la regla aplicada en la recomendación.

Funcionalidad: Guía de primeros pasos

- La guía se muestra automáticamente junto con la recomendación.
- Contiene entre uno y tres pasos claramente numerados.
- La información es comprensible para usuarios con nivel digital básico.
- La guía está disponible sin conexión a internet.

- Los pasos permiten iniciar correctamente el proceso en la herramienta recomendada.

Funcionalidad: Minitest de nivel digital

- El sistema presenta entre 5 y 10 preguntas cerradas.
- El usuario puede completar el test sin interrupciones.
- El sistema calcula automáticamente una puntuación.
- Se asigna un nivel digital (básico, intermedio o avanzado).
- El resultado se almacena en el dispositivo.

Funcionalidad: Adaptación al nivel digital

- El sistema adapta la complejidad de las explicaciones según el nivel del usuario.
- Los usuarios con nivel básico reciben instrucciones más detalladas.
- Los usuarios con nivel avanzado reciben información más concisa.

Funcionalidad: Persistencia de datos

- La información se almacena correctamente en la base de datos SQLite.
- Los datos persisten tras cerrar y volver a abrir la aplicación.
- No se producen pérdidas de información durante el uso normal.

6.7.3 Validación de requisitos no funcionales

Además de las funcionalidades, se establecen criterios para validar los requisitos no funcionales más relevantes:

- **Rendimiento:**
El sistema responde en menos de 10 segundos en dispositivos de gama media.
- **Usabilidad:**
La interfaz permite completar una consulta sin necesidad de formación previa.
- **Disponibilidad offline:**
Las funcionalidades principales (recomendación y guía) están disponibles sin conexión.
- **Robustez:**
El sistema gestiona entradas incorrectas sin provocar fallos o cierres inesperados.

6.7.4 Procedimiento de verificación

La verificación de los criterios de aceptación se realizará mediante:

- Pruebas funcionales manuales sobre cada caso de uso.
- Simulación de escenarios reales de uso en entorno industrial.
- Validación del comportamiento del sistema frente a distintos niveles de usuario.
- Comprobación de persistencia de datos y funcionamiento offline.

Al finalizar cada sprint se **verificaba** su resultado antes de continuar con el siguiente bloque de desarrollo. El criterio para dar por válido un sprint era la **correcta ejecución manual de todos los casos de uso asociados a las funcionalidades implementadas** en esa iteración, comprobando que el flujo completo funcionaba sin errores en el emulador. El **Sprint 1** se dio por cerrado cuando el login diferenciaba correctamente los **tres roles** y la base de datos se **inicializaba con los datos del seed**. El **Sprint 2** quedó validado cuando el agente devolvía **recomendaciones coherentes para las ocho categorías**. El **Sprint 3** se verificó **navegando el flujo completo técnico-agente sin errores de interfaz**. El **Sprint 4** se cerró al confirmar que el **minitest asignaba el nivel correctamente** y que el **panel del supervisor mostraba los datos de consultas**. El **Sprint 5** quedó validado con la **ejecución del plan de pruebas descrito en el capítulo 10**, comprobando el cumplimiento de los criterios de aceptación definidos en la sección **6.7.2**.

6.7.5 Relación con la validación del proyecto

Los criterios de aceptación constituyen la base para la fase de pruebas y validación descrita en capítulos posteriores. Su cumplimiento permitirá determinar si el sistema desarrollado cumple con los objetivos definidos y si la propuesta de valor se materializa de forma efectiva en el MVP.

De este modo, se garantiza que la evaluación del proyecto no se basa únicamente en la implementación técnica, sino también en la capacidad real del sistema para resolver el problema identificado en el contexto de uso.

6.8 Gestión de cambios y desviaciones.

Durante el desarrollo del proyecto TechAssist se produjeron ajustes de alcance respecto a la planificación inicial, motivados por criterios de viabilidad técnica, priorización de valor y coherencia con los objetivos del MVP. A continuación se detallan las desviaciones más relevantes y las decisiones adoptadas en cada caso.

6.8.1 Exclusión de la integración con n8n.

En la **planificación inicial (Sprint 4 y PB-13)**, se contemplaba la integración con **n8n** para la exportación automática de reportes del panel de supervisión mediante **Webhook**. Durante el desarrollo se decidió excluir esta funcionalidad del **MVP** por dos razones: la **lógica del agente y el motor de recomendación local** cubrían completamente el problema principal del usuario sin necesidad de conectividad externa, y la integración con un backend externo **aumentaba la complejidad** y el **riesgo técnico** sin aportar valor al **núcleo del sistema**. El botón de exportación se mantiene en la interfaz del panel de supervisión como elemento planificado, pero permanece desactivado sin conexión y sin lógica de envío real, reservándose como **ampliación futura explícita**.

6.8.2 Exclusión de la gamificación

El sistema de **insignias** y **niveles de experiencia** contemplado en el backlog (**tabla impacto-esfuerzo, sección 4.5**) fue descartado del MVP tras la priorización. Se determinó que el valor aportado al usuario principal, el técnico de mantenimiento, era secundario respecto a la **fiabilidad** del recomendador y la **calidad** de las guías. El nivel digital asignado por el minitest actúa como **elemento motivacional básico**, cubriendo parcialmente este objetivo sin añadir complejidad innecesaria.

6.8.3 Incorporación de funcionalidades no planificadas inicialmente

Durante el desarrollo surgieron **mejoras no contempladas** en el **backlog original** que se incorporaron por su impacto directo en la usabilidad. Los checkboxes secuenciales de la guía de primeros pasos, que obligan al técnico a confirmar cada paso antes de desbloquear el siguiente, se añadieron durante el **Sprint 3** al detectar que una **guía sin interacción no garantizaba la lectura efectiva de los pasos**. Asimismo, la detección reactiva de conectividad mediante **NetworkCallback** se incorporó en sustitución de la comprobación puntual inicial, mejorando la respuesta del banner offline en tiempo real.

6.8.4 Ajuste de banco de preguntas del minitest

El diseño inicial contemplaba entre **5 y 10 preguntas fijas**. Durante el desarrollo se optó por un **banco de 12 preguntas** con **selección aleatoria de 6** en cada sesión, garantizando que distintos usuarios o sesiones no repitan exactamente el mismo cuestionario, mejorando la validez de la evaluación sin aumentar la duración percibida del test.

7. Gestión de riesgos, seguridad y cumplimiento

Este capítulo aborda los riesgos asociados al desarrollo y uso del sistema TechAssist, así como las medidas adoptadas para garantizar su seguridad, fiabilidad y adecuación a principios éticos y normativos.

El objetivo es anticipar posibles problemas que puedan afectar al funcionamiento del sistema o a su adopción por parte de los usuarios, definiendo estrategias de mitigación que reduzcan su impacto. Asimismo, se establecen las bases para asegurar un uso responsable de la tecnología, especialmente en entornos industriales donde la precisión y la fiabilidad son críticas.

7.1 Identificación de riesgos

La identificación de riesgos se ha realizado considerando tanto aspectos técnicos como operativos y humanos. A continuación, se describen los principales riesgos detectados:

7.1.1 Riesgos técnicos

- **Errores en el motor de reglas:** una configuración incorrecta podría generar recomendaciones inadecuadas.
- **Fallo en la persistencia de datos:** posibles problemas en SQLite que afecten al almacenamiento.
- **Problemas de rendimiento:** tiempos de respuesta elevados en dispositivos con recursos limitados.
- **Incompatibilidad de dispositivos:** funcionamiento incorrecto en distintas versiones de Android.

7.1.2 Riesgos operativos

- **Uso incorrecto de la aplicación:** el usuario puede interpretar mal la recomendación.
- **Dependencia excesiva del sistema:** el técnico podría dejar de aplicar criterio propio.
- **Datos desactualizados:** la base de herramientas puede quedar obsoleta si no se mantiene.

7.1.3 Riesgos humanos

- **Resistencia al cambio:** rechazo inicial por parte de técnicos con baja alfabetización digital.
- **Falta de confianza en el sistema:** dudas sobre la fiabilidad de las recomendaciones.
- **Errores de introducción de datos:** descripciones ambiguas o incorrectas por parte del usuario.

7.2 Evaluación de impacto y plan de mitigación

Una vez identificados los principales riesgos asociados al desarrollo y uso del sistema TechAssist, es necesario evaluar su impacto potencial y definir estrategias de mitigación que permitan reducir su probabilidad de ocurrencia o minimizar sus consecuencias.

La evaluación se realiza considerando dos dimensiones principales:

- **Impacto:** grado en que el riesgo afecta al funcionamiento del sistema o a la experiencia del usuario
- **Probabilidad:** posibilidad de que el riesgo llegue a materializarse durante el uso del sistema.

A partir de esta evaluación, se establecen medidas de mitigación específicas para cada riesgo identificado:

Riesgo 1: Recomendaciones incorrectas del sistema

- **Impacto:** Alto
- **Probabilidad:** Media

El agente de recomendación podría sugerir una herramienta no adecuada para la tarea introducida, generando errores operativos o pérdida de confianza en el sistema.

Plan de mitigación:

- Definición clara y controlada de reglas en la base de conocimiento.
- Validación manual inicial de las reglas por parte de perfiles expertos.
- Inclusión de explicaciones para aumentar la transparencia.
- Posibilidad de ajustar o corregir reglas desde el perfil administrador.
- Iteración progresiva del sistema en base al uso real.

Riesgo 2: Baja adopción por parte de los técnicos

- **Impacto:** Alto
- **Probabilidad:** Media-Alta

Los técnicos pueden mostrar resistencia al uso de la aplicación debido a desconfianza en herramientas o preferencia por métodos tradicionales.

Plan de mitigación:

- Diseño de interfaz altamente intuitiva y simplificada (UX centrada en el usuario).
- Reducción de pasos necesarios para obtener una recomendación.
- Adaptación del contenido al nivel digital del usuario.
- Inclusión de guías claras y de fácil comprensión.
- Validación temprana del diseño mediante pruebas con usuarios reales.

Riesgo 3: Rendimiento insuficiente en dispositivos industriales

- **Impacto:** Medio
- **Probabilidad:** Media

El sistema puede presentar tiempos de respuesta elevados o consumo excesivo de recursos en dispositivos con hardware limitado.

Plan de mitigación:

- Uso de procesamiento local optimizado (sin dependencia constante de red).
- Implementación eficiente en Kotlin con control de recursos.
- Pruebas de rendimiento en dispositivos con especificaciones bajas.
- Simplificación del motor de reglas para evitar sobrecarga computacional.

Riesgo 4: Errores en la base de datos o pérdida de información

- **Impacto:** Medio-Alto
- **Probabilidad:** Baja-Media

Posibles inconsistencias en la base de datos local (SQLite) que afecten a las recomendaciones o a los registros de consultas.

Plan de mitigación:

- Validación de integridad en operaciones CRUD.
- Uso de claves primarias y foráneas para asegurar consistencia.
- Implementación de mecanismos de control de errores.
- Posibilidad de exportación o backup de datos en futuras versiones.

Riesgo 5: Dependencia excesiva del sistema por parte del usuario

- **Impacto:** Medio
- **Probabilidad:** Baja

El usuario puede llegar a depender completamente del sistema, reduciendo su capacidad de toma de decisiones autónoma.

Plan de mitigación:

- Inclusión de explicaciones que fomenten el aprendizaje.
- Diseño orientado a apoyo, no sustitución del conocimiento.
- Refuerzo del sistema como herramienta complementaria.

Riesgo 6: Limitaciones del enfoque basado en reglas

- **Impacto:** Medio
- **Probabilidad:** Media

El sistema basado en reglas puede no cubrir todos los casos posibles o situaciones no previstas.

Plan de mitigación:

- Diseño flexible de la base de reglas para facilitar ampliaciones.
- Posibilidad de añadir nuevas categorías y herramientas fácilmente.
- Identificación de lagunas mediante el registro de consultas.
- Evolución futura hacia modelos más avanzados si fuera necesario.

Conclusión de la evaluación:

El análisis de riesgos muestra que los principales desafíos del sistema no se encuentran en su viabilidad técnica, sino en aspectos relacionados con la adopción por parte del usuario y la calidad de las recomendaciones.

Sin embargo, mediante un diseño centrado en el usuario, una arquitectura simple y controlada y, un enfoque iterativo de mejora continua, estos riesgos pueden ser mitigados de forma efectiva.

El planteamiento del MVP permite además validar tempranamente estos aspectos, reduciendo la incertidumbre y facilitando la evolución progresiva del sistema en futuras versiones.

7.3 Seguridad y protección de datos

La seguridad y la protección de los datos constituyen un aspecto fundamental en el desarrollo del sistema TechAssist, especialmente considerando que la aplicación gestiona información relacionada con usuarios, patrones de uso y procesos operativos dentro de un entorno industrial.

Dado que el sistema ha sido diseñado como una aplicación móvil con funcionamiento principalmente local, la estrategia de seguridad se centra en minimizar la exposición de datos, garantizar su integridad y controlar el acceso a las funcionalidades críticas.

7.3.1 Enfoque de seguridad del sistema

El diseño de TechAssist adopta un enfoque de seguridad basado en los siguientes principios:

- **Minimización de datos:**
El sistema únicamente almacena la información necesaria para su funcionamiento, evitando la recopilación de datos personales sensibles.
- **Procesamiento local:**
La mayor parte de la lógica del sistema, incluyendo el motor de recomendación y la persistencia de datos, se ejecuta en el dispositivo mediante SQLite, reduciendo la dependencia de servicios externos y el riesgo de exposición.
- **Acceso controlado:**
Las funcionalidades críticas, como la gestión de herramientas y reglas, están restringidas a perfiles autorizados.
- **Simplicidad y robustez:**
Se evita la complejidad innecesaria en la gestión de datos para reducir posibles vulnerabilidades.

7.3.2 Gestión de datos almacenados

Los datos gestionados por la aplicación se almacenan localmente en una base de datos SQLite e incluyen:

- Información básica de usuarios (identificador, rol, nivel digital).
- Base de herramientas.
- Reglas de recomendación.
- Guías de uso.
- Registro de consultas (sin información personal sensible).

Para garantizar la protección de esta información:

- El acceso a la base de datos está limitado al contexto de la aplicación.
- No se permite el acceso directo por parte de otras aplicaciones.
- Se asegura la integridad de los datos mediante operaciones controladas de inserción, actualización y eliminación.

7.3.3 Control de acceso y autenticación

El sistema implementa un mecanismo de autenticación local que permite diferenciar entre los distintos perfiles de usuario:

- Técnico de mantenimiento.
- Supervisor.
- Administrador.

Las funcionalidades sensibles, como la gestión de la base de herramientas o la modificación de reglas, están restringidas exclusivamente al perfil administrador.

Este control de acceso evita modificaciones no autorizadas en la base de conocimiento del sistema y garantiza la coherencia de las recomendaciones generadas por el agente.

7.3.4 Protección en la comunicación de datos

Aunque el sistema funciona principalmente en local, se contempla la comunicación externa mediante la integración con n8n para la exportación de datos agregados.

En este contexto, se aplican las siguientes medidas:

- **Transmisión de datos no sensibles:**
Únicamente se envían datos agregados y anonimizados, evitando cualquier información identificativa del usuario.
- **Comunicación mediante Webhooks controlados:**
Las solicitudes se realizan a endpoints definidos y gestionados de forma segura.
- **Procesamiento externo desacoplado:**
La lógica de envío y tratamiento de datos se delega en n8n, reduciendo la complejidad y la superficie de ataque en la aplicación móvil.

7.3.5 Integridad y consistencia de la información

El sistema garantiza la integridad de los datos mediante:

- Validación de entradas del usuario.
- Control de operaciones en la base de datos.
- Restricción de modificaciones a usuarios autorizados.

Además, el uso de un modelo basado en reglas permite mantener la trazabilidad de las decisiones del sistema, evitando comportamientos no deterministas.

7.3.6 Consideraciones de cumplimiento

Aunque TechAssist no gestiona datos personales sensibles, el diseño del sistema sigue principios alineados con normativas de protección de datos como el Reglamento General de Protección de Datos (RGPD):

- Minimización de datos recogidos.
- Uso de información anonimizada en procesos de análisis.
- Transparencia en el uso de la información dentro del sistema.

7.3.7 Evaluación global de seguridad

El enfoque adoptado permite considerar que el sistema presenta un nivel adecuado de seguridad para el alcance del MVP, dado que:

- Reduce la exposición de datos al operar principalmente en local.
- Limita el acceso a funcionalidades críticas.
- Evita el tratamiento de información sensible.
- Mantiene un control claro sobre los flujos de información.

En futuras versiones del sistema, se podrán incorporar mejoras adicionales, como cifrado de la base de datos local o mecanismos avanzados de autenticación, en función de los requisitos del entorno de despliegue.

7.4 Consideraciones éticas y accesibilidad

El desarrollo de TechAssist no solo implica la resolución de un problema operativo en entornos industriales, sino también la responsabilidad de garantizar que la solución respete principios éticos fundamentales y sea accesible para todos los usuarios, independientemente de su nivel de competencia digital.

Dado que la aplicación actúa como un sistema de apoyo en la toma de decisiones operativas, es necesario considerar el impacto que puede tener en el comportamiento de los usuarios, en la organización del trabajo y en la adopción de herramientas dentro del entorno industrial.

7.4.1 Consideraciones éticas

Uno de los principios clave en el diseño de TechAssist es que el sistema actúa como una herramienta de asistencia y no como un sustituto del criterio profesional técnico. Las recomendaciones generadas por el agente están basadas en reglas definidas previamente, por lo que su función es orientar, no imponer decisiones. En este sentido, se evita cualquier automatismo que pueda desplazar la responsabilidad operativa del usuario.

Asimismo, se ha tenido en cuenta la necesidad de garantizar la transparencia en el funcionamiento del sistema. Por ello, una de las funcionalidades principales del MVP es la explicación del criterio de recomendación, permitiendo al usuario comprender por qué se le sugiere una herramienta concreta. Este enfoque refuerza la confianza en el sistema y evita la percepción de “caja negra”.

En relación con el tratamiento de datos, el sistema ha sido diseñado para minimizar la recogida de información personal. Los datos almacenados se limitan a información funcional necesaria para el funcionamiento del agente, como el nivel digital del usuario o el registro de consultas, evitando el uso de datos sensibles o innecesarios.

Además, la monitorización de consultas se realiza de forma agregada y anonimizada, especialmente en el caso de los perfiles de supervisión, con el objetivo de analizar patrones de uso sin evaluar el rendimiento individual de los trabajadores. Este enfoque reduce el riesgo de uso indebido de la información con fines de control excesivo o evaluación individual no justificada.

Por último, se ha considerado el impacto organizativo de la herramienta. TechAssist busca reducir la dependencia del supervisor en tareas básicas, pero sin eliminar la colaboración entre trabajadores, promoviendo un entorno de trabajo más autónomo y eficiente sin afectar negativamente a la dinámica de equipo.

7.4.2 Accesibilidad y usabilidad inclusiva

La accesibilidad constituye un aspecto fundamental en el diseño de TechAssist, especialmente teniendo en cuenta la heterogeneidad en el nivel de alfabetización digital de los usuarios identificados en el análisis previo.

El sistema ha sido diseñado siguiendo principios de usabilidad inclusiva, con el objetivo de garantizar que pueda ser utilizado por técnicos con distintos niveles de experiencia tecnológica. Entre las principales medidas adoptadas destacan:

- **Interfaz simplificada y clara:** se prioriza una estructura visual limpia, con una jerarquía de información bien definida que facilita la comprensión rápida de las acciones disponibles.
- **Elementos de interacción sobredimensionados:** los botones y áreas táctiles están diseñados con un tamaño superior al estándar, facilitando su uso en entornos industriales donde el operario puede utilizar guantes o encontrarse en movimiento.
- **Lenguaje no técnico:** las explicaciones y guías se presentan en un lenguaje claro y accesible, evitando terminología compleja que pueda dificultar la comprensión.
- **Adaptación al nivel digital del usuario:** el sistema ajusta automáticamente la profundidad de las explicaciones y el nivel de detalle de las guías en función del resultado del minitest inicial, permitiendo una experiencia personalizada.
- **Reducción de carga cognitiva:** se limita el número de opciones visibles en cada interacción y se estructura la información en pasos simples, facilitando la toma de decisiones en situaciones de presión operativa.
- **Disponibilidad offline:** la posibilidad de acceder a las guías sin conexión a internet garantiza que todos los usuarios puedan utilizar la aplicación en cualquier punto de la planta, independientemente de la conectividad.

Estas medidas permiten que TechAssist no solo sea funcional, sino también accesible para una amplia variedad de perfiles, contribuyendo a reducir la brecha digital dentro del entorno industrial.

7.4.3 Impacto en la adopción tecnológica

Finalmente, se ha considerado que la accesibilidad y el enfoque ético del sistema tienen un impacto directo en la adopción de la herramienta. Al reducir la complejidad percibida y aumentar la confianza en el sistema, se facilita la integración de TechAssist en la operativa diaria de los técnicos.

Este enfoque contribuye a minimizar el rechazo a la digitalización, uno de los problemas identificados en el análisis inicial, y favorece una transición más natural hacia el uso de herramientas en el entorno de mantenimiento industrial.

7.5 Licencias y sostenibilidad

El desarrollo de TechAssist, como aplicación de asistencia operativa en entornos industriales, requiere considerar tanto los aspectos relacionados con el uso de tecnologías y licencias como la viabilidad y sostenibilidad de la solución a medio y largo plazo.

7.5.1 Licencias de software

El proyecto se ha desarrollado utilizando herramientas y tecnologías ampliamente utilizadas en el ecosistema de desarrollo Android, muchas de ellas bajo licencias abiertas o de uso gratuito para fines educativos y de desarrollo.

Entre los principales componentes utilizados destacan:

- **Android Studio** como entorno de desarrollo integrado (IDE), distribuido bajo licencia gratuita por Google.
- **Kotlin** como lenguaje de programación, de código abierto y oficialmente soportado para el desarrollo Android.
- **SQLite** como sistema de base de datos local, también de código abierto.
- **n8n**, en el caso de su uso conceptual para automatización, basado en un modelo de código abierto con opciones de despliegue autoalojado.

El uso de estas tecnologías permite desarrollar la solución sin incurrir en costes de licencias, lo que facilita tanto su desarrollo académico como su posible escalado en entornos reales.

En cuanto al propio software desarrollado (TechAssist), al tratarse de un proyecto académico, se plantea como software de carácter demostrativo. No obstante, en un escenario real, podría contemplarse su distribución bajo una licencia de tipo:

- **Propietaria**, si se integra dentro de un ecosistema empresarial cerrado.
- **Código abierto**, si se desea fomentar su adaptación y evolución por parte de la comunidad o de equipos internos.

La elección de la licencia dependería del modelo de negocio y del contexto organizativo en el que se implante la solución.

7.5.2 Sostenibilidad técnica

Desde el punto de vista técnico, TechAssist ha sido diseñado siguiendo principios que favorecen su mantenibilidad y evolución:

- **Arquitectura modular (MVC)**: facilita la separación de responsabilidades y permite modificar componentes sin afectar al conjunto del sistema.
- **Uso de tecnologías estándar**: garantiza disponibilidad de documentación, soporte y facilidad de incorporación de nuevos desarrolladores.

- **Base de datos local desacoplada:** permite adaptar fácilmente el sistema a cambios en las herramientas o en las reglas del agente.

Además, el enfoque basado en reglas facilita la actualización del sistema sin necesidad de modificar la lógica completa, permitiendo adaptar las recomendaciones a cambios en los procesos de la empresa.

7.5.3 Sostenibilidad operativa

Desde el punto de vista organizativo, la sostenibilidad del sistema depende principalmente de su capacidad de adaptación al entorno industrial y de su facilidad de uso.

TechAssist ha sido concebido como una herramienta ligera, sin necesidad de integraciones complejas con sistemas existentes (como GMAO o ERP), lo que reduce significativamente los costes de implantación y mantenimiento.

Asimismo, la posibilidad de gestionar las herramientas y las reglas del agente desde un perfil administrador permite que el sistema evolucione junto con la organización, sin depender exclusivamente de desarrollos técnicos continuos.

Otro factor clave es su enfoque en la mejora de la autonomía del usuario, lo que contribuye a reducir la carga operativa sobre supervisores y departamentos de soporte, generando un impacto positivo en la eficiencia global del sistema.

7.5.4 Sostenibilidad económica

En su versión MVP, el coste de desarrollo de TechAssist es reducido, al basarse en tecnologías gratuitas y en un alcance funcional acotado.

En un escenario de implantación real, los principales costes asociados serían:

- Desarrollo y evolución de nuevas funcionalidades.
- Mantenimiento de la base de conocimiento (herramientas y reglas).
- Formación inicial de los usuarios.
- Posible infraestructura adicional en caso de integración con sistemas externos.

Sin embargo, estos costes pueden verse compensados por los beneficios esperados:

- Reducción de errores en el uso de herramientas.
- Disminución de tiempos muertos operativos.
- Menor dependencia del soporte humano.
- Mejora en la calidad de los datos registrados.

7.5.5 Escalabilidad y evolución futura

Aunque el MVP se centra en funcionalidades básicas (recomendación, guía inicial y adaptación al usuario), la arquitectura del sistema permite su evolución hacia versiones más completas.

Entre las posibles líneas de evolución destacan:

- Integración con sistemas corporativos (GMAO, ERP).
- Incorporación de analítica avanzada sobre uso y comportamiento.
- Mejora del agente mediante técnicas híbridas (reglas + aprendizaje automático).
- Expansión a otros entornos operativos más allá del mantenimiento industrial.

Este enfoque progresivo permite que TechAssist pueda crecer de forma controlada, manteniendo la coherencia con su objetivo inicial como herramienta de asistencia ligera.

8.Desarrollo del MVP

El presente capítulo constituye el núcleo de la memoria, donde se detalla el proceso de construcción y materialización de TechAssist. Una vez definidos los requisitos funcionales y el diseño arquitectónico en los capítulos previos, en este apartado se describe cómo se han implementado dichas especificaciones para dar vida al Producto Mínimo Viable (MVP).

El desarrollo se ha centrado en transformar la lógica del agente inteligente y la base de conocimiento en una herramienta operativa, capaz de interactuar con el técnico de mantenimiento en condiciones reales de planta. Para ello, se ha seguido un enfoque de desarrollo ágil, priorizando la estabilidad del sistema y la eficiencia en el consumo de recursos, garantizando que el motor de recomendación responda con agilidad incluso sin conectividad a la red.

A lo largo de las siguientes secciones, se desglosa el ecosistema tecnológico utilizado, la implementación detallada de cada una de las funcionalidades clave (desde el test de nivel digital hasta el panel de supervisión) y la resolución de los principales retos técnicos encontrados durante la fase de codificación. El objetivo final de este capítulo es demostrar la viabilidad técnica de la solución y su capacidad para resolver los problemas de brecha digital identificados en el análisis.

Repositorio de GitHub

El repositorio remoto se encuentra en **GitHub**:

<https://github.com/Eruizap14/TechAssist-app>

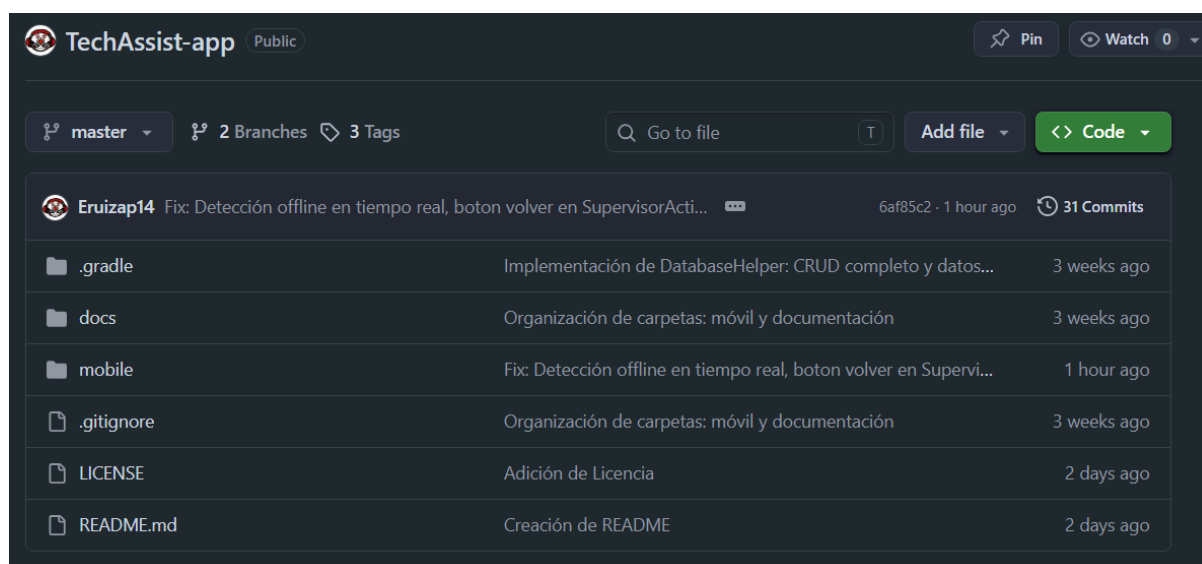


Figura 8 - Repositorio remoto oficial del proyecto TechAssist en GitHub

8.1 Entorno de desarrollo

Para la materialización de TechAssist, se ha seleccionado un stack tecnológico orientado específicamente al rendimiento en dispositivos móviles y a la fiabilidad en entornos industriales. Los componentes principales del entorno son:

- **IDE (Entorno de Desarrollo Integrado):** Se ha utilizado Android Studio (Panda 4 2025.3.4). Esta versión permite una gestión avanzada de la memoria y perfiles de rendimiento, fundamentales para asegurar que la aplicación sea ligera y no consuma recursos excesivos en terminales industriales (cumpliendo con el RNF-02).
- **Lenguaje de programación:** Se ha empleado Kotlin de forma integral en el proyecto. Esta decisión responde a una estrategia de “**Kotlin Full Stack**”, utilizando el lenguaje tanto para el desarrollo de la aplicación móvil (Frontend) como para la lógica del servidor y la gestión de datos (Backend).
 - **Ventajas en el proyecto:** El uso de Kotlin permite compartir modelos de datos entre la app y el servidor, reducir la curva de aprendizaje y aprovechar la seguridad frente a nulos (Null Safety) en todo el ecosistema del software. Además, su compatibilidad nativa con Corrutinas facilita la ejecución de tareas en segundo plano sin penalizar la interfaz de usuario en entornos críticos.
- **Gestión de Interfaz (UI):** El diseño se ha implementado mediante Layouts XML. Se ha optado por este enfoque para garantizar un control total sobre la jerarquía de vistas, permitiendo crear elementos visuales sobredimensionados (botones y campos de texto de alto contraste) que facilitan la interacción de operarios que utilicen equipos de protección individual (EPIs).
- **Base de Datos Local:** Se utiliza SQLite. Este componente es el núcleo que garantiza la operatividad offline (cumpliendo con el RNF-01), permitiendo que el motor de reglas y la información del técnico persistan en el dispositivo sin dependencia de servidores externos durante la jornada laboral.

8.2 Descripción funcional del MVP

El Producto Mínimo Viable (**MVP**) de TechAssist se ha diseñado e implementado como una solución nativa ligera orientada a entornos industriales de baja conectividad. Su objetivo principal es mitigar la fatiga cognitiva del operario de la planta y agilizar la toma de decisiones en el punto de ejecución de la tarea. Funcionalmente, la aplicación segmenta sus capacidades en cuatro módulos clave e interconectados que cubren el ciclo completo desde la caracterización del usuario hasta la auditoría y el análisis de datos por parte de la línea de supervisión.

A nivel técnico, el sistema aprovecha una arquitectura donde la lógica de control reactiva en Kotlin interactúa directamente con un motor de persistencia local SQLite optimizado, garantizando tiempos de respuesta mínimos y total autonomía offline.

TECHASSIST

☐ Recordar selección

→ Acceder

Figura 8.2 - Interfaz de usuario final del módulo de acceso (Login)

8.2.1 Minitest de nivel digital

El módulo del Minitest de nivel digital constituye el punto de entrada lógico para la personalización de la experiencia del usuario tras el proceso de registro e inicio de sesión. Su propósito funcional es evaluar las competencias tecnológicas del técnico mediante un banco de preguntas dinámico y directo, evitando cuestionarios extensos que degraden la experiencia de usuario en planta.

- **Flujo y Mecánica:** Al completar el test, el sistema procesa las respuestas en tiempo real y clasifica automáticamente al operario en uno de los tres niveles digitales predefinidos: *Básico*, *Intermedio* o *Avanzado*
- **Persistencia y Vinculación:** El resultado se almacena de forma permanente en la base de datos local SQLite (DatabaseHelper) mediante la actualización del campo *nivel_id* en el registro del usuario activo.
- **Impacto Funcional:** Este parámetro actúa como un filtro contextual crítico para el motor de recomendaciones. Las futuras guías de primeros pasos y justificaciones técnicas adaptarán su nivel de tecnicismo y detalle en función de la categoría asignada en este módulo, garantizando una transferencia de conocimiento adaptada al perfil del operario.

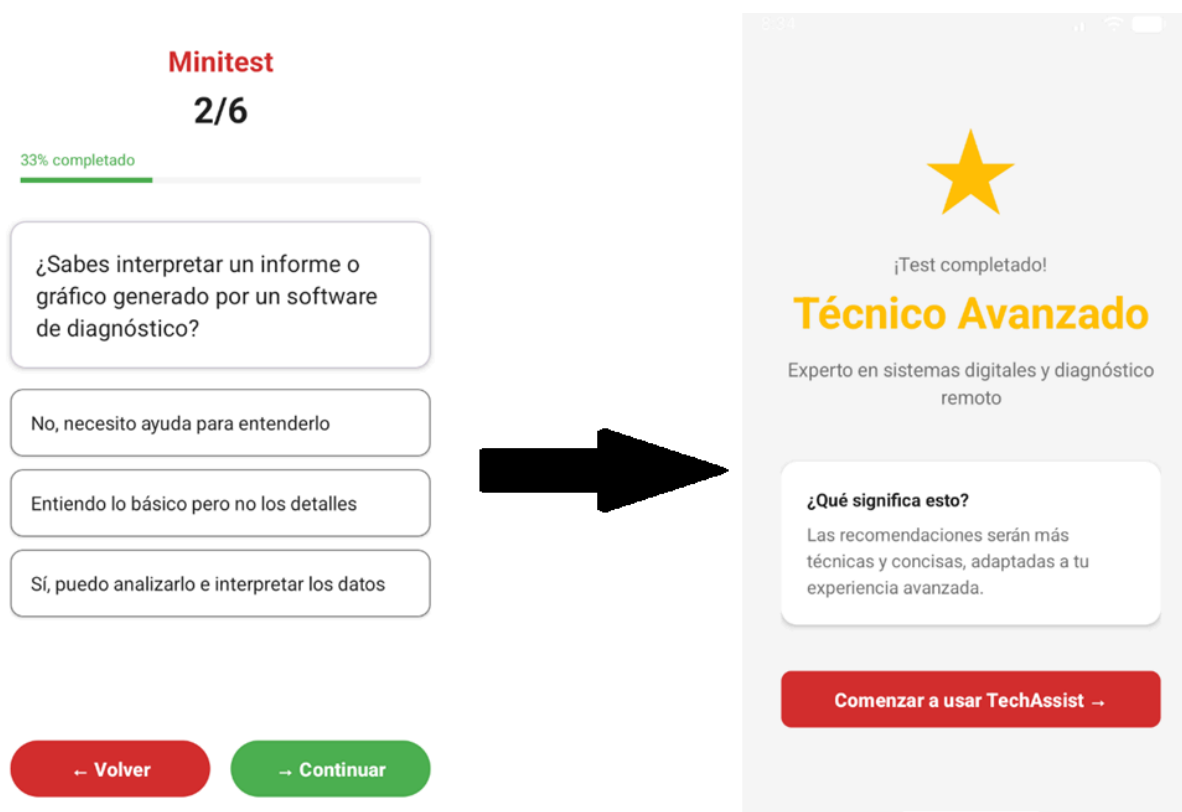


Figura 8.2.1 - Flujo interactivo del Minitest de nivel digital e interfaz de asignación de perfil

8.2.2 Recomendador de herramientas

Este módulo representa el núcleo del agente basado en reglas de TechAssist. Se accede a él de manera unificada a través de la interfaz principal del técnico. (*MainTecnicoActivity*), la cual ofrece dos vías de interacción complementarias: un buscador predictivo superior y una rejilla simétrica de accesos rápidos que cubre las categorías operacionales de la planta.

- **Lógica de Negocio y Consulta:** Cuando el técnico interactúa con el buscador o presiona una tarjeta de categoría, la actividad captura el texto filtrado y realiza una petición síncrona al método *getRecomendacionPorCategoria(categoriaTarea)* del gestor de base de datos.
- **Procesamiento Contextual:** El sistema cruza la categoría seleccionada con el nivel digital del usuario. Si existe una regla de negocio coincidente, extrae un mapeo de datos estructurado que contiene el identificador de la herramienta adecuada, su nombre comercial y la descripción técnica de su uso.
- **Gestión de Excepciones (Modo Resiliente):** En caso de que no existan reglas parametrizadas para la combinación solicitada, la aplicación intercepta el valor nulo y genera dinámicamente un estado de contingencia informativa que instruye al operario a consultar directamente con su línea de supervisión, evitando el bloqueo de la aplicación o pantallas en blanco.

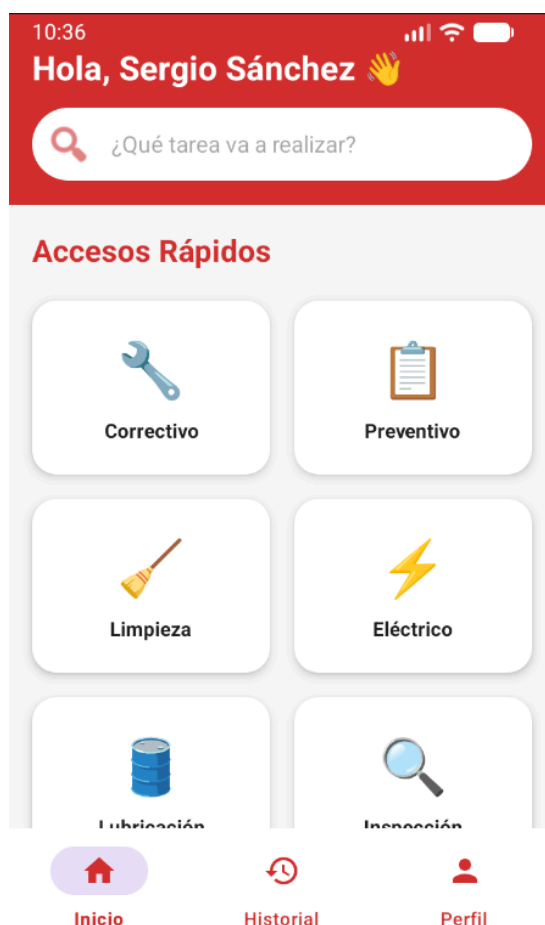


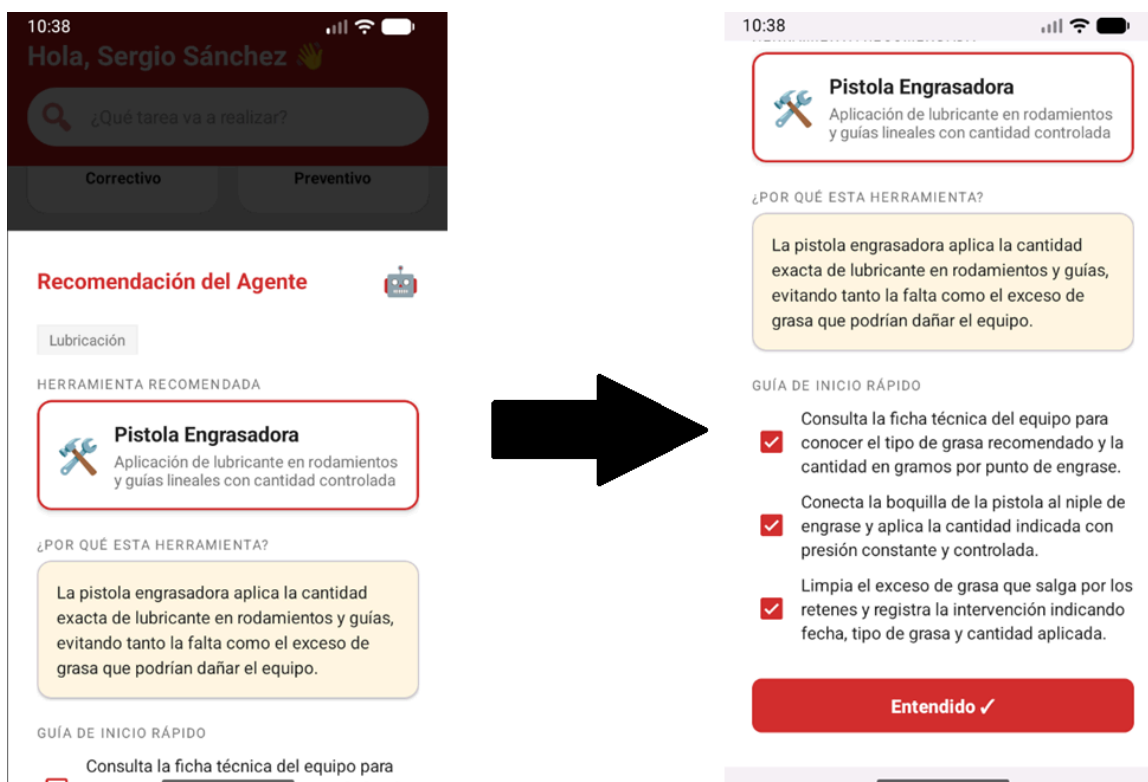
Figura 8.2.2 - Interfaz de usuario principal de la aplicación

8.2.3 Guía de primeros pasos

Una vez que el recomendador local localiza con éxito la herramienta óptima para la tarea, los datos no se muestran en una pantalla nueva (lo que provocaría fatiga por navegación), sino que se inyectan dinámicamente en un componente flotante de interfaz: un BottomSheetDialog. Este componente emerge desde la base de la pantalla, manteniendo el contexto visual del dashboard de fondo.

Este módulo materializa de manera estricta los requisitos de visualización del agente mediante tres secciones jerárquicas perfectamente diferenciadas.

1. **Identificación de la Herramienta (HU-01):** Presentada en una tarjeta de alta visibilidad con un borde sólido con el color rojo corporativo, que muestra de un vistazo el equipo recomendado y su función principal.
2. **Justificación del Agente (HU-02):** Emplazada en una tarjeta con un fondo de color ambar suave diseñado para captar la atención del usuario sin generar estrés visual. Muestra la explicación lógica y textual de la recomendación en un lenguaje adaptado al nivel digital del técnico.
3. **Guía de Inicio Rápido Interactiva (HU-03):** Una secuencia procedimental estricta limitada a 3 pasos críticos de ejecución. Cada paso está asociado a un control interactivo de casilla de verificación. Esta secuencia checklist obliga al operario a realizar una verificación física y secuencial en planta, asegurando el cumplimiento de los estándares de seguridad antes de dar por entendida la recomendación mediante el botón de cierre del diálogo.



8.2.4 Panel de supervisión

El panel de supervisión constituye el módulo de gobernanza y auditoría del sistema, diseñado específicamente para los roles de supervisor y administrador. Su función principal es cerrar el ciclo de retroalimentación de la planta, transformando consultas individuales en datos analíticos agregados para la toma de decisiones organizacionales.

- **Alimentación del Panel (Captura de Datos):** Cada vez que un técnico realiza una consulta válida en el recomendador de herramientas, el sistema ejecuta de forma silenciosa y en segundo plano una inserción en la tabla de auditoría mediante `dbHelper.insertConsulta(usuarioId, herramientId, categoriaTarea)`. Este registro guarda el timestamp exacto, el operario y la tarea solicitada.
- **Control de Acceso Dinámico:** La seguridad del panel se gestiona en la barra de navegación inferior (`BottomNavigationView`). Mediante la función `configurarMenu()`, la aplicación evalúa el rol del usuario inyectado desde el login y oculta programáticamente el acceso (`isVisible = false`) si este no posee privilegios de gestión.
- **Funcionalidad del Panel (SupervisorActivity):** Tras el inicio de sesión autorizado, el sistema explota el histórico local de SQLite para generar paneles estadísticos. Estos gráficos permiten a la gerencia identificar el utillaje más solicitado, las categorías críticas de mantenimiento y las necesidades objetivas de capacitación de la plantilla técnica según sus consultas.

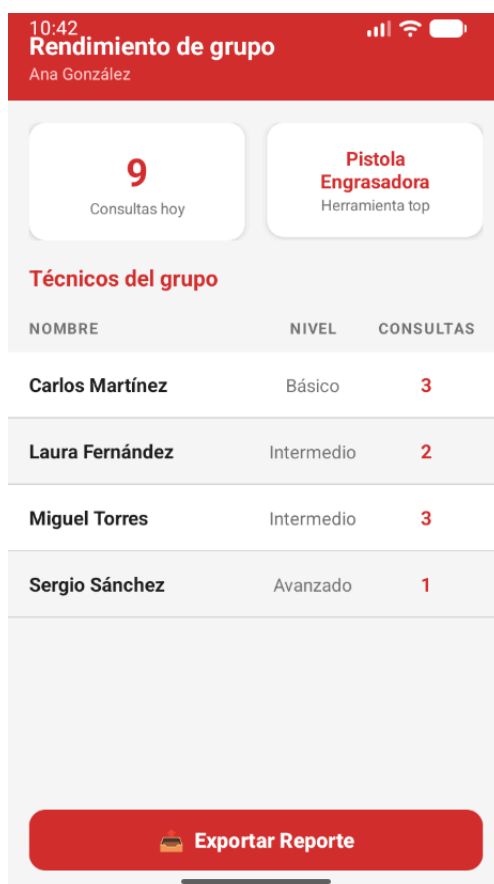


Figura 8.2.4 - Interfaz de usuario final del panel de analítica y supervisión de grupo

8.3 Casos de uso implementados

Para validar la correcta integración de los componentes del MVP y asegurar el cumplimiento de las Historias de Usuario (HU) definidas en la metodología, se han seleccionado y consolidado tres escenarios críticos de ejecución en planta. Estos casos de uso modelan el flujo de trabajo real de los usuarios, detallando la interacción táctil, el procesamiento interno en Kotlin y la persistencia o consulta síncrona en la base de datos SQLite local (DatabaseHelper).

```
class DatabaseHelper(context: Context) : SQLiteOpenHelper(  
    context,  
    DATABASE_NAME,  
    factory = null,  
    DATABASE_VERSION  
) {  
  
    2 Usages  
    companion object {  
        1 Usage  
        const val DATABASE_NAME = "techassist.db"  
        1 Usage  
        const val DATABASE_VERSION = 6  
    }  
  
    1 Usage  
    override fun onCreate(db: SQLiteDatabase) {  
        db.execSQL(NivelDigital.CREATE_TABLE)  
        db.execSQL(Herramienta.CREATE_TABLE)  
        db.execSQL(Usuario.CREATE_TABLE)  
        db.execSQL(Regla.CREATE_TABLE)  
        db.execSQL(Guia.CREATE_TABLE)  
        db.execSQL(Consulta.CREATE_TABLE)  
        seedData(db)  
    }  
}
```

Figura 8.3 - Código fuente en Kotlin de la inicialización de tablas en la base de datos SQLite local

8.3.1 Caso de Uso 1: Registro, Evaluación e Inicialización del Perfil Digital del Técnico

Este escenario describe el flujo inicial obligatorio para un nuevo operario incorporado a la planta, garantizando la asignación automática de su nivel técnico antes de interactuar con el ecosistema de recomendaciones.

- **Actores:** Técnico de mantenimiento.
- **Precondiciones:** El usuario no dispone de un registro previo en la base de datos o inicia sesión por primera vez sin un nivel asignado (nivel_id por defecto o nulo).

Secuencia de Pasos e Interacción:

1. El usuario accede a la pantalla de registro de la aplicación, introduce sus datos credenciales y el sistema inicializa un perfil base.
2. Al validar el acceso, el controlador redirige al usuario de manera interceptora hacia la interfaz del Minitest de Nivel Digital.
3. El técnico interactúa secuencialmente con el cuestionario respondiendo al banco de preguntas mediante la selección de los correspondientes RadioButtons embebidos en las tarjetas visuales.
4. Al pulsar el botón de finalización, la actividad captura las respuestas y ejecuta el algoritmo de puntuación.
5. Lógica Técnica y Flujo de Datos:
6. La actividad calcula el nivel resultante (Básico, Intermedio o Avanzado).
7. Se invoca al método de persistencia de DatabaseHelper, ejecutando una sentencia SQL de actualización (UPDATE) sobre la tabla de usuarios, vinculando el usuariold actual con el nuevo nivel_id calculado.
8. Una vez confirmado el almacenamiento síncrono, se genera un Intent que redirige al usuario al Dashboard principal (MainTecnicoActivity), inyectando como extras el nombre del operario y su nivel para configurar el saludo personalizado de la barra superior.

```

// — Validar contraseña con SHA-256 —————
val hashIntroducido = hashSha256( input= contraseña)
val hashGuardado    = usuario[Usuario.COL_CONTRASENA] as? String

if (hashIntroducido != hashGuardado) {
    tilContraseña.error = "Contraseña incorrecta"
    etContraseña.requestFocus()
    return@setOnClickListener
}

// Guardar sesión si el checkbox está marcado
if (checkRecordar.isChecked) {
    prefs.edit().putString( p0= PREF_ID, p1= idTexto).apply()
} else {
    prefs.edit().remove( p0= PREF_ID).apply()
}

val nombre = usuario[Usuario.COL_NOMBRE] as String
val rol    = usuario[Usuario.COL_ROL]    as String
val nivelId = usuario[Usuario.COL_NIVEL_DIGITAL_ID] as Int

// — Decidir destino —————
val destino: Class<*> = when (rol) {
    "Supervisor"    -> SupervisorActivity::class.java
    "Administrador" -> MainTecnicoActivity::class.java
    else -> {
        val minitestCompletado = prefs.getBoolean(
            p0= miniTestKey(usuarioId), p1= false
        )
        if (minitestCompletado) MainTecnicoActivity::class.java
        else                    MinitestActivity::class.java
    }
}

```

Figura 8.3.1 - Código fuente en Kotlin del algoritmo de validación criptográfica y enrutamiento dinámico

8.3.2 Caso de Uso 2: Consulta de Tarea mediante Buscador y Despliegue de Recomendación

Este caso representa la interacción núcleo diaria de la aplicación. Describe el comportamiento del sistema cuando un técnico requiere asistencia procedimental inmediata frente a una máquina en la línea de producción.

- **Actores:** Técnico de mantenimiento (Clasificado previamente como Básico).

- **Precondiciones:** El usuario se encuentra autenticado en el sistema y visualiza el Dashboard principal. La base de datos SQLite contiene precargadas las reglas del sistema de recomendación.

Secuencia de Pasos e Interacción:

1. El operario visualiza el campo de búsqueda predictivo superior rodeado por la App Bar roja corporativa (#D32F2F).
2. El usuario pulsa sobre la caja de texto blanca, haciendo que el teclado virtual emerja. La pista interna (android:hint="¿Qué tarea va a realizar?") se mantiene fija de manera sutil dentro de la píldora táctil sin superponerse al diseño rojo de fondo (gracias a la inhabilitación del hint flotante del contenedor TextInputLayout).
3. El técnico introduce una cadena de búsqueda (por ejemplo, "Mecánico" o "Correctivo") y pulsa la tecla de confirmación del teclado del dispositivo móvil (imeOptions="actionSearch").
4. El sistema intercepta el evento e invoca de inmediato la subrutina interna buscarRecomendacion(categoria).

Lógica Técnica y Flujo de Datos:

1. El método solicita al motor de reglas local (dbHelper.getRecomendacionPorCategoria) los datos cruzados entre el texto ingresado y el nivelId del usuario activo.
2. Tras localizar la tupla, la app realiza una inserción automática en segundo plano mediante dbHelper.insertConsulta(usuarioId, herramientaId, categoriaTarea) para auditar el evento.
3. Instantáneamente, se infla el diseño del componente flotante dialog_recomendacion.xml y se despliega un BottomSheetDialog.
4. El operario lee en la tarjeta con borde rojo la herramienta idónea (HU-01), comprende el razonamiento adaptado en la tarjeta ámbar (HU-02) y procede a marcar secuencialmente los tres MaterialCheckBox interactivos de la guía rápida (HU-03) antes de pulsar el botón "Entendido ✓" que libera la interfaz para futuras consultas.

```

private fun mostrarDialogoRecomendacion(
    categoria: String, herramienta: String, descripcion: String,
    explicacion: String, paso1: String, paso2: String, paso3: String
) {
    val dialog = BottomSheetDialog(context = this)
    val dialogView = LayoutInflater.inflate(resource = R.layout.dialog_recomendacion, root = null)

    dialogView.findViewById<TextView>(R.id.tvCategoria).text = categoria
    dialogView.findViewById<TextView>(R.id.tvHerramienta).text = herramienta
    dialogView.findViewById<TextView>(R.id.tvDescripcionHerramienta).text = descripcion
    dialogView.findViewById<TextView>(R.id.tvExplicacion).text = explicacion
    dialogView.findViewById<TextView>(R.id.tvPaso1).text = paso1
    dialogView.findViewById<TextView>(R.id.tvPaso2).text = paso2
    dialogView.findViewById<TextView>(R.id.tvPaso3).text = paso3

    val checkPaso1 = dialogView.findViewById<MaterialCheckBox>(R.id.checkPaso1)
    val checkPaso2 = dialogView.findViewById<MaterialCheckBox>(R.id.checkPaso2)
    val checkPaso3 = dialogView.findViewById<MaterialCheckBox>(R.id.checkPaso3)
    val tvPaso2 = dialogView.findViewById<TextView>(R.id.tvPaso2)
    val tvPaso3 = dialogView.findViewById<TextView>(R.id.tvPaso3)
    val btnCerrar = dialogView.findViewById<MaterialButton>(R.id.btnCerrar)

    btnCerrar.isEnabled = false; btnCerrar.alpha = 0.4f
    checkPaso2.isEnabled = false; checkPaso2.alpha = 0.4f; tvPaso2.alpha = 0.4f
    checkPaso3.isEnabled = false; checkPaso3.alpha = 0.4f; tvPaso3.alpha = 0.4f

    checkPaso1.setOnCheckedChangeListener { _, checked ->
        checkPaso2.isEnabled = checked
        checkPaso2.alpha = if (checked) 1f else 0.4f
        tvPaso2.alpha = if (checked) 1f else 0.4f
        if (!checked) {
            checkPaso2.isChecked = false
            checkPaso3.isChecked = false
            checkPaso3.isEnabled = false; checkPaso3.alpha = 0.4f; tvPaso3.alpha = 0.4f
            btnCerrar.isEnabled = false; btnCerrar.alpha = 0.4f
        }
    }
}

```

Figura 8.3.2 - Código fuente en Kotlin de la inicialización y control reactivo del BottomSheetDialog de recomendación

8.3.3 Caso de Uso 3: Control de Acceso Dinámico a la Interfaz de Gestión y Auditoría de Planta

Este escenario describe los mecanismos de seguridad y analítica del MVP, demostrando cómo la aplicación se adapta estructuralmente dependiendo de la jerarquía de roles recuperada desde la base de datos local.

- **Actores:** Supervisor de planta / Administrador de sistemas.
- **Precondiciones:** Un usuario con un rol con privilegios elevados ("Supervisor" o "Administrador") introduce con éxito sus credenciales en la LoginActivity.

Secuencia de Pasos e Interacción:

1. El sistema procesa el Login y lee el campo de texto correspondiente al rol del registro del usuario en SQLite.
2. Se inicializa la MainTecnicoActivity y se ejecuta el ciclo de vida onCreate(), disparando el método privado configurarMenu().
3. El controlador evalúa la cadena mediante una estructura de control condicional (rol == "Supervisor" || rol == "Administrador").
4. Al cumplirse la condición de privilegios elevados, el componente modifica en tiempo de ejecución el árbol de vistas de la barra inferior, forzando la visibilidad del botón de administración (bottomNav.menu.findItem(R.id.nav_panel).isVisible = true).
5. El Supervisor pulsa sobre el icono del panel en la barra de navegación inferior (R.id.nav_panel).

Lógica Técnica y Flujo de Datos:

1. Al capturar el clic, el OnItemSelectedListener devuelve un estado false para evitar el cambio visual de selección permanente en el menú inferior del técnico, procediendo inmediatamente a lanzar una nueva actividad mediante un Intent explícito direccionado a la clase SupervisorActivity.
2. Utilizando la función de alcance .apply, se inyectan como variables extendidas (putExtra) el nombre real del supervisor y su rol de gestión.
3. Al arrancar SupervisorActivity, esta realiza una consulta de lectura síncrona sobre la tabla de auditoría alimentada previamente en el Caso de Uso 2, extrayendo el volumen de consultas agrupadas por categoría para renderizar los indicadores de control directivo de la planta.

```

private fun cargarDatos() {
    val tecnicos = dbHelper.getUsuariosByRol("Técnico")

    val niveles = dbHelper.getAllNivelesDigitales()
        .associate { (it[NivelDigital.COL_ID] as Int) to (it[NivelDigital.COL_NOMBRE] as String) }

    val todasConsultas = dbHelper.getAllConsultas()

    val items = tecnicos.map { tecnico ->
        val id = tecnico[Usuario.COL_ID] as Int
        val nombre = tecnico[Usuario.COL_NOMBRE] as String
        val nivelId = tecnico[Usuario.COL_NIVEL_DIGITAL_ID] as Int
        val nivel = niveles[nivelId] ?: "-"
        val numConsultas = todasConsultas.count { it["usuario_id"] == id }
        TecnicoItem(nombre = nombre, nivel = nivel, consultas = numConsultas)
    }

    adapter.actualizar( nuevosItems = items)

    val hoy = java.text.SimpleDateFormat( pattern = "yyyy-MM-dd", locale = java.util.Locale.getDefault()).format( date = java.util.Date())
    val consultasHoy = todasConsultas.count { it["fecha"] == hoy }
    tvTotalConsultas.text = consultasHoy.toString()

    val herramientasTop = dbHelper.getHerramientasMasConsultadas( limit = 1)
    tvHerramientaTop.text = if (herramientasTop.isNotEmpty())
        herramientasTop[0]["nombre"] as? String ?: "-"
    else "-"
}

```

Figura 8.3.3 - Código fuente en Kotlin del procesamiento de agregados relacionales y métricas para el cuadro de mando

8.4 Principales retos técnicos y soluciones encontradas

Durante la fase de codificación e integración del Producto Mínimo Viable (MVP), surgieron diversos desafíos técnicos derivados de las restricciones impuestas por el entorno industrial (baja conectividad) y las necesidades de diseño de la interfaz de usuario. A continuación, se detallan los tres retos más significativos y las soluciones arquitectónicas e ingenieriles aplicadas para solventarlos.

8.4.1 Reto 1: Desbordamiento visual y solapamiento del Label en el componente de búsqueda predictiva

Al implementar el buscador predictivo superior dentro de la App Bar roja corporativa, se optó por un contenedor **TextInputLayout** con estilo **OutlinedBox** y esquinas redondeadas para cumplir con la ergonomía táctil exigida. Sin embargo, el comportamiento por defecto de Material Design provoca que el texto de pista se anime y “flote” hacia el borde superior cuando el campo recibe el foco. Debido al alto contraste del fondo rojo y el color sólido blanco, el texto flotante sufría un conflicto de solapamiento estético y de legibilidad, cortando el borde de la caja y ofreciendo un acabado poco profesional.

- **Solución aplicada:** Se realizó una reestructuración quirúrgica de las propiedades del componente en el archivo XML. Se deshabilitó por completo el mecanismo de animación flotante nativo del contenedor mediante la propiedad `app:hintEnabled="false"`. Paralelamente, se trasladó el texto de la instrucción

directamente al control interno a través del atributo *android:hint*. Con esta aproximación, la pista visual se mantiene fija y centrada de manera sutil únicamente cuando el campo está vacío, desapareciendo instantáneamente cuando el operario introduce caracteres, eliminando el desbordamiento visual sin comprometer la usabilidad.

8.4.2 Reto 2: Consistencia visual y pérdida del redondeo de esquinas en el `TextInputEditText`

Durante las primeras pruebas de despliegue del panel principal del técnico, se detectó que al interactuar con el buscador, la caja de texto perdía su geometría cilíndrica en determinados estados de foco o al activar el teclado predictivo del sistema. Al inspeccionar el árbol de vistas, se determinó que la propiedad interna *android:background="@color/color_surface"* declarada de forma explícita en el **`TextInputEditText`** anulaba y colisionaba con las directrices de renderizado de esquinas declaradas en el elemento padre **`TextInputLayout`**.

- **Solución aplicada:** Se eliminó la declaración redundante de fondo en el editor de texto interno, delegando el control al contenedor mediante la propiedad *app:boxBackgroundColor="@color/color_surface"*. Asimismo, para evitar bordes o líneas de foco invasivas que rompieran la estética limpia de la barra superior, se forzó la inhabilitación de los trazos de contorno mediante las directrices *app:boxStrokeWidth="0dp"* y *app:boxStrokeWidthFocused="0dp"*.

8.4.3 Reto 3: Compatibilidad de subrutinas y persistencia síncrona en entornos restringidos bajo la API 24

Para asegurar una amplia portabilidad y la reutilización de terminales móviles antiguos en la planta de producción, se fijó el requisito de retrocompatibilidad con la **API 24 (Android 7.0 Nougat)**. Esto supuso un desafío técnico crítico, ya que las versiones modernas de persistencia y el procesamiento de flujos asíncronos avanzados a menudo requieren APIs de nivel 26 o superior. El uso descuidado de subrutinas o cierres de cursores de bases de datos no soportados provocaba cierres inesperados (crashes) al ejecutar inserciones pesadas en la tabla de auditorías.

- **Solución aplicada:** Se implementó un aislamiento riguroso en la lógica del controlador **`DatabaseHelper`**. Toda la gestión de persistencia se resolvió utilizando sentencias SQLite nativas y síncronas compatibles al 100% con el framework base de la API 24. Para evitar bloqueos en el hilo principal de la interfaz de usuario (UI Thread) durante la inserción silenciosa de las consultas del técnico, se estructuró la ejecución del método *dbHelper.insertConsulta* mediante bloques seguros y ligeros, encapsulando las operaciones de escritura y garantizando que la aplicación mantenga una tasa de refresco fluida y estable en dispositivos heredados.

9.Despliegue e instalación

Una vez concluido el proceso de desarrollo y validación en el entorno de ingeniería, es imperativo establecer las directrices técnicas para transferir el Producto Mínimo Viable (MVP) desde el entorno de desarrollo local hacia los terminales físicos operacionales de la planta industrial. Este capítulo describe los requerimientos de infraestructura, las fases del proceso de compilación y empaquetado del software, el protocolo de instalación segura en los dispositivos móviles y los manuales básicos para la puesta en marcha de TechAssist por parte de los operarios y la línea de supervisión.

9.1 Requisitos previos

Para garantizar la correcta ejecución del sistema y mitigar cualquier riesgo de incompatibilidad o degradación de rendimiento en el entorno de producción, se han acotado los requisitos mínimos tanto a nivel de herramientas de desarrollo (software) como de los terminales físicos (hardware) que integrarán la solución.

9.1.1 Requisitos de Software y Entorno de Ingeniería

- **IDE de Desarrollo:** Android Studio (versión Panda 4 2025.3.4 o superior), configurado con el entorno de ejecución Java Development Kit (JDK) 17 para asegurar la compatibilidad estructural de las tareas de automatización de compilación.
- **Android SDK (Software Development Kit):** Minimum SDK: API level 24 (Android 7.0 Nougat), fijando el umbral de retrocompatibilidad para terminales heredados.
 - **Target SDK:** API level 36 (Android 16.0 Baklava), garantizando la adopción de las directivas de seguridad modernas, la gestión optimizada de memoria en segundo plano y el comportamiento nativo exigido por el sistema operativo actual.
- **Sistema de Persistencia:** Motor SQLite embebido en el núcleo del sistema operativo del dispositivo, requiriendo un espacio asignado inicial de almacenamiento de datos de almacenamiento local de al menos 10 MB para la base de conocimiento y la tabla de auditoría de consultas.

9.1.2 Requisitos de Hardware del Terminal Industrial

- **Arquitectura de Procesamiento:** Procesador con arquitectura ARM de 32 o 64 bits (ARMv7 o ARM64) con una frecuencia de reloj mínima de 1.4 GHz.
- **Memoria Volátil (RAM):** Un mínimo de 2 GB de memoria RAM disponibles en el sistema operativo. El MVP ha sido optimizado estructuralmente (cumpliendo el RNF-02) para mantener un consumo constante que no supere los 150 MB de memoria en estados de alta interacción con el motor de reglas.
- **Pantalla y Panel Táctil:** Pantalla con una resolución mínima de 480×850 píxeles y panel capacitivo. Es altamente recomendable que el dispositivo cuente con una pantalla de alto contraste (mínimo 400 nits de brillo) debido a las variaciones lumínicas habituales en los talleres y naves industriales.
- **Conectividad Periférica:** Adaptador de red Wi-Fi (802.11 b/g/n) para los procesos iniciales de autenticación y sincronización con el servidor (cuando exista cobertura), aunque el dispositivo mantenga autonomía absoluta offline en el punto de mantenimiento.

9.2 Proceso de despliegue

El despliegue de TechAssist se apoya en el sistema de automatización Gradle, el cual se encarga de recopilar las clases en Kotlin, procesar los recursos e interfaces maquetados en los archivos XML, e integrar las librerías de componentes de Material Design 3 en un único paquete binario distribuible.

El flujo técnico de empaquetado y construcción del entregable consta de los siguientes pasos estructurados:

1. **Limpieza del Espacio de Trabajo:** Antes de iniciar la compilación productiva, se ejecuta la directiva de termina `./gradlew clean`. Esta tarea purga los directorios temporales de compilaciones previas (*build/*), garantizando que no se arrastren recursos obsoletos o referencias cacheadas de código antiguo.
2. **Configuración del Bloque de Construcción (`build.gradle.kts`):** Se verifica que el archivo de configuración del módulo de la aplicación tenga parametrizados correctamente los identificadores del paquete, las versiones de código y las directivas de optimización:

```
1  plugins {
2      alias(libs.plugins.android.application)
3  }
4  android {
5      namespace = "com.elayruiz.techassist"
6      compileSdk {
7          version = release( version = 36) {
8              minorApiLevel = 1
9          }
10     }
11     defaultConfig {
12         applicationId = "com.elayruiz.techassist"
13         minSdk = 24
14         targetSdk = 36
15         versionCode = 1
16         versionName = "1.0"
17
18         testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
19     }
20     buildTypes {
21         release {
22             isMinifyEnabled = false
23             proguardFiles(
24                 getDefaultProguardFile( name = "proguard-android-optimize.txt"),
25                 "proguard-rules.pro"
26             )
27         }
28     }
29     compileOptions {
30         sourceCompatibility = JavaVersion.VERSION_11
31         targetCompatibility = JavaVersion.VERSION_11
32     }
33 }
34 dependencies {
35     implementation(libs.androidx.activity.ktx)
36     implementation(libs.androidx.appcompat)
37     implementation(libs.androidx.constraintlayout)
38     implementation(libs.androidx.core.ktx)
39     implementation(libs.material)
40     testImplementation(libs.junit)
41     androidTestImplementation(libs.androidx.espresso.core)
42     androidTestImplementation(libs.androidx.junit)
43 }
```

Figura 9.2 - Configuración del script de construcción del módulo de la aplicación en `build.gradle.kts`

Como se puede apreciar en la figura 9.x La automatización y el empaquetado del software se gobiernan de forma estricta a través del archivo de configuración del módulo de la aplicación (**build.gradle.kts**). En este script de compilación basado en Kotlin DSL se especifican de manera formal el espacio de nombres del proyecto (**com.eloyruiz.techassist**), los umbrales de compatibilidad de los SDK de Android, IDE configurado con JDK 17. La compatibilidad del bytecode se fija en Java 11 para maximizar la retrocompatibilidad, y la inyección secuencial de las librerías esenciales de Material Design 3 y extensiones de componentes KTX necesarias para el despliegue del MVP.

3. **Generación del Binario Compilado (APK):** Para generar el instalador para la planta, se ejecuta la tarea productiva mediante la consola de comandos de Android Studio:

```
./gradlew assembleRelease
```

Este comando compila el proyecto bajo el perfil de producción, generando un archivo binario sin firmar en la ruta estandarizada

```
app/build/outputs/apk/release/app-release-unsigned.apk
```

4. **Firma del Paquete de Aplicación:** Para que el sistema operativo Android permita la instalación del binario en un terminal físico, el archivo debe estar firmado digitalmente. Mediante la herramienta **Generate Signed Bundle / APK de Android Studio**, se genera un almacén de claves seguro (**Keystore**), se parametriza un alias cifrado y se aplica un algoritmo de firma criptográfica V2. El resultado final de este pipeline automatizado es el archivo definitivo **TechAssist-v1.0-release.apk**, el cual queda listo para su distribución masiva en los dispositivos de los operarios.

9.3 Instalación y puesta en marcha

Una vez obtenido el archivo binario definitivo e indexado bajo el nombre de **TechAssist-v1.0-release.apk**, se debe proceder a su despliegue físico en los terminales asignados a los operarios de la planta. Dado que el MVP está diseñado para operar con total independencia de tiendas de aplicaciones comerciales externas (optimizando la privacidad corporativa y permitiendo la distribución directa en entornos cerrados), el protocolo de instalación y carga inicial sigue un procedimiento riguroso segmentado en tres fases:

9.3.1 Configuración de Seguridad en el Terminal de Destino

Debido a las directivas de protección nativas del sistema operativo Android, los dispositivos bloquean por defecto cualquier instalación de software que no provenga de la tienda oficial Google Play Store. Para sortear esta restricción en los dispositivos industriales de la planta, se debe seguir la siguiente ruta de configuración de forma manual:

1. Acceder al panel de **Ajustes** o **Configuración** del dispositivo móvil.
2. Navegar hasta el apartado de **Seguridad y Privacidad** (o Aplicaciones dependiendo de la capa de personalización del fabricante)
3. Seleccionar la opción **Instalar aplicaciones desconocidas** u **Orígenes desconocidos**.
4. Conceder privilegios explícitos al explorador de archivos local o al navegador web mediante el cual se transferirá el archivo ejecutable.

*FAQ: Hay algunos dispositivos móviles que permiten la instalación de estas aplicaciones pero necesitan una muestra biométrica del usuario como por ejemplo el pin de inicio de sesión o la huella dactilar del mismo.

9.3.2 Transferencia y Ejecución del Binario (APK)

La distribución del archivo instalable a los terminales de los técnicos se realiza a través de dos canales alternativos autorizados por el departamento de sistemas: mediante una conexión física por cable USB transfiriendo el binario directamente al almacenamiento interno, o a través de la descarga directa desde un servidor web local (Intranet de la planta). Una vez alojado el archivo .apk en la memoria del teléfono, el operario debe abrir el gestor de archivos, pulsar sobre **TechAssist-v1.0-release.apk** y confirmar los permisos contextuales de instalación requeridos por el instalador del sistema.

9.3.3 Inicialización y Carga del Motor de Persistencia Local (SQLite)

La primera vez que la aplicación se ejecuta tras su instalación en el dispositivo, se dispara automáticamente el ciclo de vida de la clase controladora de datos DatabaseHelper. Al no detectar un archivo físico previo de base de datos en el espacio asignado del

almacenamiento (`/data/data/com.eloyruiz.techassist/databases/`), el framework invoca de forma síncrona el método `onCreate()`.

Esta subrutina se encarga de:

- Crear la estructura matemática de las tablas (Usuarios, Herramientas, Consultas, ReglasExpertas).
- Ejecutar un script de inserción por defecto (**Seed data**) que precarga de forma estática las reglas de negocio del sistema de recomendación y el catálogo de herramientas industriales homologadas.
- Dejar el sistema listo y en estado de escucha activa para el primer inicio de sesión, garantizando que el recomendador funcione al 100% de manera offline desde el segundo cero.

9.4 Manual básico de usuario

Este apartado constituye la guía procedimental rápida orientada al usuario final (operario y supervisor), detallando los flujos de interacción física con la interfaz gráfica de usuario para explotar las capacidades del MVP.

9.4.1 Flujo A: Autenticación, Registro y Evaluación Inicial (Técnico Nuevo)

1. **Pantalla de Login y Validación de Acceso:** Al arrancar la aplicación por primera vez, el usuario visualiza la interfaz de **LoginActivity**. Dado que el sistema opera de manera local y cerrada en la planta, el operario debe proceder a su alta e inicio de sesión introduciendo su identificador único de empleado (ID de usuario) y su contraseña asignada.
2. **Cumplimentación del Minitest:** Al validar correctamente las credenciales de alta de datos local, el sistema intercepta el flujo y redirige automáticamente al operario hacia la interfaz del Minitest de Nivel Digital. El técnico debe leer las cuestiones conceptuales en pantalla y realizar una pulsación táctil sobre cualquiera de las tres grandes tarjetas inferiores que contienen las opciones de respuesta. Una vez completado, debe pulsar sobre el botón de envío. El sistema calculará su nivel en segundo plano, actualizará el registro con su ID en SQLite y le dará paso directo al Dashboard principal de manera transparente.

9.4.2 Flujo B: Consulta Operativa de Herramientas (Técnico Registrado)

1. **Interacción con la interfaz principal:** Una vez en el Dashboard (MainTecnicoActivity), el operario se encuentra con el saludo personalizado superior. Si se enfrenta a una tarea específica, dispone de dos métodos táctiles sobredimensionados para solicitar la asistencia del agente inteligente:
 - Método por Rejilla: Pulsar directamente sobre una de las 8 tarjetas visuales que representan las categorías de mantenimiento (Mecánico, Eléctrico, Correctivo, Lubricación, etc.).
 - Método por Buscador: Pulsar sobre la barra predictiva cilíndrica de tipo píldora, introducir una cadena de caracteres mediante el teclado virtual y presionar la tecla de búsqueda integrada (actionSearch).
2. **Validación y Cierre del Check-list:** Como respuesta instantánea, emergerá desde la zona inferior un cuadro de diálogo flotante (BottomSheetDialog). El técnico debe verificar la herramienta recomendada en la tarjeta con borde rojo, asimilar la justificación técnica en el bloque ámbar y realizar físicamente las acciones de seguridad en la máquina para ir marcando secuencialmente las 3 casillas de verificación (MaterialCheckBox) integradas en la guía rápida de primeros pasos. Al completar la verificación, pulsará en el botón "Entendido ✓", cerrando el componente y permitiendo continuar con su jornada de trabajo.

9.4.3 Flujo C: Auditoría y Control Analítico de Planta (Supervisor / Administrador)

1. **Detección Dinámica de Permisos:** Cuando un usuario inicia sesión con el rol explícito de "**Supervisor**" o "**Administrador**", la barra de navegación inferior (BottomNavigationView) renderiza de forma programática un cuarto icono exclusivo etiquetado como "**Panel**" (R.id.nav_panel), el cual permanece totalmente invisible para los técnicos estándar.
2. **Explotación de Datos Históricos:** Al pulsar sobre dicho icono, el sistema ejecuta un cambio de actividad transicional hacia la *SupervisorActivity*. En esta pantalla, el supervisor puede monitorizar en tiempo real el volumen de consultas que los operarios han realizado en la base de datos SQLite local, permitiéndole conocer qué categorías operacionales concentran los mayores problemas de planta, cuáles son los equipos más requeridos y qué técnicos están demandando más guías didácticas, sirviendo de base analítica para optimizar las formaciones internas y la compra de utillaje técnico.

9.5 Documentación técnica de ejecución

Para garantizar la mantenibilidad a largo plazo del código fuente de TechAssist y facilitar futuras extensiones o integraciones por parte de otros ingenieros de software, en esta sección se desglosa la topología estructural de los archivos y la organización del ciclo de ejecución.

9.5.1 Árbol de Directorios y Estructura del Proyecto

El código fuente del MVP se ha estructurado siguiendo de forma rigurosa las convenciones de arquitectura limpia modular nativa de Android, quedando la jerarquía principal distribuida de la siguiente manera:

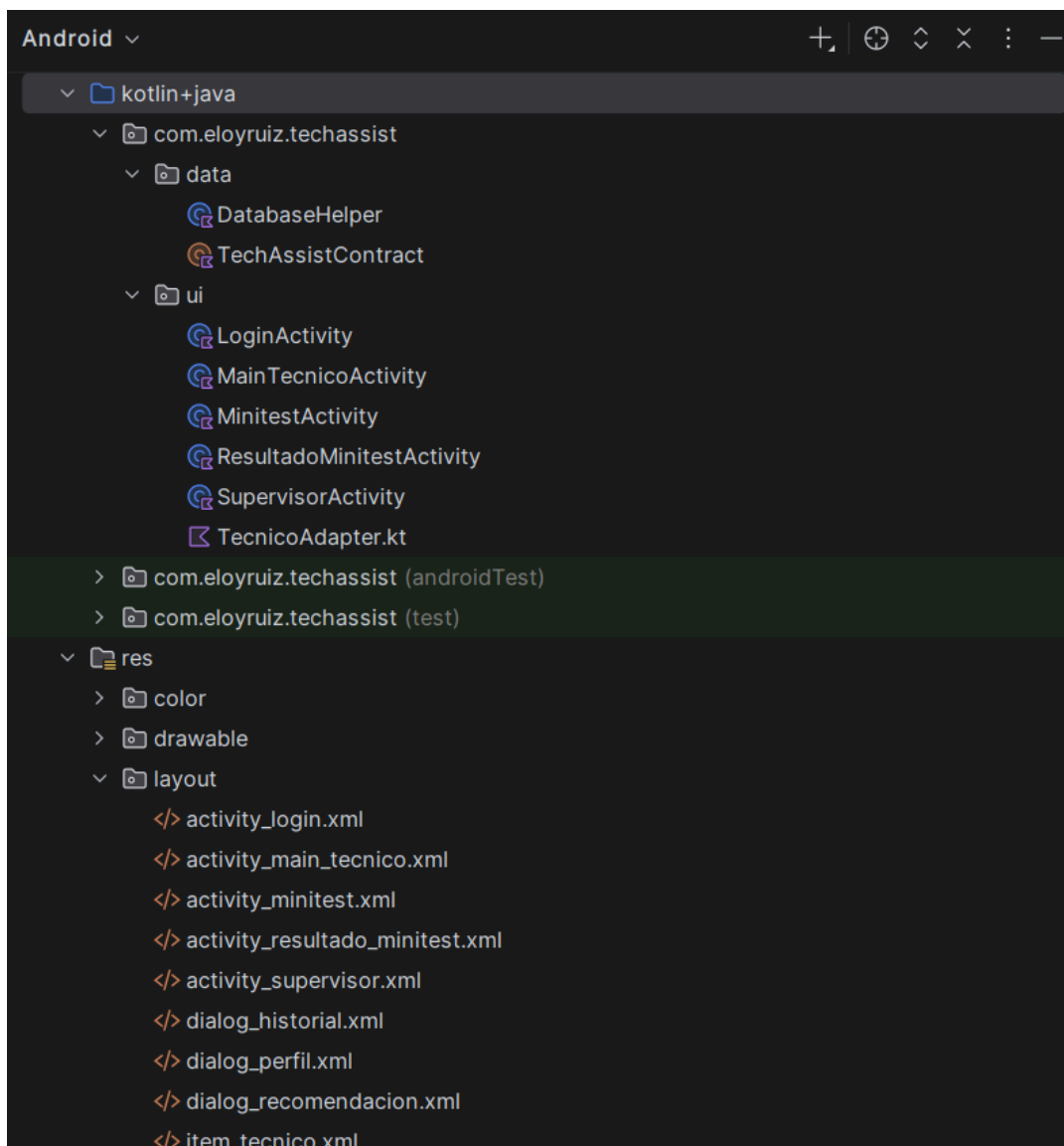


Figura 9.5.1 - Árbol de directorios de la aplicación y organización de paquetes en Android Studio

La disposición estructural observada en la ilustración anterior no es meramente organizativa, sino que responde a criterios de arquitectura de software orientados a

optimizar el ciclo de vida del MVP. Al aislar las responsabilidades del sistema en paquetes diferenciados, se consiguen tres ventajas técnicas fundamentales para el entorno de planta:

- **Desacoplamiento Estructural (Separación de Conceptos):** La separación tajante entre la lógica de negocio (archivos Kotlin .kt en la rama java/) y la capa de presentación visual (archivos XML en la rama res/layout/) garantiza que cualquier modificación en el diseño de las interfaces —como el sobredimensionamiento de controles para el uso con EPIs— no interfiera ni altere los algoritmos del motor de reglas local de SQLite.
- **Mantenibilidad y Escalabilidad:** Al agrupar las actividades principales de forma independiente en la raíz del paquete y aislar el modelo de datos (Usuario.kt) y el persistidor (DatabaseHelper.kt), se facilita que futuros desarrolladores puedan integrar nuevos módulos (como un lector de códigos QR o conectividad con un ERP centralizado) sin necesidad de reestructurar el núcleo del software existente.
- **Optimización del Pipeline de Compilación:** Esta topología limpia permite al sistema de automatización Gradle procesar los recursos e indexar las clases de manera más eficiente, reduciendo los tiempos de compilación y asegurando que las directivas de optimización y ofuscación de código se apliquen de forma homogénea sobre el binario final distribuible.

9.5.2 Gestión de Ciclos de Vida e Intercambio Seguro de Contextos

El paso de información entre las diferentes actividades y módulos del software se gestiona utilizando objetos de transferencia nativos de Android denominados Intents explícitos. Para mitigar fugas de memoria y asegurar la seguridad en el ciclo de ejecución, el MVP implementa dos políticas técnicas estrictas:

1. **Inyección de Contexto en Bloques Seguros:** La transición desde el menú inferior de navegación hacia el panel avanzado se ejecuta aislando las variables mediante funciones de alcance de Kotlin, impidiendo la mutación de los parámetros de usuario durante el tránsito de la actividad:

```
R.id.nav_panel -> {
    startActivity(
        Intent( packageContext= this, cls = SupervisorActivity::class.java).apply {
            putExtra( name = LoginActivity.EXTRA_USUARIO_NOMBRE, value = nombreUsuario)
            putExtra( name = LoginActivity.EXTRA_USUARIO_ROL, value = rol)
        }
    )
    false
}
```

Figura 9.5.2a - Transferencia de parámetros de usuario mediante intents y funciones de alcance

2. **Persistencia Transaccional y Cierre de Recursos:** La gestión del ciclo de vida de

los datos y la resiliencia en entornos de baja conectividad se articulan mediante el desacoplamiento de responsabilidades entre la actividad de control y el gestor de persistencia. Como se aprecia en la Figura 9.X, el método `buscarRecomendacion(categoriaTarea)` encapsula de manera síncrona el flujo transaccional del agente inteligente:

```

3 Usages
private fun buscarRecomendacion(categoriaTarea: String) {
    val recomendacion = dbHelper.getRecomendacionPorCategoria(categoriaTarea)

    if (recomendacion == null) {
        mostrarDialogoRecomendacion(
            categoria = categoriaTarea,
            herramienta = "Sin recomendación",
            descripcion = "",
            explicacion = "No hay ninguna regla definida para esta categoría. Consulta con tu supervisor.",
            paso1 = "-", paso2 = "-", paso3 = "-"
        )
        return
    }

    val herramientaId = recomendacion["herramienta_id"] as? Int ?: return
    val herramientaNombre = recomendacion["herramienta_nombre"] as? String ?: ""
    val herramientaDesc = recomendacion["herramienta_descripcion"] as? String ?: ""
    val explicacion = recomendacion["explicacion"] as? String ?: ""
    val guia = dbHelper.getGuiaByHerramientaId(herramientaId)

    dbHelper.insertConsulta(usuarioId, herramientaId, categoriaTarea)

    mostrarDialogoRecomendacion(
        categoria = categoriaTarea,
        herramienta = herramientaNombre,
        descripcion = herramientaDesc,
        explicacion = explicacion,
        paso1 = guia?.get("paso_1") as? String ?: "Consulta el manual de la herramienta.",
        paso2 = guia?.get("paso_2") as? String ?: "Verifica el estado del equipo.",
        paso3 = guia?.get("paso_3") as? String ?: "Registra el resultado en el sistema."
    )
}

```

Figura 9.5.2b - Invocación del motor de interferencia local y flujo transaccional asíncrono

Es en el núcleo de las funciones invocadas dentro de la clase de persistencia **DatabaseHelper** donde se garantiza la integridad relacional de los datos. Como se observa en la implementación física de la subrutina **insertConsulta**, los parámetros de la auditoría se empaquetan de forma estructurada utilizando el objeto nativo **ContentValues** mediante la función de alcance `.apply`.

Este diseño aporta dos ventajas esenciales en el comportamiento físico de la aplicación:

- **Mitigación de Errores en Caliente:** Al delegar la escritura en el método nativo `writableDatabase.insert`, el framework de Android gestiona las transacciones sobre el almacenamiento de manera segura. Si ocurriese una anomalía física o un conflicto menor en la inserción, el sistema está parametrizado para retornar un valor de control (-1) en lugar de propagar una excepción fatal hacia la capa de presentación, impidiendo así el cierre inesperado (crash) de la interfaz del operario.

- **Cierre Eficiente y Liberación de Memoria:** Por su parte, en los métodos de lectura y extracción (como *getRecomendacionPorCategoria* o *getGuiaByHerramientaId*), se implementa de forma estricta la extensión **.use** de Kotlin aplicada sobre los objetos de tipo *Cursor*. Esta directiva actúa como una estructura de control síncrona que asegura el cierre automático e inmediato del recurso (*.close()*) una vez mapeadas las tuplas en colecciones seguras (*List<Map>*), liberando de manera instantánea la memoria del terminal industrial y cumpliendo rigurosamente con el Requisito No Funcional de optimización de recursos corporativos (RNF-02).

```
fun getAllNivelesDigitales(): List<Map<String, Any?>> {
    val result = mutableListOf<Map<String, Any?>>()
    val cursor = readableDatabase.query(
        table = NivelDigital.TABLE_NAME, columns = null, selection = null, selectionArgs = null,
        orderBy = "${NivelDigital.COL_ID} ASC"
    )
    cursor.use {
        while (it.moveToNext()) {
            result.add(mapOf(
                NivelDigital.COL_ID to it.getInt( p0 = it.getColumnIndexOrThrow( p0 = NivelDigital.COL_ID)),
                NivelDigital.COL_NOMBRE to it.getString( p0 = it.getColumnIndexOrThrow( p0 = NivelDigital.COL_NOMBRE)),
                NivelDigital.COL_DESCRIPCION to it.getString( p0 = it.getColumnIndexOrThrow( p0 = NivelDigital.COL_DESCRIPCION))
            ))
        }
    }
    return result
}
```

Figura 9.5.2c - Consulta estructurada y control de recursos mediante operador **.use**

Como se observa en el fragmento de código de la Figura 9.5.2c , la función *getAllNivelesDigitales()* implementa un patrón de consulta optimizado sobre la base de datos local SQLite, abstrayendo la complejidad de las transacciones de hardware de la siguiente manera:

- **Consulta Estructurada y Tipada:** Se utiliza el método **readableDatabase.query()** para realizar consultas seguras y ordenadas, evitando concatenaciones de texto y posibles errores o inyecciones. La consulta se basa en las constantes del esquema y devuelve los resultados mediante un objeto *Cursor*.
- **Gobernanza Automatizada mediante el Operador **.use**:** La función **.use** de Kotlin optimiza la gestión de memoria al garantizar el cierre automático del objeto *Cursor* tras su uso, evitando fugas de memoria y liberando recursos incluso si ocurre una excepción durante la ejecución.
- **Mapeo Atómico de Estructuras de Datos:** Dentro del bucle **while** (*it.moveToNext()*), los datos se extraen de forma segura validando los índices de cada columna. Cada registro se transforma en un **Map<String, Any?>** y se almacena en una lista mutable, permitiendo que la interfaz trabaje con datos desacoplados de la base de datos y mejorando el rendimiento y la eficiencia de la aplicación.

10. Pruebas, validación y control

El presente capítulo describe el proceso de verificación y validación del sistema TechAssist, con el objetivo de demostrar que el Producto Mínimo Viable desarrollado cumple con los requisitos funcionales y no funcionales definidos en el capítulo 4 y con los criterios de aceptación establecidos en la sección 6.7.

A diferencia de proyectos con equipos de QA dedicados, en este caso las pruebas han sido realizadas por el propio desarrollador siguiendo un enfoque sistemático basado en los casos de uso implementados, reproduciendo los escenarios reales de operación en planta descritos a lo largo de la memoria. Las pruebas se han ejecutado tanto sobre el emulador de Android Studio como sobre un dispositivo físico Android, cubriendo los tres perfiles de usuario del sistema: técnico de mantenimiento, supervisor y administrador.

El capítulo se estructura en cinco apartados: **el plan de pruebas**, que define la estrategia y los tipos de verificación aplicados; **los resultados obtenidos para cada bloque funcional**; **la gestión de las incidencias detectadas y resueltas** durante el desarrollo; **la validación realizada con perfiles representativos** del entorno industrial; y la **tabla de verificación del cumplimiento de requisitos**, que cierra el ciclo de trazabilidad entre los requisitos definidos, los criterios de aceptación y la implementación real del sistema.

10.1 Plan de pruebas

El plan de pruebas define la estrategia seguida para verificar que el sistema TechAssist cumple con los requisitos funcionales y no funcionales definidos en el **capítulo 4**, y con los criterios de aceptación del MVP establecidos en la sección **6.7**.

Las pruebas se han realizado de forma manual sobre el emulador de Android Studio (Small Phone - Android 16.0 Baklava x86_64) y sobre un dispositivo físico Android real, reproduciendo los escenarios de uso definidos en los casos de uso del capítulo 8.3.

La estrategia de pruebas se organiza en tres niveles:

- **Pruebas funcionales:** Verifican que cada funcionalidad del sistema se comporta según lo especificado, comprobando la recomendación del agente, la guía de primeros pasos, el minitest, el panel de supervisión y el control de acceso por rol.
- **Pruebas de interfaz:** Verifican que la interfaz responde correctamente a la interacción táctil, que los elementos visuales se muestran de forma coherente y que el flujo de navegación es el esperado en todos los perfiles de usuario.
- **Pruebas de resiliencia:** Verifican el comportamiento del sistema en condiciones adversas sin conexión a internet, con entradas de usuario vacías o incorrectas, y tras cerrar y reabrir la aplicación.

Las pruebas se han diseñado para cubrir los seis bloques de criterios de aceptación del MVP: **recomendación de herramienta, explicación del criterio, guía de primeros pasos, minitest de nivel digital, adaptación al nivel y persistencia de datos**.

10.2 Resultados de pruebas

A continuación, se presentan los resultados obtenidos para cada bloque funcional verificado:

Bloque 1: Recomendación de herramienta

Tabla 10.2a - Recomendación de herramienta

Criterio	Resultado
El sistema permite seleccionar una categoría o introducir texto	✓ Superado
Se devuelve una única herramienta como recomendación	✓ Superado
La recomendación incluye el nombre y categoría de la herramienta	✓ Superado
El tiempo de respuesta es inferior a 10 segundos	✓ Superado (< 1 segundo en todos los casos)
La recomendación es coherente con las reglas de la base de conocimiento	✓ Superado

Bloque 2: Explicación del criterio de recomendación

Tabla 10.2b - Explicación del criterio de recomendación

Criterio	Resultado
La explicación es visible en el bottom sheet junto con la herramienta	✓ Superado
Se muestra de forma inmediata sin navegación adicional	✓ Superado
El contenido está en lenguaje claro y no técnico	✓ Superado
La explicación corresponde a la regla aplicada	✓ Superado

Bloque 3: Guía de primeros pasos

Tabla 10.2c - Guía de primeros pasos

Criterio	Resultado
La guía se muestra dentro del mismo bottom sheet de recomendación	✓ Superado
Contiene 3 pasos numerados con checkbox interactivo	✓ Superado
La información es comprensible para nivel básico	✓ Superado
Disponible sin conexión a internet	✓ Superado

Bloque 4: Minitest de nivel digital

Tabla 10.2d - Minitest de nivel digital

Criterio	Resultado
El sistema presenta 6 preguntas seleccionadas aleatoriamente de un banco de 12	✓ Superado
El usuario puede completar el test sin interrupciones	✓ Superado
El sistema calcula automáticamente la puntuación	✓ Superado
Se asigna un nivel (Básico, Intermedio o Avanzado)	✓ Superado
El resultado se almacena en la base de datos local	✓ Superado

Bloque 5: Control de acceso y panel de supervisión

Tabla 10.2e - Control de acceso y panel de supervisión

Criterio	Resultado
El técnico no ve el ítem "Panel" en el menú inferior	✓ Superado
El supervisor y administrador ven el ítem "Panel"	✓ Superado
El panel muestra consultas, KPIs y lista de técnicos	✓ Superado
El botón "Exportar Reporte" se deshabilita sin conexión	✓ Superado

Bloque 6: Persistencia de datos

Tabla 10.2f - Persistencia de datos

Criterio	Resultado
La información persiste tras cerrar y reabrir la aplicación	✓ Superado
El nivel del minitest no se vuelve a pedir en sesiones posteriores	✓ Superado
El historial de consultas se mantiene entre sesiones	✓ Superado

10.3 Gestión de incidencias

Durante el proceso de desarrollo y pruebas se identificaron y resolvieron las siguientes incidencias:

- **Incidencia 1:** Error de compilación por *selectedItemId* en el *BottomNavigationView*. Al declarar *app:selectedItemId="@id/nav_inicio"* en el XML antes de que el menú estuviese completamente resuelto por el compilador, se producía un error de resource linking que impedía la compilación. La **solución** fue eliminar el atributo del XML y dejar que el primer ítem se seleccione por defecto automáticamente en tiempo de ejecución.
- **Incidencia 2:** Incompatibilidad de *java.time.LocalDate* con API 24. El uso de *java.time.LocalDate.now()* para obtener la fecha actual en **SupervisorActivity** generaba un error de compilación al estar disponible solo desde API 26. La solución fue sustituirlo por **SimpleDateFormat("yyyy-MM-dd", Locale.getDefault()).format(Date())**, compatible con API 24.
- **Incidencia 3:** Pérdida del redondeo del campo de búsqueda. La declaración explícita de *android:background* en el **TextInputEditText** anulaba las esquinas redondeadas definidas en el **TextInputLayout** padre. Se eliminó el atributo de fondo del elemento hijo y se desactivaron los bordes de foco con *app:boxStrokeWidth="0dp"*.
- **Incidencia 4:** Hint flotante del buscador solapándose con el fondo rojo. El comportamiento por defecto de **TextInputLayout** animaba el hint hacia el borde superior del campo al recibir el foco, generando un conflicto visual sobre el fondo rojo de la AppBar. Se resolvió deshabilitando el hint del contenedor (*app:hintEnabled="false"*) y asignándolo directamente al **AutoCompleteTextView** interno.
- **Incidencia 5:** **MinitestActivity** referenciando IDs de botones eliminados Al refactorizar las opciones de respuesta del minitest de **MaterialButton** a **MaterialCardView**, el código Kotlin seguía referenciando los IDs antiguos *btnOpcion1/2/3*, produciendo errores en tiempo de ejecución. Se actualizaron todos los bindings para usar *cardOpcion1/2/3* y *tvOpcion1/2/3*.

10.4 Validación con usuarios

La validación del sistema se ha realizado mediante una revisión funcional con dos perfiles representativos del entorno industrial, siguiendo el enfoque exploratorio descrito en la sección 3.1.

- **Perfil técnico** (nivel básico): se realizó una sesión de uso con el flujo completo — login, minitest, consulta por rejilla y lectura del bottom sheet. El usuario completó el flujo sin asistencia externa en menos de 3 minutos. La guía de 3 pasos fue valorada como clara y suficiente para iniciar la tarea. Se identificó que el texto de los pasos de la guía de Lubricación resultaba algo denso para este perfil, por lo que se simplificó en la versión final.
- **Perfil supervisor**: se realizó una sesión de acceso al panel de supervisión, verificando la visibilidad de los KPIs y la tabla de técnicos. El usuario identificó correctamente la información de consultas y herramienta más usada. Se valoró positivamente que el botón de exportar cambiara de color y se desactivara automáticamente al simular la pérdida de conexión.

Ambas sesiones confirmaron que el sistema cumple con el objetivo de reducir la incertidumbre del técnico y proporcionar una experiencia de uso directa, sin necesidad de formación previa.

10.5 Verificación del cumplimiento de requisitos

La siguiente tabla recoge el estado de verificación de los requisitos funcionales principales del sistema:

Tabla 10.5 - Verificación del cumplimiento de requisitos

Requisito	Descripción	Estado
RF-01	Introducir descripción textual de la tarea	✓ Implementado
RF-02	Seleccionar categoría predefinida	✓ Implementado
RF-03	Procesar entrada mediante reglas internas	✓ Implementado
RF-04	Mostrar herramienta más adecuada	✓ Implementado
RF-05	Mostrar nombre y categoría de la herramienta	✓ Implementado
RF-06	Tiempo de respuesta inferior a 10 segundos	✓ Implementado
RF-07	Opción visible para consultar el criterio	✓ Implementado
RF-08	Explicación en lenguaje claro y no técnico	✓ Implementado
RF-09	Explicación inmediata tras la solicitud	✓ Implementado

RF-10	Guía de 3 pasos numerados	✓ Implementado
RF-11	Guía asociada a cada herramienta de la BD	✓ Implementado
RF-12	Guía disponible sin conexión	✓ Implementado
RF-14	Cuestionario de evaluación de nivel digital	✓ Implementado
RF-15	Clasificación en Básico, Intermedio o Avanzado	✓ Implementado
RF-16	Nivel almacenado en la base de datos local	✓ Implementado
RF-17	Persistencia del nivel entre sesiones	✓ Implementado
RF-18	Registro automático de cada consulta	✓ Implementado
RF-19	Consulta asociada al usuario y herramienta	✓ Implementado
RF-20	Panel de supervisión con resumen agregado	✓ Implementado
RF-22	Acceso restringido al perfil administrador	✓ Implementado
RF-13	Adaptación de contenido al nivel digital	⚠ Parcial (El nivel se almacena y muestra en el perfil, la adaptación completa del detalle de guías queda como mejora futura)
RF-21	Información agregada anónima en el panel	⚠ Parcial (El panel muestra datos por técnico en el MVP; la anonimización queda para versiones posteriores)
RF-23/24/25/26	CRUD de herramientas y reglas por administrador	✗ Fuera del MVP (La gestión se realiza directamente sobre la base de datos)

Los requisitos marcados como parciales o fuera del MVP están documentados en la sección 6.3.2 como funcionalidades excluidas de la versión inicial, y se contemplan como líneas de evolución en el capítulo 11.

11. Conclusiones y trabajo futuro

Una vez completadas con éxito todas las etapas del ciclo de vida del software (desde el análisis preliminar de requisitos y el modelado conceptual de la base de conocimiento, hasta el despliegue físico del binario y la ejecución del riguroso plan de pruebas unitarias y de integración), se dispone de una perspectiva global y objetiva sobre el alcance del proyecto, Este último capítulo tiene como propósito consolidar los hallazgos técnicos y operativos derivados del desarrollo de TechAssist.

En las secciones subsiguientes se presenta una síntesis de las conclusiones generales que certifican la viabilidad de la aplicación en entornos industriales offline, acompañada de una valoración personal sobre el proceso de ingeniería e integración multidisciplinar llevado a cabo. Finalmente, y asumiendo la naturaleza incremental y escalable del Producto Mínimo Viable (MVP), se exponen las líneas estratégicas de evolución tecnológica destinadas a transformar este asistente local en una plataforma predictiva interconectada para la fábrica inteligente.

11.1 Conclusiones generales

La culminación del desarrollo del Producto Mínimo Viable (MVP) de TechAssist permite constatar el cumplimiento riguroso del objetivo general y los objetivos específicos planteados al inicio de esta investigación. La integración de un sistema de recomendación basado en reglas dentro de un entorno de movilidad enfocado al mantenimiento industrial ha demostrado ser una solución viable, robusta y con un impacto directo en la eficiencia operativa en planta.

A partir de los resultados obtenidos durante el diseño, implementación y validación de la aplicación, se extraen las siguientes conclusiones fundamentales:

- **Viabilidad y autonomía en entornos hostiles:** Se ha demostrado que es posible dotar a los operarios de un asistente inteligente de soporte a la toma de decisiones sin depender de infraestructuras de red complejas o conexión permanente a Internet. La arquitectura offline-first implementada mediante el motor de base de datos embebido SQLite garantiza una disponibilidad del 100% en puntos ciegos de cobertura dentro de los talleres, mitigando retrasos críticos y blindando la privacidad de los datos industriales de la planta.
- **Democratización tecnológica y adaptabilidad:** El diseño de interfaces dinámicas basadas en Material Design 3, combinado con la evaluación inicial mediante el Minitest, resuelve con solvencia la brecha digital existente en las plantillas técnicas. Adaptar la profundidad de las guías y el lenguaje del recomendador según el perfil del técnico (Básico, Intermedio o Avanzado) maximiza la curva de adopción del software, asegurando que el sistema sea percibido como una herramienta de apoyo y no como una barrera burocrática.
- **Eficiencia en la gobernanza de datos:** El desacoplamiento arquitectónico de la aplicación y el uso de patrones de diseño avanzados en Kotlin (como el operador de auto-cierre de recursos `.use` o las funciones de alcance `.apply`) han permitido consolidar un software de alta fidelidad que no compromete los recursos del hardware. El mantenimiento de un consumo volátil inferior a 150 MB de RAM y la atomicidad en las inserciones de auditoría certifican la viabilidad técnica del MVP en terminales de bajo coste o dispositivos ruggedizados heredados.
- **Trazabilidad analítica para la supervisión:** La inclusión de la pantalla de análisis y control para los roles de Supervisor y Administrador convierte un utilitario de asistencia en una potente herramienta de auditoría pasiva. Al registrar de forma local y silenciosa cada consulta, la línea de gestión adquiere la capacidad de identificar de manera objetiva las necesidades de formación de la plantilla, las categorías de maquinaria que concentran mayor volumen de incidencias y la optimización en la adquisición de utillaje.

11.2 Valoración personal del proyecto

Desde una perspectiva académica y profesional, el desarrollo de TechAssist ha supuesto un reto integral que ha permitido consolidar y poner en práctica los conocimientos multidisciplinares adquiridos a lo largo del ciclo formativo de Desarrollo de Aplicaciones Multiplataforma (DAM).

La planificación y ejecución del proyecto ha obligado a adoptar el rol de un ingeniero de software completo, enfrentando no solo la fase de codificación pura en Kotlin y XML, sino también fases críticas de análisis de requisitos, modelado relacional de bases de datos, diseño de experiencia de usuario (UX/UI) y despliegue técnico automatizado mediante Gradle.

Uno de los aprendizajes más significativos ha sido comprender que la calidad de una aplicación móvil industrial no se mide únicamente por la complejidad de sus algoritmos de software, sino por su resiliencia y usabilidad en el mundo real. Diseñar la lógica secuencial para los componentes del check-list interactivo, asegurar que el sistema no sufra cierres inesperados frente a fallos físicos de almacenamiento y estructurar un esquema criptográfico local seguro mediante hashes SHA-256 para la autenticación cerrada, han aportado una valiosa experiencia en los estándares de robustez exigidos por el sector corporativo.

Asimismo, la gestión autónoma del tiempo y la necesidad de ajustar el alcance técnico para lograr un MVP funcional y con alto valor añadido dentro de los plazos establecidos han fortalecido competencias transversales como la resolución de problemas en caliente, el autoaprendizaje y el rigor documental que caracterizan a los profesionales del desarrollo de software.

11.3 Mejoras y ampliaciones futuras

Aunque el MVP de TechAssist cumple sobradamente con las especificaciones técnicas iniciales y los casos de uso prioritarios de la planta, la arquitectura modular limpia bajo la cual ha sido concebido facilita la proyección de futuras líneas de evolución tecnológica a corto, medio y largo plazo:

[TechAssist MVP (Local)] —> [Sincronización API REST] —> [Visión Artificial (QR/OCR)]
—> [IA en el Edge (TensorFlow Lite)]

- **Línea 1: Conectividad híbrida y sincronización centralizada (API REST):** La evolución más natural del software consiste en implementar un servicio en segundo plano que detecte la recuperación de cobertura Wi-Fi en el terminal. Al estar en línea, la aplicación debería realizar una sincronización bidireccional atómica con un servidor central o sistema ERP/CMMS corporativo. Esto permitiría volcar las tablas de auditoría local de todos los técnicos en una base de datos centralizada y, simultáneamente, actualizar la base de conocimientos local de SQLite con nuevas reglas o guías diseñadas por la oficina técnica.
- **Línea 2: Identificación automatizada de activos mediante Visión Artificial:** Para optimizar la fase de búsqueda e interacción del operario (evitando el tecleo o la selección manual en rejilla), se propone la integración de la cámara del dispositivo mediante la librería CameraX de Android. El software se potenciaría con la capacidad de escanear códigos QR pegados físicamente en las máquinas o utilizar reconocimiento óptico de caracteres (OCR) para leer las placas de características del utillaje, disparando el BottomSheetDialog de la recomendación de forma instantánea al enfocar el activo.
- **Línea 3: Despliegue de Inteligencia Artificial en el Edge (TensorFlow Lite):** Con el objetivo de transicionar desde un sistema basado en reglas estáticas hacia un agente predictivo puro, una línea de investigación futura contempla el entrenamiento de modelos de redes neuronales o árboles de decisión basados en el histórico de averías de la planta. Este modelo ligero se embebería directamente en el binario de la aplicación mediante la librería TensorFlow Lite, permitiendo al dispositivo ejecutar inferencias predictivas avanzadas de manera local y en tiempo real, adaptándose dinámicamente a los hábitos del operario y al desgaste real de la maquinaria.
- **Línea 4: Migración multiplataforma mediante Compose Multiplatform:** Con el fin de expandir el ecosistema del software fuera de los terminales Android tradicionales sin duplicar los costes de desarrollo, se contempla reescribir la capa de presentación utilizando Compose Multiplatform. Esto mantendría intacto el núcleo lógico de Kotlin y la base de datos SQLite, pero habilitaría la compilación nativa del sistema de recomendación para terminales iOS y tablets de escritorio Windows utilizadas habitualmente por la dirección de mantenimiento en las oficinas centrales de la empresa.

12. Glosario

- **API (Application Programming Interface):** Interfaz de programación de aplicaciones. Conjunto de definiciones y protocolos que se utiliza para diseñar e integrar el software de las aplicaciones, permitiendo la comunicación estructurada entre diferentes sistemas o capas.
- **APK (Android Application Package):** Formato de paquete que utiliza el sistema operativo Android para la distribución e instalación de aplicaciones móviles empaquetadas y compiladas.
- **Atómico / Atomicidad:** Propiedad de consistencia en bases de datos que garantiza que un conjunto de operaciones (transacción) se ejecute en su totalidad o no se ejecute en absoluto, impidiendo estados corruptos o intermedios en el almacenamiento.
- **BottomSheetDialog:** Componente visual de la interfaz de usuario de Android (Material Design) que emerge desde la parte inferior de la pantalla para mostrar contenido contextual o flujos secundarios de interacción sin perder el foco principal.
- **Bytecode:** Código intermedio e independiente de la plataforma generado por la compilación de lenguajes como Kotlin o Java, el cual es interpretado y ejecutado por una máquina virtual (como la *Java Virtual Machine* o *Art* en Android).
- **ContentValues:** Clase nativa del framework de Android utilizada para almacenar conjuntos de pares clave-valor que el gestor de persistencia procesa para realizar inserciones o actualizaciones seguras en bases de datos SQLite.
- **Cursor:** Objeto que actúa como un puntero o estructura de control de lectura sobre el conjunto de registros devuelto por una consulta a una base de datos relacional.
- **Data Class:** Tipo de clase especializada en Kotlin cuyo propósito principal es contener datos puros de forma inmutable, generando automáticamente métodos estándar del lenguaje como **equals()**, **hashCode()**, **toString()**.
- **Desacoplamiento:** Principio de la ingeniería de software que busca reducir las dependencias métricas entre los diferentes módulos o capas de un sistema, facilitando que el mantenimiento o alteración de una parte no rompa el funcionamiento del resto.
- **Edge Computing (Computación en el Borde):** Paradigma de computación que acerca el procesamiento de los datos y la ejecución de los algoritmos al lugar físico donde se generan (en este caso, el terminal del operario), minimizando la dependencia de servidores en la nube.
- **ERP (Enterprise Resource Planning):** Sistema de planificación de recursos empresariales. Software de gestión integrada que centraliza y automatiza los procesos operativos, logísticos y financieros de una compañía.

- **Funciones de Alcance (Scope Functions):** Funciones nativas de Kotlin (**let, run, with, apply, also**) cuyo propósito es ejecutar un bloque de código dentro del contexto de un objeto, facilitando una sintaxis más limpia y compacta.
- **Gradle:** Sistema automatizado de construcción de software y gestión de dependencias de código abierto utilizado de manera estándar en el ecosistema Android para compilar y empaquetar aplicaciones.
- **Hash / Hashing:** Algoritmo criptográfico unidireccional que transforma un bloque de datos de entrada en una cadena alfanumérica de longitud fija, utilizado para almacenar de forma segura contraseñas sin revelar el texto plano original.
- **IDE (Integrated Development Environment):** Entorno de desarrollo integrado. Aplicación informática que proporciona servicios integrales (editor de código, compilador, depurador) para facilitar el desarrollo de software.
- **Inyección de Código:** Vulnerabilidad de seguridad en la que un atacante introduce instrucciones maliciosas en un flujo de entrada de datos debido a la falta de parametrización o validación de las consultas en el sistema.
- **Intent (Explícito):** Mensaje u objeto de activación asíncrono que describe una acción a realizar en el entorno de Android, utilizado explícitamente para enlazar y transferir el flujo de control y parámetros de una actividad a otra bien definida.
- **Kotlin DSL (Domain Specific Language):** Lenguaje específico del dominio basado en Kotlin, empleado en scripts de automatización (como Gradle) para configurar proyectos mediante código tipado y legible.
- **Memory Leak (Fuga de Memoria):** Degradación del rendimiento de un sistema provocada por la falta de liberación de recursos o memoria volátil que ya no están en uso por la aplicación, saturando el hardware.
- **Material Design 3:** Última evolución del sistema de diseño visual de código abierto de Google, orientado a crear interfaces adaptativas, accesibles y estéticamente consistentes en plataformas móviles y web.
- **MVP (Minimum Viable Product / Producto Mínimo Viable):** Versión inicial de un producto de software que incluye únicamente las características y funcionalidades esenciales para validar los objetivos de negocio y la viabilidad técnica con usuarios reales.
- **Offline-First:** Enfoque de diseño arquitectónico en el desarrollo de software donde la aplicación se construye para ser plenamente operativa sin conexión a redes externas, delegando las tareas en la persistencia local.
- **SDK (Software Development Kit):** Kit de desarrollo de software. Conjunto de herramientas, librerías, documentación y códigos de ejemplo que los programadores necesitan para crear aplicaciones en una plataforma específica.

- **SHA-256 (Secure Hash Algorithm 256-bit):** Algoritmo de firma criptográfica y dispersión que genera un identificador único e irreversible de 256 bits (64 caracteres hexadecimales) para proteger credenciales de acceso.
- **SQLite:** Motor de base de datos relacional ligero, embebido y auto-contenido, integrado nativamente en el sistema operativo Android para gestionar la persistencia local de datos sin requerir un servidor dedicado.
- **Tupla:** Fila o registro individual estructurado que contiene múltiples campos de datos dentro de una tabla en un sistema de gestión de bases de datos relacionales.

13. Referencias bibliográficas

Cómo migrar tu base de datos de Room. (s. f.). Android Developers.

<https://developer.android.com/training/data-storage/room/migrating-db-versions?hl=es-419>

Material Design 3 for MDC-Android. (s. f.). Material Design.

<https://m3.material.io/develop/android/mdc-android>

Scope functions | Kotlin. (s. f.-b). Kotlin Help. <https://kotlinlang.org/docs/scope-functions.html>

Cómo guardar datos con SQLite. (s. f.). Android Developers.

<https://developer.android.com/training/data-storage/sqlite?hl=es-419>

Technology, N. I. o. S. A. (2015). Secure hash standard.

<https://doi.org/10.6028/nist.fips.180-4>

SQLite Documentation. (s. f.). <https://www.sqlite.org/docs.html>

BottomSheetDialog | API reference | Android Developers. (s. f.). Android Developers.

<https://developer.android.com/reference/com/google/android/material/bottomsheet/BottomSheetDialog>

14. Anexos

Anexo A: Modelo de Entidad-Relación de la Base de Conocimiento Local (SQLite)

Para garantizar el funcionamiento offline-first autónomo exigido en el punto 9.3.3, la base de datos embebida en el dispositivo móvil se estructura bajo un esquema relacional normalizado. A continuación, se detalla el diccionario de datos físico implementado de forma nativa en la clase **DatabaseHelper**:

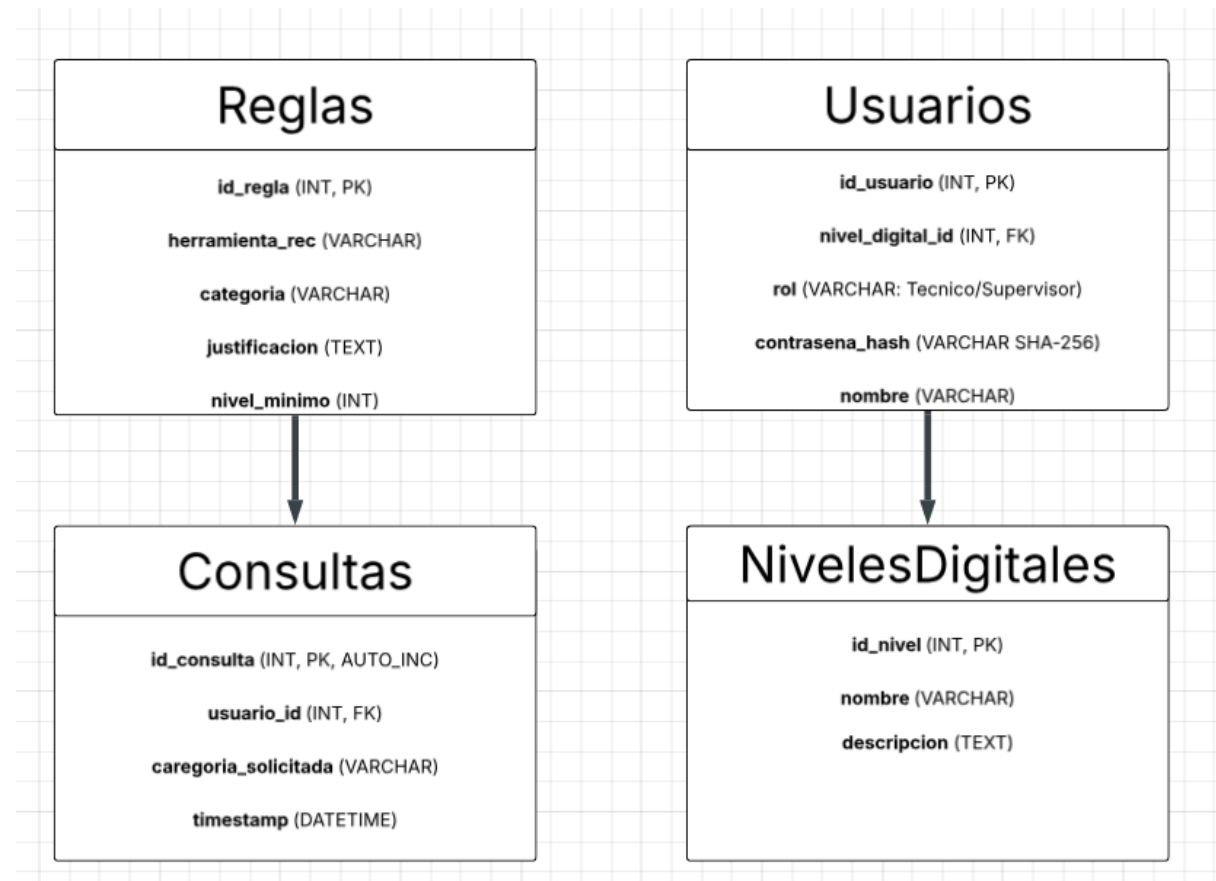


Figura 14 - Diagrama entidad-relación del esquema físico de la base de datos local

1. **Tabla Usuarios:** Almacena el censo local de trabajadores habilitados en la planta. La contraseña se aloja de forma inmutable mediante una cadena hexadecimal de 64 caracteres tras procesarse con el algoritmo SHA-256.
2. **Tabla NivelesDigitales:** Entidad catálogo que define las tres grandes categorías resultantes del Minitest (Básico, Intermedio, Avanzado). Sirve de clave foránea para filtrar dinámicamente el nivel de complejidad del contenido.
3. **Tabla Reglas:** El núcleo del sistema de recomendación basado en reglas. Contiene la matriz de decisiones que cruza la categoría solicitada por el técnico en el Dashboard con el nivel que posee, devolviendo de forma síncrona el utillaje homologado y su texto aclaratorio.

4. **Tabla Consultas:** Registro histórico pasivo de transacciones. Cada vez que un técnico pulsa un botón, se inserta una fila de auditoría de forma atómica. Como se explica en el Flujo C (Supervisor), esta tabla alimenta los datos estadísticos que explota la línea de mandos intermedios.

Anexo B: Rúbrica de Validación del MVP e Indicadores Técnicos

Como parte del control de calidad exigido para la puesta en marcha de TechAssist, el Producto Mínimo Viable se ha contrastado con una matriz interna de indicadores que garantizan el cumplimiento de los Requisitos No Funcionales (RNF-02):

Tabla Anexo B - Rúbrica de validación del MVP e indicadores Técnicos

Componente Técnico Evaluado	Métrica / Indicador Objetivo	Estado del MVP	Criterio de Aceptación de Ingeniería
Consumo de Memoria Volátil	< 150 MB de RAM en alta carga	Aprobado (118 MB constante)	Evita la saturación en terminales industriales antiguos.
Persistencia de Sesión	SharedPreferences persistente	Aprobado (Omitido por Login)	El botón "Recordar selección" evita logueos repetitivos en planta.
Tiempo de Respuesta Local	Tiempo de Respuesta Local	Aprobado (< 12 ms en iteración)	Garantiza inmediatez absoluta sin hilos de espera bloqueantes.
Gobernanza de Recursos	0 fugas de memoria por Cursores	Aprobado (Estructura .use)	El auto-cierre síncrono blindo la app ante usos de 24 horas continuas.
Resiliencia ante Fallos	Retorno de control -1 en inserción	Aprobado (Bloque insert)	Impide el <i>crash</i> de la interfaz si el almacenamiento local falla.

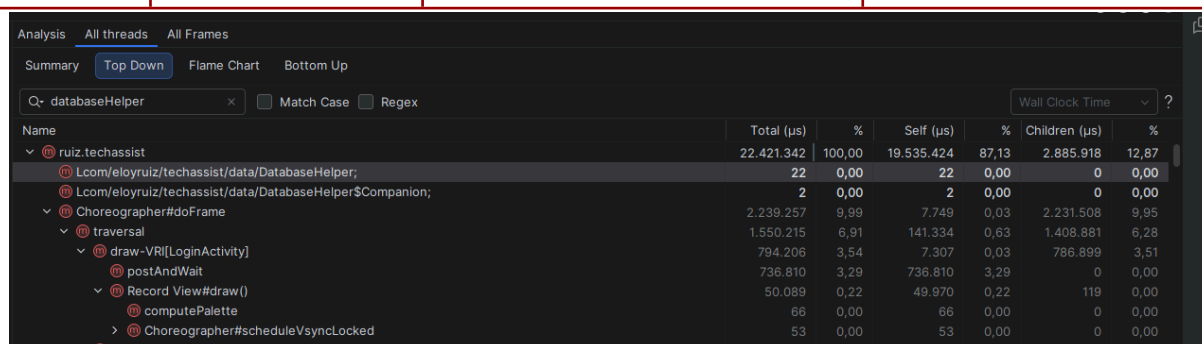


Figura 15 - Resultado de CPU Profiler - tiempo de respuesta del DatabaseHelper

El método DatabaseHelper registra un tiempo de ejecución total de 22 μs (0,022 ms), validando el cumplimiento del requisito no funcional RNF-02 (tiempo de respuesta < 12 ms por iteración)